

A PRACTITIONER'S TEXTBOOK

Trust at Scale

Applied Cryptography from First Principles to the Post-Quantum Era

Cryptography

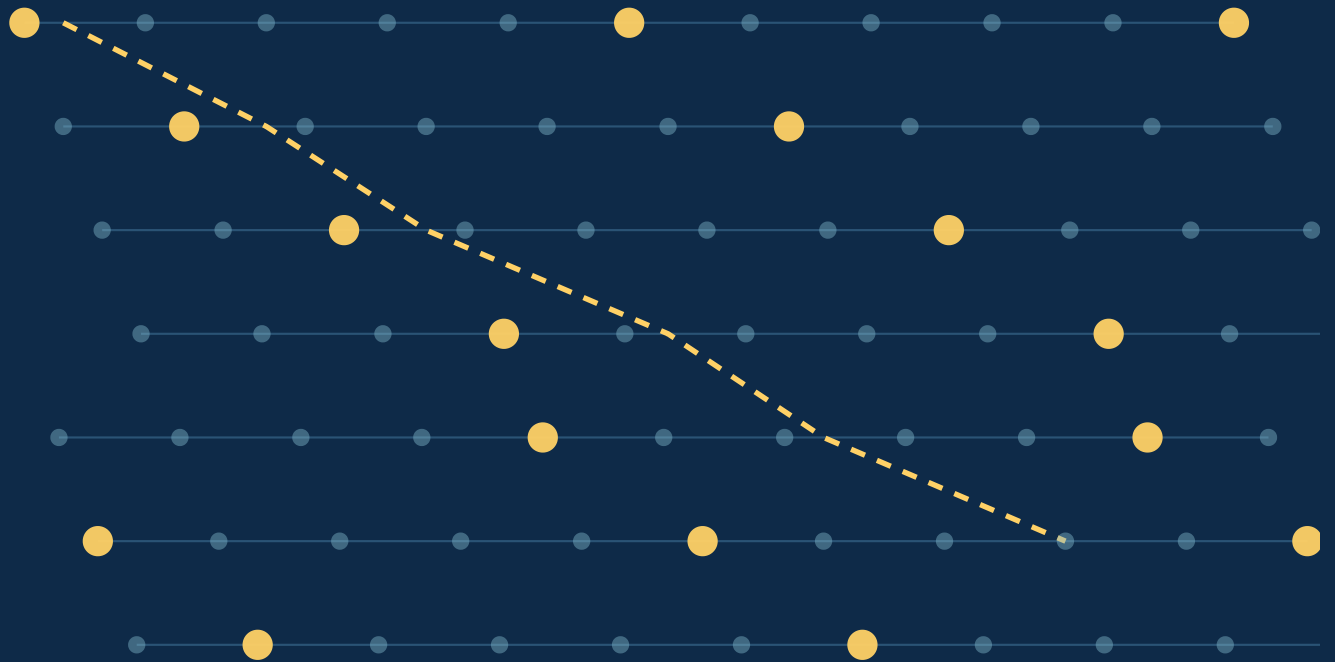
PKI

Key management

Secrets

HSMs

Post-quantum



Bolaji Akinyele

First Edition · 2026

Keys · Certificates · Secrets · Post-Quantum

Trust at Scale

Applied Cryptography from First Principles to the Post-Quantum Era



Bolaji Akinyele

First Edition · 2026

Trust at Scale

Applied Cryptography from First Principles to the Post-Quantum Era

Copyright © 2026 Bolaji Akinyele. All rights reserved.

This book describes general industry practice and publicly documented standards, incidents and research. It does not describe the systems, configuration or practices of any specific organisation. Product and company names mentioned for illustration belong to their respective owners, and their mention does not imply endorsement.

Standards, browser policies and regulatory timelines change frequently. Facts are current to October 2026; always confirm dates and requirements against the primary source before relying on them.

First Edition · 2026

Preface



Cryptography quietly holds the digital world together. It protects every login, card payment, software update, private message and backup. Yet most engineers learn it in fragments: a TLS setting here, a certificate renewal there, an audit question about key storage that nobody can quite answer. The fragments rarely connect, and the gaps between them are where outages and breaches happen.

This book connects the fragments. It is written for engineers, architects and security professionals who want to understand cryptography deeply enough to make good decisions, explain them clearly, and run cryptographic systems safely at scale.

Who this book is for

You do not need a mathematics degree. You do need curiosity and a willingness to work through examples. If you can read a little code and use a command line, you can complete every lab. Readers who already work with certificates, keys or secrets will find the later parts immediately useful, and the early parts will fill gaps they may not know they have.

How the book is organised

The book follows the order in which the ideas depend on each other.

- **Part I, Foundations**, explains what cryptography is for, the mathematics behind it, and the randomness it depends on.
- **Part II, Symmetric Cryptography**, covers block ciphers, authenticated encryption, hashing, MACs and key derivation.
- **Part III, Public-Key Cryptography**, covers RSA, Diffie-Hellman, elliptic curves, digital signatures and how security is proved.
- **Part IV, Protocols**, shows how these pieces become TLS, SSH, VPNs, secure messaging and identity systems.
- **Part V, Certificates and PKI**, explains certificates, the web's trust system, revocation, private CAs and lifecycle automation.
- **Part VI, Keys, Secrets and Hardware**, covers key management, key management services, secrets, HSMs and system architecture.
- **Part VII, The Post-Quantum Transition**, explains the quantum threat, the new standards, and how to migrate.
- **Part VIII, Frontiers and Reference**, surveys advanced techniques and provides a glossary and reading guide.

How each chapter works

Every chapter opens with what you will be able to do by the end, and every term is defined in plain words where it first appears. Throughout the text you will find three kinds of highlighted box:

Box	What it contains
In practice	How the idea shows up in real systems, and what to check in your own environment
Current development	Recent changes to standards, browser policies or regulations, current to October 2026
Watch out	Common mistakes that cause outages or breaches

Most chapters end with a hands-on lab using free tools, a failure story drawn from a real, publicly documented incident, a summary, and review questions. Do the labs.

Cryptography becomes intuitive only when you have broken a toy system with your own hands.

A note on currency

Cryptographic practice is changing faster now than at any time in the last twenty years. Public certificate lifetimes are shrinking toward 47 days. Browsers already use post-quantum key exchange for most connections. Governments have published deadlines for retiring RSA and elliptic-curve cryptography. Where this book cites a date, policy or statistic, it reflects the public record as of October 2026. Always check the primary source before relying on a date for a decision.

Acknowledgements

This book stands on the work of the researchers, standards bodies and open-source maintainers whose publications, specifications and incident reports are cited throughout. Any errors are mine.

Bolaji Akinyele

CONTENTS

Contents

PART I · FOUNDATIONS	8
1 How to Think About Cryptography	9
2 The Mathematics You Actually Need	17
3 Randomness, Probability and Hardness	26
PART II · SYMMETRIC CRYPTOGRAPHY	34
4 Block Ciphers and AES	35
5 Modes of Operation and Authenticated Encryption	43
6 Hash Functions, MACs and Key Derivation	52
PART III · PUBLIC-KEY CRYPTOGRAPHY	61
7 RSA	62
8 Diffie-Hellman and Elliptic Curves	70
9 Digital Signatures	78
10 Security Definitions and Proofs, Made Practical	86
PART IV · PROTOCOLS	93
11 Building a Secure Channel with TLS 1.3	94
12 Deploying TLS Well	102
13 Beyond TLS: SSH, VPNs and Secure Messaging	110
14 Identity Protocols: OAuth, OpenID Connect, JWT and Passkeys	118
PART V · CERTIFICATES AND PKI	126
15 X.509 Certificates Explained	127
16 The Web PKI: Certificate Authorities, Root Programs and Transparency	135
17 Path Validation and Revocation	143
18 Building and Running a Private CA	151
19 Certificate Lifecycle Automation at Scale	159

PART VI · KEYS, SECRETS AND HARDWARE		169
20	Key Management Fundamentals	170
21	Key Management Services, BYOK and HYOK	179
22	Secrets Management and Workload Identity	186
23	HSMs, TPMs and Confidential Computing	193
24	Cryptographic Architecture, Agility and Governance	201
PART VII · THE POST-QUANTUM TRANSITION		209
25	The Quantum Threat	210
26	The New Algorithms: ML-KEM, ML-DSA, SLH-DSA and Beyond	218
27	Migrating to Post-Quantum Cryptography	227
PART VIII · FRONTIERS AND REFERENCE		236
28	Frontiers: Zero Knowledge, MPC, FHE and What Comes Next	237
A	Glossary	243
B	Standards, Further Reading and Practice	251
C	Command Reference	254

PART I

Foundations

The ideas every later chapter depends on: what cryptography is for, the mathematics behind it, and the randomness and hardness that make it work.

In this part

- 1 How to Think About Cryptography
- 2 The Mathematics You Actually Need
- 3 Randomness, Probability and Hardness

How to Think About Cryptography

Cryptography is not magic dust you sprinkle on a system to make it secure. It is a small set of precise tools, each of which solves one specific problem under specific assumptions. This chapter gives you the mental model the rest of the book builds on: what cryptography can do, what it cannot do, and how professionals reason about it.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Name the four security goals cryptography provides and match each to a tool
- Explain Kerckhoffs's principle and why secret algorithms fail
- Describe a threat model and use it to judge a design
- Recognise the most common ways real systems fail despite strong cryptography

IN THIS CHAPTER

- | | |
|-----|--|
| 1.1 | What cryptography is for |
| 1.2 | Keys are the whole game |
| 1.3 | Symmetric and asymmetric: two families |
| 1.4 | Threat models: secure against whom? |
| 1.5 | How strong is strong enough? Security levels |
| 1.6 | What cryptography cannot do |
| 1.7 | The golden rules of applied cryptography |

1.1 What cryptography is for

Every system that handles information has to answer a few basic questions. Who can read this? Has anyone changed it? Who sent it? Can the sender later deny sending it?

Cryptography gives mathematical answers to these questions, so that the answers hold even when the information travels over networks you do not control and sits on disks you cannot watch.

The answers are grouped into four **security goals**. Learn these four words now, because every tool in this book serves one or more of them.

Goal	The question it answers	Everyday example	Main tools
Confidentiality	Can only the intended people read it?	Nobody on the café Wi-Fi can read your banking session	Encryption (AES, ChaCha20)
Integrity	Has it been changed, accidentally or on purpose?	A downloaded file is exactly what the publisher released	Hash functions, MACs
Authentication	Is it really from who it claims to be from?	Your browser knows it is talking to your bank, not an impostor	MACs, digital signatures, certificates
Non-repudiation	Can the sender later deny it?	A signed contract or a signed software release	Digital signatures

Two points about this table trip people up.

First, **confidentiality does not give you integrity**. Encryption hides content, but on its own it does not stop an attacker from changing the encrypted bytes. With some older encryption methods, an attacker who flips a bit in the encrypted data flips the same bit in the decrypted result, without ever knowing the key. Modern designs combine encryption with integrity protection for exactly this reason (Chapter 5).

Second, **authentication is about the origin of data, not about people logging in**. When cryptographers say "authenticated", they mean "we can prove this came from the holder of a particular key". Whether that key belongs to the right person is a separate problem, solved by certificates and identity systems (Parts IV and V).

1.2 Keys are the whole game

Almost every cryptographic tool works the same way: a public, well-known **algorithm** plus a secret **key**. The algorithm is like the design of a lock, which anyone can study. The key is like the physical key, which only the owner holds.

This leads to the single most important principle in the field, stated by the Dutch cryptographer Auguste Kerckhoffs in 1883:

Kerckhoffs's principle. *A system should remain secure even if everything about it, except the key, is public knowledge.*

The principle sounds obvious, yet people break it constantly. They invent a private encryption scheme and assume it is safe because nobody knows how it works. History shows that secret designs get reverse-engineered, leaked or guessed, and then every copy of the system is broken at once, with no way to fix it except replacing the design everywhere. A public algorithm that has survived years of attack by the world's experts is far safer, and if a key leaks, you replace one key instead of a whole design.

The consequence for your work is simple and permanent: **protecting keys is the job**. The mathematics of a well-chosen algorithm is rarely what fails. Keys are generated badly, stored carelessly, copied too widely, never rotated, or stolen. Most of this book after Part III is about keys: how to create them, where to keep them, who may use them, how to replace them, and how to prove all of that to an auditor.

1.3 Symmetric and asymmetric: two families

Cryptographic tools come in two families, and you need to know which family a tool belongs to.

Symmetric cryptography uses one shared secret key. The same key encrypts and decrypts, or creates and checks an integrity tag. It is fast, which makes it the workhorse for encrypting data in bulk: every web page, every file on an encrypted disk, every database backup. Its weakness is distribution: both sides need the same secret, and getting it to them safely is a problem.

Asymmetric cryptography, also called **public-key cryptography**, uses a pair of linked keys. The **public key** can be shared with anyone. The **private key** is kept secret by one owner. What one key does, only the other can undo or check. It solves the distribution problem: you can publish your public key on a website, and anyone can use it to send you something only you can read, or to check something only you could have signed. Its weakness is speed: it is hundreds to thousands of times slower than symmetric cryptography.

Real systems use both, each for what it is good at. This pattern, called **hybrid encryption**, appears everywhere:

1. Use asymmetric cryptography to agree on, or deliver, a fresh symmetric key.
2. Use that symmetric key to protect the actual data.

A TLS connection to a website does exactly this. A few milliseconds of public-key work at the start produce a shared session key, and everything after that is fast symmetric encryption.

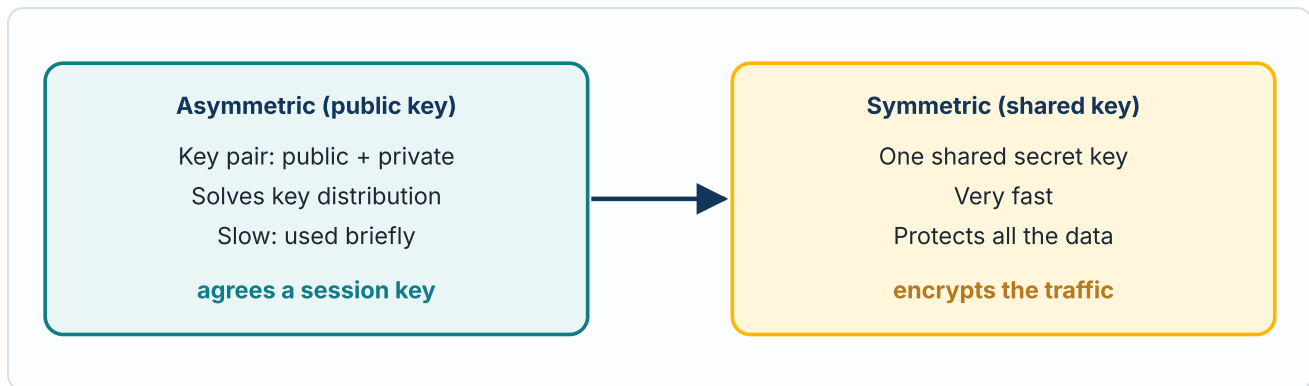


Figure 1.1 Hybrid encryption: public-key cryptography sets up a shared key, then symmetric cryptography does the heavy lifting.

1.4 Threat models: secure against whom?

"Is this secure?" has no answer on its own. The useful question is "secure against whom, doing what, for how long?" Writing that down is called a **threat model**. A short threat model has four parts:

1. **Assets.** What are you protecting? Customer records, a signing key, a payment flow.
2. **Adversaries.** Who might attack it, and what can they do? A curious employee, a criminal group, a nation-state with a large budget.
3. **Capabilities.** What can the attacker see or touch? Network traffic only? A stolen laptop? A copy of your backups? Physical access to a server?
4. **Time.** How long must the protection last? A one-time login code needs seconds. Medical records need decades.

The last point matters more every year. An attacker can record encrypted traffic today and keep it until a future technology, such as a large quantum computer, can break the encryption. This is called **harvest now, decrypt later**. For data that must stay secret for 10 or more years, the threat model already includes quantum attackers (Part VII).

Cryptographers also describe attackers by what they can do to the cryptography itself. You will meet these terms in later chapters:

Attack model	What the attacker can do	Why it matters
Ciphertext-only	Sees encrypted messages	The weakest attacker; any modern cipher must resist far more
Known-plaintext	Sees some messages and their encryptions	Common in reality: file headers and protocol messages are predictable
Chosen-plaintext	Can get messages of their choice encrypted	Realistic on the web, where attackers can trigger requests
Chosen-ciphertext	Can submit altered ciphertexts and learn something from the reaction	Behind many real breaks, such as padding oracle attacks (Chapter 5)

Modern algorithms are designed to resist the strongest of these attackers. When a real system is broken, it is usually because the surrounding software leaks extra information, turning a strong algorithm into a weak system.

1.5 How strong is strong enough? Security levels

Cryptographic strength is measured in **bits of security**. An algorithm with 128-bit security needs roughly 2^{128} operations to break by the best known attack. That number is so large that if every computer on Earth worked on it until the Sun burned out, it would still not be finished.

Different algorithms reach the same security level with very different key sizes, which confuses many newcomers:

Security level	Symmetric key	Hash output	RSA key	Elliptic curve key
112 bits (minimum today)	112	224	2048	224
128 bits (standard)	128	256	3072	256
192 bits	192	384	7680	384
256 bits	256	512	15360	512

The values follow NIST guidance (SP 800-57). The practical lesson is that a 256-bit elliptic curve key is about as strong as a 3072-bit RSA key, and a 128-bit AES key is stronger than a 2048-bit RSA key. Comparing raw key lengths across algorithm families is meaningless.

CURRENT DEVELOPMENT NIST's draft transition plan (IR 8547, published for comment in November 2024) proposes deprecating RSA and elliptic-curve algorithms at the 112-bit level after 2030 and disallowing them entirely after 2035, because quantum computers would break them regardless of key size. Symmetric algorithms survive with larger keys. Part VII explains why.

1.6 What cryptography cannot do

Cryptography is powerful but narrow. Knowing its limits will save you from expensive mistakes.

- **It cannot protect data while it is being used.** Data must be decrypted to be processed. If an attacker controls the server that decrypts it, encryption does not help. (Confidential computing, Chapter 23, narrows this gap.)
- **It cannot fix bad key handling.** A key stored next to the data it protects, or printed in a log file, gives no protection.
- **It cannot tell you who someone really is.** It proves that a message came from a key holder. Linking that key to a real person or server is the job of certificates and identity processes.
- **It cannot stop authorised people from misusing access.** An administrator who can decrypt a database can copy it.
- **It cannot survive broken randomness.** Keys made from predictable random numbers are guessable, however strong the algorithm (Chapter 3).

A useful habit: when someone says "it's encrypted, so it's safe", ask three questions. Encrypted with what? Where is the key? Who can use the key?

1.7 The golden rules of applied cryptography

Experienced practitioners follow a short list of rules. You will see each one justified in later chapters.

1. **Do not invent your own algorithms or protocols.** Use well-studied standards.
2. **Do not implement primitives yourself in production.** Use mature libraries such as OpenSSL, BoringSSL, libsodium, or your platform's built-in cryptography. Tiny implementation details, such as how long a comparison takes, can leak keys.
3. **Prefer high-level, hard-to-misuse interfaces.** A function called `encrypt(key, message)` that handles nonces and integrity for you is safer than assembling AES, a

mode and a MAC by hand.

4. **Always authenticate what you encrypt.** Use authenticated encryption (Chapter 5).
5. **Treat keys as the most valuable asset.** Generate them with good randomness, store them in protected places, give them owners and lifetimes, and plan their replacement from day one.
6. **Design for change.** Algorithms age. Build systems that can switch algorithms and keys without a rewrite. This property, **crypto agility**, is now a regulatory expectation because of the post-quantum transition.

IN PRACTICE

- When you review a design, write its threat model in five lines before discussing algorithms.
- Ask where every key lives, who can use it, and how it would be replaced.
- Be suspicious of anything described as "proprietary encryption" or "military-grade" without naming a standard.

CASE STUDY

Failure story: the debugging change that broke a million keys

In 2006, a maintainer of the Debian Linux distribution removed two lines from OpenSSL's random number generator. A code analysis tool had warned that the lines used uninitialised memory, and removing them made the warning go away. Unfortunately, those lines were how the generator collected most of its unpredictability. After the change, the only varying input was the process ID, a number usually below 32,768.

For almost two years, every SSH key, TLS key and other key generated on Debian and Ubuntu systems came from a pool of only about 32,000 possibilities per key type and size. An attacker could simply generate all of them and try each one. When the flaw was found in May 2008, administrators worldwide had to replace keys, revoke certificates and re-issue credentials.

The algorithms were never weak. RSA and DSA were fine. The failure was in the surrounding engineering: a well-meant change to code the maintainer did not fully understand, in the most sensitive part of the system, without review by someone who did. Three of this chapter's rules apply directly: do not modify cryptographic code casually, randomness is a single point of failure, and plan for the day every key must be replaced.

KEY TAKEAWAYS

Chapter summary

- Cryptography provides four goals: confidentiality, integrity, authentication and non-repudiation. Encryption alone provides only the first.
- Security rests on keys, not secret designs (Kerckhoffs's principle). Protecting and managing keys is the real work.
- Symmetric cryptography is fast but needs a shared secret; asymmetric cryptography solves distribution but is slow. Hybrid designs use both.
- A threat model names assets, adversaries, capabilities and time. Long-lived secrets must already consider quantum attackers.
- Strength is measured in bits of security, and key sizes are not comparable across algorithm families.
- Cryptography cannot fix bad key handling, bad randomness or compromised endpoints.

CHECK YOURSELF

Review questions

1. Give an example where data is confidential but not protected for integrity, and explain why that is dangerous.
2. State Kerckhoffs's principle in your own words. Why is a public algorithm safer than a secret one?
3. Why do real systems combine symmetric and asymmetric cryptography?
4. Write a five-line threat model for a password reset email.
5. Which is stronger: a 2048-bit RSA key or a 128-bit AES key? Why can you not compare them by length?
6. What does "harvest now, decrypt later" mean, and which kinds of data does it threaten?
7. In the Debian incident, which golden rule was broken, and what single process change would most likely have caught it?

The Mathematics You Actually Need

You do not need a mathematics degree to master applied cryptography, but you do need a handful of ideas: arithmetic that wraps around, groups, a few hard problems, and a first look at elliptic curves and lattices. This chapter explains each one with small numbers you can check by hand, and shows exactly where each idea is used later in the book.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Calculate with modular arithmetic and explain why it suits cryptography
- Describe groups, generators and finite fields in plain terms
- Explain the factoring and discrete logarithm problems and why they are believed hard
- Describe elliptic curves and lattices at the level needed for later chapters

IN THIS CHAPTER

- | | |
|-----|--|
| 2.1 | Arithmetic that wraps around |
| 2.2 | Primes and why factoring is hard |
| 2.3 | Groups: the structure under everything |
| 2.4 | The discrete logarithm problem |
| 2.5 | Finite fields |
| 2.6 | Elliptic curves, without the pain |
| 2.7 | A first look at lattices |
| 2.8 | Where each idea is used |

2.1 Arithmetic that wraps around

A clock is the best introduction to cryptographic arithmetic. If it is 10 o'clock and you add 5 hours, you get 3 o'clock, not 15. The numbers wrap around after 12. Mathematicians write this as:

$$10 + 5 = 3 \pmod{12}$$

and say "10 plus 5 is congruent to 3, modulo 12". The number after "mod" is the **modulus**. Working "mod n" means you only keep the remainder after dividing by n. So $17 \bmod 5 = 2$, because 17 divided by 5 is 3 with remainder 2.

All the usual operations work in modular arithmetic:

- **Addition:** $(4 + 9) \bmod 11 = 13 \bmod 11 = 2$
- **Multiplication:** $(4 \times 9) \bmod 11 = 36 \bmod 11 = 3$
- **Exponentiation:** $3^4 \bmod 7 = 81 \bmod 7 = 4$

Why does cryptography love this? Two reasons. First, numbers stay a fixed size: no matter how many operations you do, the result is always between 0 and n minus 1, which computers handle efficiently. Second, wrapping around scrambles the relationship between input and output. Knowing that $3^x \bmod 7 = 4$ does not easily tell you x, because the results jump around instead of growing steadily.

Fast exponentiation

Public-key cryptography raises numbers to enormous powers, with exponents hundreds of digits long. Multiplying 3 by itself that many times would never finish. Instead, computers use **square-and-multiply**: write the exponent in binary, square at every step, and multiply in the base whenever the binary digit is 1. Computing $3^{13} \bmod 7$:

```
13 in binary = 1101
start with 1
bit 1: square (1) then multiply by 3 -> 3
bit 1: square (9 mod 7 = 2) then multiply by 3 -> 6
bit 0: square (36 mod 7 = 1) -> 1
bit 1: square (1) then multiply by 3 -> 3
result: 3^13 mod 7 = 3
```

The number of steps grows with the number of digits in the exponent, not its size, so a 3072-bit exponent takes a few thousand steps instead of an impossible number.

IN PRACTICE The square-and-multiply pattern is also a classic source of leaks. If "multiply" steps take measurable time, an attacker timing the operation can read the exponent's bits, which is the private key. Production libraries use **constant-time** versions that do the same work for every bit (Chapter 9).

Inverses and the greatest common divisor

In ordinary arithmetic, dividing by 4 means multiplying by $1/4$. In modular arithmetic, "dividing" means multiplying by an **inverse**: a number that gives 1 when multiplied. Mod 11, the inverse of 4 is 3, because $4 \times 3 = 12 = 1 \pmod{11}$.

An inverse exists only when the number and the modulus share no common factor, that is, when their **greatest common divisor (gcd)** is 1. The ancient **Euclidean algorithm** computes gcds quickly, and its extended version finds inverses. RSA key generation (Chapter 7) depends on it.

2.2 Primes and why factoring is hard

A **prime** is a whole number greater than 1 whose only divisors are 1 and itself: 2, 3, 5, 7, 11, 13 and so on. Every whole number breaks down into a unique product of primes, for example $60 = 2 \times 2 \times 3 \times 5$.

Two facts about primes power a large part of cryptography:

- 1. Multiplying two large primes is easy.** A computer multiplies two 1536-bit primes in a fraction of a millisecond.
- 2. Splitting the result back into its primes is hard.** For a 3072-bit product, no known method on classical computers can find the factors in any useful amount of time.

This one-way behaviour is the **factoring problem**, and RSA is built on it. Note the word "known". Nobody has proved factoring is hard. It is believed hard because the best mathematicians have tried for centuries. That belief has a sharp exception: a large quantum computer running **Shor's algorithm** could factor efficiently (Chapter 25).

Finding large primes

If factoring is hard, how do we find primes in the first place? We do not factor anything. We pick a random large odd number and run a **probabilistic primality test**, such as Miller-Rabin, which quickly answers "definitely composite" or "almost certainly prime". Repeating the test enough times pushes the chance of error below one in 2^{128} . Primes are common enough among large numbers that a few hundred random tries usually find one.

2.3 Groups: the structure under everything

A **group** is a set of things plus one operation that behaves nicely. Formally, the operation must combine any two members into another member, have an identity element that changes nothing, give every member an inverse, and be associative. You do not need to memorise the definition. What matters is the idea: a group is a closed world where you can combine elements as many times as you like and always stay inside.

Two groups appear constantly in this book:

- **The multiplicative group mod a prime p .** The numbers 1 to p minus 1, combined by multiplication mod p . Used by classic Diffie-Hellman and DSA.
- **The points on an elliptic curve.** Combined by a geometric "point addition" rule. Used by ECDH, ECDSA and EdDSA.

Generators and order

In some groups, one element can produce every other element by repeated combination. That element is called a **generator**. Mod 7, the number 3 is a generator:

$$3^1 = 3, 3^2 = 2, 3^3 = 6, 3^4 = 4, 3^5 = 5, 3^6 = 1 \pmod{7}$$

Every number from 1 to 6 appears exactly once. The number of distinct elements a generator produces before repeating is its **order**. Cryptographic designs choose groups whose order is a large prime, or contains one, because small orders let attackers break the problem into easy pieces. Careless choices here caused real attacks, such as small-subgroup attacks, which is one reason modern standards fix the groups for you.

2.4 The discrete logarithm problem

In the table above, going forward is easy: given 3 and 4, computing $3^4 \bmod 7 = 4$ takes one step. Going backward is the hard direction: given that $3^x \bmod 7 = 4$, find x . This is the **discrete logarithm problem**, and with a 3072-bit prime, it is as hard to reverse as factoring.

Diffie-Hellman key exchange (Chapter 8) uses it like this. Alice picks a secret a and publishes g^a . Bob picks a secret b and publishes g^b . Each raises the other's public value to their own secret, and both reach $g^{(ab)}$. An eavesdropper sees g^a and g^b but cannot compute $g^{(ab)}$ without solving a discrete logarithm.

2.5 Finite fields

A **field** is a set where you can add, subtract, multiply and divide (except by zero), with all the familiar rules holding. A **finite field** is a field with a finite number of elements. Two kinds matter:

- **Prime fields, written $GF(p)$.** The numbers 0 to p minus 1 with arithmetic mod a prime p . Elliptic curves such as P-256 and Curve25519 are defined over prime fields.
- **Binary fields, written $GF(2^n)$.** Elements are strings of n bits, and addition is bitwise XOR. AES does its internal mixing in $GF(2^8)$, the field of all byte values (Chapter 4), and the GCM mode computes its integrity tag in $GF(2^{128})$ (Chapter 5).

The useful intuition: a finite field is a set of fixed-size values with arithmetic that never overflows and always has a clean answer, which is exactly what both hardware and cryptographers want.

2.6 Elliptic curves, without the pain

An **elliptic curve** is the set of points (x, y) satisfying an equation such as:

$$y^2 = x^3 + ax + b$$

Over ordinary real numbers, this draws a smooth curve. Cryptography uses the same equation over a finite field, so the "curve" becomes a scattered set of points, but the rules carry over.

The key trick is **point addition**. Draw a line through two points on the curve. It hits the curve in exactly one more point. Reflect that point across the x-axis, and you have the "sum" of the first two. Adding a point to itself uses the tangent line instead. With this rule, the points form a group.

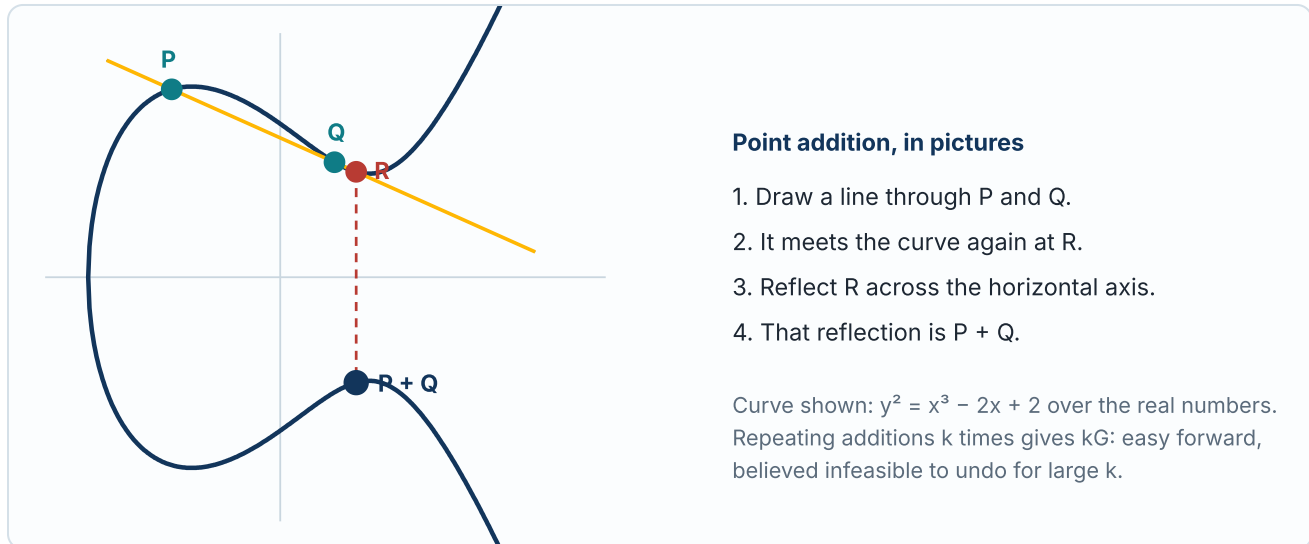


Figure 2.1 The geometric idea behind elliptic curve point addition. Real cryptographic curves use the same rules over a finite field.

Now the hard problem. Pick a starting point G and a secret whole number k . Computing kP , meaning G added to itself k times, is fast using the same doubling trick as square-and-multiply. But given G and the result, finding k is the **elliptic curve discrete logarithm problem (ECDLP)**, and it is much harder than the ordinary discrete logarithm for the same size of numbers. That is why a 256-bit elliptic curve key matches a 3072-bit RSA key.

The curves you will meet are standardised and named:

Curve	Used in	Notes
P-256 (secp256r1)	TLS, certificates, ECDSA, many HSMs	NIST curve; the most widely deployed
P-384	High-assurance government and financial uses	NIST curve; required by some regulated profiles
Curve25519 / X25519	Key exchange in TLS 1.3, SSH, Signal, WireGuard	Designed to be fast and hard to implement wrongly
Ed25519	Signatures in SSH, software signing, many modern protocols	Edwards form of Curve25519

2.7 A first look at lattices

Everything above, factoring and both discrete logarithm problems, falls to Shor's algorithm on a large quantum computer. The replacement algorithms standardised by NIST in 2024 rest mostly on a different kind of mathematics: **lattices**.

Picture a sheet of graph paper stretched and skewed so its grid lines run at odd angles. The crossing points form a lattice. In two dimensions, finding the crossing point closest to some random spot is easy. In 500 or 1000 dimensions, with a deliberately awkward description of the grid, it becomes extremely hard, for classical and quantum computers alike, as far as anyone knows.

The specific problem behind ML-KEM and ML-DSA is called **Learning With Errors (LWE)**. In words: you are given many equations that would be easy to solve, except that each one has a small random error added. The errors make the system unsolvable in practice. Chapter 26 builds on this idea. For now, remember one sentence: **post-quantum cryptography replaces "hard to factor" and "hard to find the logarithm" with "hard to find the secret hidden under small noise"**.

2.8 Where each idea is used

Idea	Used by	Chapter
Modular arithmetic and fast exponentiation	RSA, Diffie-Hellman, DSA	7, 8
Primes and factoring	RSA	7
Groups and discrete logarithms	Diffie-Hellman, DSA	8, 9
Binary finite fields	AES, GCM	4, 5
Elliptic curves	ECDH, ECDSA, EdDSA, X25519	8, 9
Lattices and LWE	ML-KEM, ML-DSA	26

HANDS-ON LAB

Lab: check the mathematics yourself

You only need Python 3, which is already installed on most computers. Open a terminal and type `python3`.

1. Modular arithmetic and fast exponentiation. Python's built-in `pow` takes a third argument, the modulus, and uses square-and-multiply internally:

```
pow(3, 13, 7)          # 3, matching the worked example
pow(5, 10**18, 10**9+7) # instant, despite the huge exponent
```

2. Inverses. Since Python 3.8, a power of minus 1 computes the modular inverse:

```
pow(4, -1, 11)  # 3, because 4 x 3 = 12 = 1 mod 11
```

3. A toy discrete logarithm. Find x such that $3^x \bmod 1000003 = 123456$ by brute force, and time it:

```
p, g, h = 1000003, 3, 123456
x = next(x for x in range(p) if pow(g, x, p) == h)
print(x, pow(g, x, p))
```

This finishes in a second or so because p has only 7 digits. Each extra digit multiplies the work by about ten. At 925 digits, the size of a 3072-bit prime, brute force is hopeless, and even the cleverest known methods are far out of reach.

4. Toy RSA. Build and break RSA with tiny primes:

```
p, q, e = 61, 53, 17
n, phi = p*q, (p-1)*(q-1)
d = pow(e, -1, phi)          # private exponent
c = pow(42, e, n)            # encrypt 42
print(pow(c, d, n))          # decrypt: 42
# now break it: factor n by trial division
print([f for f in range(2, n) if n % f == 0][:2])
```

Factoring 3233 is instant. That is the whole security argument of RSA in miniature: everything rests on n being too large to factor.

KEY TAKEAWAYS

Chapter summary

- Modular arithmetic keeps numbers a fixed size and scrambles input-output relationships. Fast exponentiation makes huge powers practical, and must be constant-time in real code.
- Multiplying primes is easy; factoring their product is believed hard. RSA rests on this.
- Groups give a closed world for repeated operations. In well-chosen groups, finding the discrete logarithm is believed hard. Diffie-Hellman rests on this.
- Elliptic curves give a group where the discrete logarithm is much harder, allowing much smaller keys.
- All of these fall to large quantum computers. Lattice problems such as Learning With Errors are the main replacement.

CHECK YOURSELF

Review questions

1. Compute $7 \times 8 \bmod 13$ and $2^{10} \bmod 11$ by hand.
2. Why does square-and-multiply need only a few thousand steps for a 3072-bit exponent?
3. Why must cryptographic groups have a large prime order?
4. In your own words, what is the discrete logarithm problem, and which protocol relies on it?
5. Why can a 256-bit elliptic curve key match the strength of a 3072-bit RSA key?
6. Which mathematical problem underlies ML-KEM, and why is it described as "a secret hidden under small noise"?

Randomness, Probability and Hardness

Every key, nonce and salt in this book starts as a random number. If that number is predictable, nothing built on top of it is secure. This chapter explains what good randomness is, where computers get it, how probability decides the safe sizes of keys and nonces, and what cryptographers mean when they call a problem "hard".

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Explain entropy and the difference between true and pseudo-random numbers
- Choose the correct randomness source in any common language
- Apply the birthday bound to nonces, hashes and identifiers
- Describe what "computationally hard" and "negligible" mean in security claims

IN THIS CHAPTER

- | | |
|-----|---|
| 3.1 | Why randomness is the foundation |
| 3.2 | Entropy: measuring unpredictability |
| 3.3 | Where computers get randomness |
| 3.4 | Probability that every engineer should know: the birthday bound |
| 3.5 | What "hard" means |
| 3.6 | Security in practice: work factors |

3.1 Why randomness is the foundation

A 128-bit key is strong only if an attacker has to try all 2^{128} possibilities. If the key was generated by a process that can only produce a million different values, the attacker only has to try a million, and the 128 bits on paper mean nothing. The Debian incident in Chapter 1 is exactly this: keys that looked long but came from a pool of about 32,000 values.

Cryptography consumes randomness in several roles:

Item	What it is	What goes wrong if it is predictable
Key	The secret that protects everything	Attacker guesses the key
Nonce ("number used once")	A value that must never repeat under the same key	Encryption leaks data; signatures leak the private key
Initialisation vector (IV)	A starting value for some encryption modes	Patterns in data become visible
Salt	A random value mixed into password hashing	Precomputed tables crack many passwords at once
Session ID or token	An identifier that grants access	Attacker guesses a valid session and takes it over

3.2 Entropy: measuring unpredictability

Entropy is the measure of how unpredictable something is, counted in bits. A fair coin flip has 1 bit of entropy. A fair six-sided die has about 2.6 bits. A truly random 128-bit key has 128 bits.

Two traps catch people:

- **Length is not entropy.** A 20-character password made from a dictionary word plus a year, such as "Summer2026!", has far less entropy than its length suggests, because attackers guess common patterns first.
- **Average is not worst case.** Cryptographers care about **min-entropy**: how likely the single most likely value is. A source that produces a fixed value 1 percent of the time is dangerous even if its other outputs look random, because an attacker simply guesses the common value first.

3.3 Where computers get randomness

Computers are built to be predictable, so randomness has to be harvested from the physical world and then stretched.

- 1. Entropy sources.** Unpredictable physical events: timing jitter between hardware interrupts, electrical noise inside dedicated hardware random number generators (such as the RDRAND and RDSEED instructions in modern processors), and similar sources.
- 2. The operating system's pool.** The kernel mixes many sources together, so that one weak or compromised source cannot ruin the result.
- 3. A cryptographically secure pseudo-random number generator (CSPRNG).** A deterministic algorithm, seeded from the pool, that produces an endless stream of output that nobody can predict without knowing the seed.

A CSPRNG differs from an ordinary pseudo-random generator in one crucial property: even an attacker who sees millions of outputs cannot predict the next one or work out earlier ones. Ordinary generators, such as those used for simulations and games, do not have this property. Their output can often be predicted after seeing a few values.

The rule: always use the operating system's CSPRNG

In almost every situation, the correct answer is to ask the operating system or your language's dedicated cryptographic API:

Language or platform	Use this	Never use this for secrets
Linux	<code>getrandom()</code> system call, or <code>/dev/urandom</code>	Custom generators
Windows	<code>BCryptGenRandom</code>	<code>rand()</code>
Python	<code>secrets</code> module, <code>os.urandom</code>	<code>random</code> module
Java	<code>java.security.SecureRandom</code>	<code>java.util.Random</code>
JavaScript (browser and Node.js)	<code>crypto.getRandomValues</code> , <code>crypto.randomBytes</code>	<code>Math.random()</code>
Go	<code>crypto/rand</code>	<code>math/rand</code>

A widely repeated myth says `/dev/random` is better than `/dev/urandom` on Linux because it "blocks when entropy runs out". On modern Linux kernels, both are fed by the same CSPRNG, and `/dev/urandom` is the right choice once the system has been seeded at boot. The `getrandom()` call handles the one genuinely risky moment, very early boot before seeding, by waiting until the pool is ready.

IN PRACTICE The riskiest places for randomness today are not laptops and servers but freshly booted virtual machines, cloned VM images, containers started from snapshots, and small embedded devices. A cloned VM can start with the same random state as its siblings. Make sure images are reseeded on first boot, and generate long-term keys after boot, never inside the image.

3.4

Probability that every engineer should know: the birthday bound

How many people must be in a room before there is a 50 percent chance that two share a birthday? Most people guess around 180. The answer is 23.

This **birthday paradox** happens because you are not comparing everyone with one person; you are comparing every pair, and the number of pairs grows much faster than the number of people. The general rule, called the **birthday bound**, is:

If values are drawn at random from N possibilities, a repeat becomes likely after about the square root of N draws.

For bit strings, the square root of 2^n is $2^{(n/2)}$. So among random 64-bit values, you expect a repeat after about 2^{32} , roughly 4 billion, draws. That sounds like a lot until you remember that a busy service can generate 4 billion values in days.

The birthday bound sets the safe size of many things:

Item	Size	Repeat expected after about	Consequence
Random 64-bit IDs	64 bits	4 billion	Too small for large systems
AES-GCM random nonces	96 bits	2^{48} messages	NIST limits random-nonce use to 2^{32} messages per key, for a large safety margin
Block cipher output blocks	128 bits (AES block)	2^{64} blocks	Rekey well before this; old 64-bit block ciphers fell to attacks like Sweet32
Hash outputs	256 bits (SHA-256)	2^{128} hashes	Collisions infeasible: this is why hash security is "half the output size"
UUID version 4	122 random bits	2^{61}	Fine as an identifier; not a substitute for a secret token

The last row of the hash line explains a rule you saw in Chapter 1: a hash function with an n -bit output gives only $n/2$ bits of security against collisions. SHA-256 gives 128-bit collision resistance.

3.5 What "hard" means

Security claims say things like "breaking this requires solving a hard problem". In cryptography, **hard** has a specific meaning: no algorithm running in a practical amount of time succeeds with more than a tiny probability.

Three terms make this precise:

- **Polynomial time** describes algorithms whose running time grows reasonably with input size, such as doubling when the input doubles. Honest users must be able to do their part in polynomial time.
- **Superpolynomial or exponential time** describes algorithms whose running time explodes, such as doubling every time the input grows by one bit. Attackers should face this.
- **Negligible** describes a success probability so small that it shrinks faster than any polynomial. In practice, below about one in 2^{80} to 2^{128} .

Two cautions keep these claims honest:

1. **Hardness is believed, not proved.** No one has proved that factoring or lattice problems are hard. Confidence comes from decades of failed attacks.
2. **Hardness is relative to a computer model.** Factoring is hard for classical computers and easy for large quantum ones. A problem's hardness can change when technology changes, which is exactly what is happening now.

3.6 Security in practice: work factors

Engineers translate abstract hardness into concrete **work factors**, the expected number of operations to break a system. Some reference points help calibrate:

Operations	Roughly equals
2^{40}	A laptop for a few hours
2^{64}	A large cluster for months; feasible for well-funded attackers (SHA-1 collisions were found with about 2^{63})

Operations	Roughly equals
2^{80}	Beyond any single organisation today; considered the old minimum
2^{112}	NIST's current minimum security level
2^{128}	The standard target; far beyond all computing on Earth
2^{256}	Exceeds the number of atoms in the observable universe (about 2^{266})

Choose 128-bit security for almost everything. Choose 256-bit symmetric keys where data must stay secret for decades, as a margin against both unknown advances and quantum search (Chapter 25).

HANDS-ON LAB

Lab: see weak randomness fail

This lab uses Python 3.

1. Predicting a non-cryptographic generator. Python's `random` module uses the Mersenne Twister. After observing 624 consecutive 32-bit outputs, its internal state can be completely reconstructed. You do not need to do the reconstruction to see the danger: seed it with a time value and watch it repeat.

```
import random, time
seed = int(time.time())
random.seed(seed); a = [random.getrandbits(32) for _ in range(3)]
random.seed(seed); b = [random.getrandbits(32) for _ in range(3)]
print(a == b)    # True: same seed, same "random" numbers
```

An attacker who knows roughly when a token was created can try every second in that window.

2. The right way.

```
import secrets
print(secrets.token_hex(32))    # 256 bits, suitable for keys and tokens
print(secrets.token_urlsafe(32)) # for URLs and reset links
```

3. The birthday bound in action. Draw random 24-bit values until one repeats:

```
import secrets
seen, n = set(), 0
while True:
    v = secrets.randbits(24); n += 1
    if v in seen: break
    seen.add(v)
print(n) # usually a few thousand, near the square root of 2^24 = 4096
```

Run it several times. The answer clusters around a few thousand, not millions. Scale the same intuition up to 64-bit identifiers and 96-bit nonces.

CASE STUDY

Failure story: the console that signed with a constant

In 2010, a group of researchers presented a complete break of the code-signing system protecting the PlayStation 3. The console accepted only software signed with the manufacturer's private key, using the ECDSA signature algorithm (Chapter 9). ECDSA requires a fresh random number for every signature. The implementation used the **same** number every time.

With two signatures made using the same random value, simple algebra reveals the private key. The researchers recovered the master signing key, and from then on anyone could sign software the console would trust. The key could not be changed in devices already sold.

Nothing was wrong with ECDSA or with the curve. One value that had to be random was constant. Modern implementations avoid this whole class of failure by deriving the per-signature value deterministically from the key and message (RFC 6979, and the design of EdDSA), so a broken random generator cannot leak the key.

KEY TAKEAWAYS

Chapter summary

- Every key, nonce, IV, salt and token depends on unpredictable randomness. Predictable randomness silently destroys security.
- Entropy measures unpredictability; length and appearance do not. Min-entropy is what attackers exploit.

- Always use the operating system's CSPRNG through your language's cryptographic API. Watch cloned VMs, containers and embedded devices.
- The birthday bound says repeats become likely after about the square root of the number of possibilities. It sets safe sizes for nonces, identifiers and hash outputs.
- "Hard" means infeasible for practical computers, as far as we know, and can change with technology.

CHECK YOURSELF

Review questions

1. Why is "Summer2026!" a weak secret despite being 11 characters long?
2. Which function would you use to generate a password reset token in Python, Java and JavaScript?
3. Why are cloned virtual machines a randomness risk, and what should you do about it?
4. Using the birthday bound, roughly how many random 32-bit session IDs can you issue before a collision is likely?
5. Why does SHA-256 provide only 128 bits of collision resistance?
6. What single design change prevents the PlayStation 3 failure from happening in modern signature schemes?

PART II

Symmetric Cryptography

The fast workhorses: block ciphers, modes of operation, authenticated encryption, hashing, MACs and key derivation.

In this part

- 4 Block Ciphers and AES
- 5 Modes of Operation and Authenticated Encryption
- 6 Hash Functions, MACs and Key Derivation

Block Ciphers and AES

A block cipher is the most important building block in symmetric cryptography, and AES is the block cipher the world runs on. This chapter explains what a block cipher is, how AES works inside without drowning you in detail, why its key sizes are what they are, and how it is implemented safely and quickly on modern hardware.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Explain what a block cipher does and why it is modelled as a pseudo-random permutation
- Describe the structure of AES and the purpose of each round step
- Choose between AES-128 and AES-256 with reasons
- Explain side-channel risks and why hardware AES instructions matter

IN THIS CHAPTER

- | | |
|-----|--|
| 4.1 | What a block cipher does |
| 4.2 | A short history: from DES to AES |
| 4.3 | Inside AES |
| 4.4 | AES-128 or AES-256? |
| 4.5 | Implementing AES safely: side channels |
| 4.6 | Other block and stream ciphers you will meet |

4.1 What a block cipher does

A **block cipher** takes a fixed-size block of data and a key, and produces a scrambled block of the same size. With the same key, the scrambling can be reversed. AES uses 128-bit blocks, so it turns 16 bytes of plaintext into 16 bytes of ciphertext.

```
ciphertext_block = AES_encrypt(key, plaintext_block)
plaintext_block  = AES_decrypt(key, ciphertext_block)
```

Think of a block cipher as a giant codebook. For every possible key, there is a different codebook that maps each of the 2^{128} possible input blocks to a unique output block. Nobody could ever write such a codebook down, but the cipher computes any entry on demand.

A good block cipher should be indistinguishable from a **random permutation**: a random shuffling of all possible blocks. An attacker who does not know the key, and who can ask for the encryption of any blocks they like, should not be able to tell whether they are talking to AES or to a truly random shuffle. Cryptographers call this being a **pseudo-random permutation (PRP)**, and it is the formal security goal for block ciphers.

A block cipher alone is not encryption

This is the first important lesson of the chapter. A block cipher only handles one 16-byte block. Real messages are longer, and encrypting each block independently with the same key leaks patterns: identical plaintext blocks produce identical ciphertext blocks. To encrypt real data, you combine the block cipher with a **mode of operation** (Chapter 5). The cipher is an engine; the mode is the vehicle built around it.

4.2 A short history: from DES to AES

The **Data Encryption Standard (DES)** was published in 1977 and protected banking and government data for two decades. It used a 56-bit key, which was small even then. By 1998, a purpose-built machine costing about 250,000 US dollars found a DES key in days. A stop-gap called **Triple DES (3DES)** applied DES three times, but it was slow and still had a 64-bit block, which became a problem of its own.

The 64-bit block matters because of the birthday bound from Chapter 3. After about 2^{32} blocks, which is only 32 gigabytes, repeats become likely and leak information. In 2016, the **Sweet32** attack recovered secret cookies from long-lived HTTPS connections using

3DES this way. NIST has since disallowed 3DES for encryption after 2023.

In 1997, NIST launched an open, international competition for a replacement. Fifteen designs were submitted and attacked publicly for three years. In 2000, NIST selected **Rijndael**, designed by two Belgian cryptographers, Joan Daemen and Vincent Rijmen. It was published in 2001 as the **Advanced Encryption Standard (AES)**, FIPS 197. The open competition model worked so well that NIST used it again for SHA-3 and for post-quantum cryptography.

4.3 Inside AES

You will never implement AES yourself, but knowing its structure helps you understand its strength, its key sizes and its attacks.

AES treats a 16-byte block as a 4 by 4 grid of bytes, called the **state**. It transforms the state through a series of **rounds**: 10 rounds for a 128-bit key, 12 for 192-bit, and 14 for 256-bit. Each round applies four steps:

Step	What it does	Why it is there
SubBytes	Replaces each byte with another using a fixed lookup table (the S-box)	Adds non-linearity, so the cipher cannot be described by simple equations
ShiftRows	Shifts each row of the grid left by 0, 1, 2 or 3 positions	Spreads bytes across columns
MixColumns	Mixes the four bytes of each column together using arithmetic in $GF(2^8)$	Spreads each byte's influence across the whole column
AddRoundKey	XORs the state with a round key derived from the main key	Brings in the secret; without it, anyone could reverse the steps

After two rounds, every output bit depends on every input bit. This property is called **diffusion**, and the non-linear S-box gives **confusion**, a term introduced by Claude Shannon in 1949 to describe hiding the relationship between key and ciphertext. Ten or more rounds give a large safety margin: the best known attacks on full AES are barely better than trying every key.

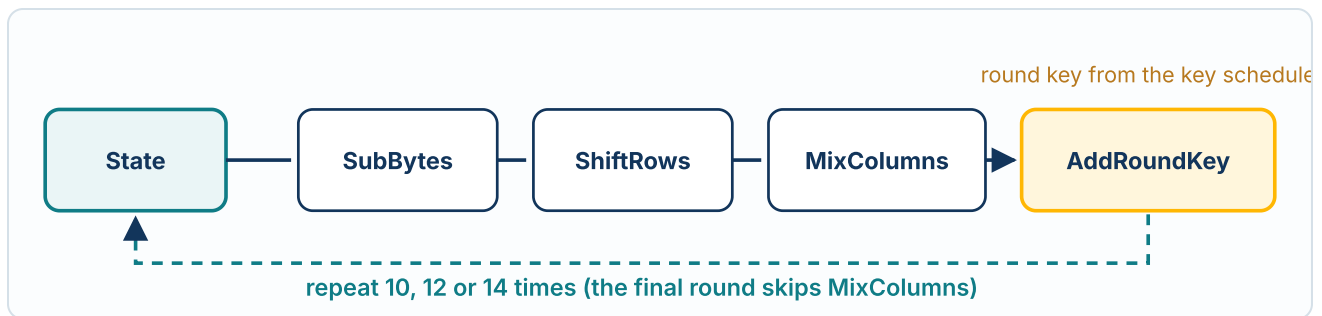


Figure 4.1 One AES round. Each step has a separate job; together they spread every input bit across the whole block.

The **key schedule** expands the main key into one round key per round. It is the part of AES most often criticised by researchers, because related-key attacks exploit it for AES-256 in artificial settings where an attacker can get data encrypted under several keys with a known relationship. Real systems never create related keys if they use proper key derivation (Chapter 6), so these attacks have no practical impact.

4.4 AES-128 or AES-256?

AES comes in three key sizes. Only two are common in practice.

Variant	Rounds	Classical security	Security against quantum search	Typical use
AES-128	10	128 bits	About 64 bits in theory, much more in practice	General-purpose encryption, TLS
AES-192	12	192 bits	About 96 bits	Rarely used
AES-256	14	256 bits	About 128 bits	Long-lived data, government profiles, key wrapping

Grover's algorithm (Chapter 25) lets a quantum computer search keys in roughly the square root of the usual time, which halves the effective key length on paper. In practice, Grover's algorithm parallelises badly and needs extraordinarily long, error-free quantum computations, so AES-128 is not considered at real risk. Still, the guidance has settled on a simple rule:

- Use **AES-256** for data that must stay confidential for decades, for keys that protect other keys, and wherever a regulation asks for it. The US NSA's CNSA 2.0 suite, for example, requires AES-256.
- **AES-128** remains perfectly sound for short-lived data such as most TLS sessions, and it is about 40 percent faster.

When in doubt, choose AES-256. The performance difference rarely matters on modern hardware.

4.5 Implementing AES safely: side channels

The mathematics of AES has held up for 25 years. Implementations have not always been so lucky. A **side-channel attack** learns secrets not from the inputs and outputs, but from how the computation behaves: how long it takes, how much power it draws, what it leaves in the processor's cache, or the electromagnetic signals it emits.

The classic AES side channel is the **cache-timing attack**. Fast software AES uses lookup tables. The table entry read depends on the data and the key, and whether that entry happens to be in the processor's cache changes the timing by a few nanoseconds. In 2005, researchers recovered full AES keys from a server over the network by measuring these timing differences across many requests.

There are two defences:

1. **Hardware AES instructions.** Almost every modern processor includes dedicated AES instructions (AES-NI on x86, the cryptography extensions on ARM). They run AES in constant time with no tables, and they are many times faster than software. Mature libraries use them automatically.
2. **Bit-sliced, constant-time software** for processors without hardware support.

Physical side channels, such as power analysis on smart cards and hardware security modules, need hardware countermeasures as well. This is one of the things certification schemes like FIPS 140-3 test for (Chapter 23).

IN PRACTICE You will almost never choose an AES implementation directly. You choose a library. Check that it is current, widely used, and built with hardware acceleration enabled. On most servers, `openssl speed -evp aes-256-gcm` reports several gigabytes per second, which tells you the hardware path is active.

4.6 Other block and stream ciphers you will meet

AES dominates, but you will see others:

Cipher	Type	Status	Where you see it
ChaCha20	Stream cipher	Recommended	TLS 1.3, WireGuard, SSH, mobile devices without AES hardware (Chapter 5)
3DES	Block cipher (64-bit block)	Disallowed for new encryption	Legacy payment systems
Blowfish	Block cipher (64-bit block)	Legacy	Old software, the bcrypt password hash
Camellia, ARIA, SM4	Block ciphers	National standards (Japan, Korea, China)	Regional compliance requirements
RC4	Stream cipher	Broken, prohibited in TLS	Very old systems only

A **stream cipher** produces a long stream of pseudo-random bits from a key and a nonce, and encrypts by XOR-ing that stream with the data. ChaCha20 is the modern standard, designed by Daniel Bernstein, and it is fast and constant-time even in plain software, which is why phones without AES hardware historically preferred it.

HANDS-ON LAB

Lab: AES at the command line

You need OpenSSL 3, which is installed on most Linux and macOS systems.

1. See the speed of hardware AES.

```
openssl speed -evp aes-128-gcm aes-256-gcm chacha20-poly1305
```

Compare the throughput columns. On most modern laptops, both AES variants run at multiple gigabytes per second.

2. See why a raw block cipher leaks patterns. Create a file with repeated 16-byte blocks and encrypt it block by block (ECB mode, which you should never use for real data):

```
python3 -c "print('YELLOW SUBMARINE'*8, end='')" > blocks.txt
openssl enc -aes-128-ecb -K 000102030405060708090a0b0c0d0e0f -in blocks.txt | xxd
```

Every 16-byte line of output is identical, revealing that the input repeats. That is exactly the leak modes of operation exist to prevent.

3. Now use a proper mode. Encrypt the same file with CTR mode and a random IV:

```
openssl enc -aes-128-ctr -K 000102030405060708090a0b0c0d0e0f \
  -iv $(openssl rand -hex 16) -in blocks.txt | xxd
```

The output no longer repeats. Chapter 5 explains why, and why even this is not enough without integrity protection.

CASE STUDY

Failure story: the encrypted penguin

The most famous image in cryptography education is a bitmap of the Linux mascot, Tux the penguin, encrypted with AES in ECB mode. The encrypted file is meaningless noise at the byte level, yet when displayed as an image, the penguin's outline is clearly visible, because every identical block of background colour encrypts to the same ciphertext block.

The penguin is a classroom example, but the same mistake has appeared in real products. In 2013, a breach at a large software company exposed about 150 million user records in which passwords had been encrypted with 3DES in ECB mode instead of being hashed. Because identical passwords produced identical ciphertext, and users' password hints were stored in plain text next to them, attackers could group users who shared a password and guess it from the combined hints, without ever recovering the key.

Two lessons follow. A strong block cipher used in the wrong mode leaks patterns. And passwords should never be encrypted at all: they should be hashed with a dedicated password hashing function (Chapter 6).

KEY TAKEAWAYS

Chapter summary

- A block cipher transforms fixed-size blocks under a key and should behave like a random permutation. On its own, it is not a complete encryption scheme.
- DES fell to brute force and 3DES to the birthday bound on its small block. AES was chosen through an open competition and has resisted attack for 25 years.
- AES rounds combine substitution, shifting, mixing and key addition to achieve confusion and diffusion.

- Prefer AES-256 for long-lived data and key protection; AES-128 remains sound for short-lived data.
- Side channels, not mathematics, are AES's practical weakness. Hardware AES instructions remove the classic timing attacks.

CHECK YOURSELF

Review questions

1. Why is a block cipher not, on its own, a complete way to encrypt a file?
2. Explain how a 64-bit block size led to the Sweet32 attack.
3. What is the purpose of the S-box in AES? What would happen without it?
4. A colleague insists AES-128 is "broken by quantum computers". How would you respond?
5. Describe a cache-timing attack in two sentences. What removes it on modern servers?
6. What two mistakes did the 2013 password breach combine, and what should have been done instead?

Modes of Operation and Authenticated Encryption

A block cipher encrypts 16 bytes. Real data is longer, and real attackers do more than read. This chapter shows how modes of operation turn a block cipher into practical encryption, why encryption without integrity has caused decades of breaks, and why authenticated encryption, especially AES-GCM and ChaCha20-Poly1305, is now the only sensible default.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Explain how CBC, CTR and GCM modes work and what each requires of its IV or nonce
- Describe padding oracle attacks and why unauthenticated encryption fails
- Use AEAD correctly, including associated data and nonce management
- Choose between AES-GCM, ChaCha20-Poly1305 and misuse-resistant modes

IN THIS CHAPTER

- | | |
|-----|--|
| 5.1 | Why modes exist |
| 5.2 | CBC: chaining blocks together |
| 5.3 | CTR: turning a block cipher into a stream cipher |
| 5.4 | Authenticated encryption: confidentiality plus integrity |
| 5.5 | AES-GCM |
| 5.6 | ChaCha20-Poly1305 |
| 5.7 | Misuse-resistant encryption |
| 5.8 | Disk encryption is a special case |
| 5.9 | Choosing a mode: the decision table |

5.1 Why modes exist

Chapter 4 ended with the encrypted penguin: encrypting each block independently (**ECB mode**) leaks every repetition in the data. A **mode of operation** is a recipe for using a block cipher on data of any length so that patterns disappear. Every mode does this by mixing in something that changes: a previous ciphertext block, a counter, or a random starting value.

That changing input has two names you must know:

- **Initialisation vector (IV):** a starting value for the mode. Some modes need it to be unpredictable.
- **Nonce:** short for "number used once". A value that must never repeat under the same key. It does not have to be secret or random, only unique.

Getting the IV or nonce wrong is the single most common way modes fail in practice.

5.2 CBC: chaining blocks together

Cipher Block Chaining (CBC) mode XORs each plaintext block with the previous ciphertext block before encrypting it. The first block is XOR-ed with the IV.

```
C1 = Encrypt(K, P1 XOR IV)
C2 = Encrypt(K, P2 XOR C1)
C3 = Encrypt(K, P3 XOR C2) ...
```

Because each ciphertext block depends on all previous blocks, identical plaintext blocks now produce different ciphertext. CBC was the default for TLS and disk encryption for many years.

CBC has three requirements and two weaknesses:

- The IV must be **unpredictable** for each message. Predictable IVs enabled the BEAST attack on TLS 1.0 in 2011.
- The plaintext must be **padded** to a multiple of 16 bytes, usually with PKCS#7 padding, where the last bytes all equal the number of padding bytes added.
- Encryption cannot run in parallel, because each block needs the previous one.
- **CBC provides no integrity.** Flipping a bit in ciphertext block C1 scrambles P1 but flips the same bit in P2. An attacker can make controlled changes without the key.

- Its padding becomes an attack surface, as the next section shows.

■ The padding oracle attack

Imagine a server that decrypts CBC data and responds differently when the padding is wrong ("padding error") versus when the content is wrong ("invalid request"). That small difference is an **oracle**: a yes-or-no answer about a secret. In 2002, Serge Vaudenay showed that an attacker can use it to decrypt any ciphertext, one byte at a time, by submitting about 128 carefully altered versions per byte and watching which ones produce a valid padding response. No key is ever recovered; the server decrypts the data for the attacker.

Padding oracles appeared in web frameworks, in TLS (the Lucky Thirteen attack in 2013 used timing differences instead of error messages, and POODLE in 2014 broke SSL 3.0), and in countless custom applications. The deeper lesson: **any encryption that can be modified without detection will eventually leak through the way systems react to the modified data.** The fix is not better error messages. It is integrity protection.

5.3 CTR: turning a block cipher into a stream cipher

Counter (CTR) mode encrypts a sequence of counter values and XORs the results with the plaintext:

```
keystream block i = Encrypt(K, nonce || counter_i)
Ci = Pi XOR keystream block i
```

CTR has attractive properties. It needs no padding, it runs in parallel, and encryption and decryption are the same operation. Its one rule is absolute:

***Never reuse a nonce with the same key.** If two messages are encrypted with the same key and nonce, they share the same keystream. XOR-ing the two ciphertexts cancels the keystream and gives the XOR of the two plaintexts, which is usually enough to recover both.*

Like CBC, CTR alone provides no integrity, and it is even easier to tamper with: flipping a ciphertext bit flips exactly the same plaintext bit. An attacker who knows a message says "pay 100" can change it to "pay 900" without knowing the key.

5.4

Authenticated encryption: confidentiality plus integrity

The solution to tampering is to attach an integrity check that only key holders can create. For years, developers did this by hand: encrypt with one key, then compute a MAC (Chapter 6) over the ciphertext with another key. The order matters. **Encrypt-then-MAC** is safe: the receiver checks the MAC first and rejects any altered ciphertext before decrypting anything. The other orderings, used by older TLS and SSH versions, led to attacks.

Hand-assembled constructions proved fragile, so cryptographers created **authenticated encryption with associated data (AEAD)**: single algorithms that encrypt and authenticate together, with a simple interface:

```
ciphertext, tag = AEAD_Encrypt(key, nonce, plaintext, associated_data)
plaintext       = AEAD_Decrypt(key, nonce, ciphertext, tag, associated_data)
                # fails completely if anything was altered
```

Associated data (AD) is information that must be protected against tampering but does not need to be secret, such as a packet header, a database record ID, or a version number. It is authenticated but not encrypted. Binding context this way prevents an attacker from moving a valid ciphertext into a different context, for example swapping the encrypted salary fields of two employees' records.

The decryption rule is strict: if the tag does not verify, the function returns an error and **no plaintext at all**. Never use or display partially decrypted data from a failed check.

5.5

AES-GCM

Galois/Counter Mode (GCM) combines CTR-mode encryption with a fast authenticator called GHASH, computed with multiplication in the finite field $GF(2^{128})$. It is the most widely used AEAD in the world: the default for TLS 1.3, IPsec, and most cloud storage encryption.

Its properties:

- Fast, especially with hardware support for AES and carry-less multiplication.
- A 96-bit nonce, which must be unique per key.
- A 128-bit tag.

GCM's weakness is how badly it fails when a nonce repeats. Reusing a nonce not only exposes the XOR of the two plaintexts, as with CTR. It also leaks enough about the internal authentication key that an attacker can forge valid tags for any message from then on. One nonce repeat turns GCM into unauthenticated encryption under attacker control.

Managing GCM nonces

You have two safe options:

1. **Counters.** Use a message counter as the nonce, with one counter per key, stored reliably. Ideal when one process owns the key.
2. **Random nonces.** Generate 96 random bits per message. Because of the birthday bound (Chapter 3), NIST limits random-nonce GCM to 2^{32} messages per key, which keeps the collision chance below about one in four billion.

If many machines encrypt with the same key, counters are hard to coordinate and random nonces hit the limit faster. The usual fix is to give each machine or each data object its own key, derived from a master key (Chapter 6), so no single key is ever used too much.

WATCH OUT Nonce reuse is not a theoretical concern. It happens when systems restore a counter from an old backup, when virtual machines are cloned with the same state, and when developers hard-code a nonce "for testing" and forget it. A 2016 internet-wide study found 184 HTTPS servers repeating GCM nonces, exposing them to forgery.

5.6 ChaCha20-Poly1305

ChaCha20-Poly1305 pairs the ChaCha20 stream cipher with the Poly1305 authenticator. It is standardised in RFC 8439, mandatory to support in TLS 1.3, and used by WireGuard, SSH and many messaging apps.

Property	AES-256-GCM	ChaCha20-Poly1305
Speed with AES hardware	Fastest	Fast
Speed without AES hardware	Slow, and hard to make constant-time	Fast and naturally constant-time
Nonce	96 bits	96 bits (XChaCha20 variant: 192 bits)
Nonce reuse consequence	Catastrophic	Catastrophic
Standards and compliance	FIPS-approved	Not FIPS-approved

Choose AES-GCM where FIPS validation is required or AES hardware is available. Choose ChaCha20-Poly1305 for software implementations on varied hardware. The extended-nonce variant **XChaCha20-Poly1305** uses a 192-bit nonce, large enough that random nonces never realistically collide, which makes it a forgiving choice for applications.

5.7 Misuse-resistant encryption

Because nonce reuse is so damaging, cryptographers designed **nonce-misuse-resistant** AEADs. The best known is **AES-GCM-SIV** (RFC 8452). It computes the authentication tag first and uses it as the effective IV. If a nonce is accidentally repeated, the only thing leaked is whether two messages were identical, which is a dramatic improvement over GCM's total failure. The cost is that encryption must make two passes over the data.

AES Key Wrap (RFC 3394 and the padded variant RFC 5649) and **AES-SIV** (RFC 5297) are related deterministic constructions used to encrypt keys with other keys, where there is no natural nonce. You will see Key Wrap constantly in key management systems and HSMs (Part VI).

5.8 Disk encryption is a special case

Encrypting a disk sector has an unusual constraint: the ciphertext must be exactly the same size as the plaintext, so there is no room for a nonce or a tag. Disk encryption therefore uses **XTS mode**, which ties each block's encryption to its sector number. XTS hides content but does not detect tampering, which is acceptable for protecting a stolen laptop, the threat disk encryption is designed for, but not for data sent over a network.

5.9 Choosing a mode: the decision table

Situation	Use	Avoid
General data encryption in an application	AES-256-GCM, or XChaCha20-Poly1305 via a high-level library	Hand-built CBC plus HMAC
Very high message volume per key, or risk of nonce reuse	AES-GCM-SIV, or derive per-message keys	Random-nonce GCM beyond 2^{32} messages
Encrypting keys with other keys	AES Key Wrap or AES-SIV	GCM with fixed nonces
Full-disk or volume encryption	AES-XTS	GCM, CBC

Situation	Use	Avoid
Legacy interoperability	CBC with Encrypt-then-MAC, only if forced	ECB, CBC without a MAC

HANDS-ON LAB

Lab: break CTR mode with a reused nonce

This Python lab uses the widely available `cryptography` package (`pip install cryptography`).

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os
key, nonce = os.urandom(32), os.urandom(16)
def ctr(msg):
    e = Cipher(algorithms.AES(key), modes.CTR(nonce)).encryptor()
    return e.update(msg) + e.finalize()
p1 = b"Transfer 100 USD to account 1234"
p2 = b"Meeting moved to Thursday at 3pm"
c1, c2 = ctr(p1), ctr(p2)          # same key, same nonce: the mistake
x = bytes(a ^ b for a, b in zip(c1, c2))
print(bytes(a ^ b for a, b in zip(x, p1))) # recovers p2 from p1 alone
```

The attacker never touched the key. Knowing (or guessing) one plaintext reveals the other.

Now tamper with an unauthenticated message:

```
c = bytearray(ctr(p1))
c[9] ^= ord("1") ^ ord("9")      # flip the bits that turn "1" into "9"
d = Cipher(algorithms.AES(key), modes.CTR(nonce)).decryptor()
print(d.update(bytes(c)))        # "Transfer 900 USD ..."
```

Finally, see AEAD refuse the same trick:

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
g, n = AESGCM(key), os.urandom(12)
c = bytearray(g.encrypt(n, p1, b"record-42"))
c[9] ^= ord("1") ^ ord("9")
g.decrypt(n, bytes(c), b"record-42") # raises InvalidTag: tampering detected
```

CASE STUDY

Failure story: Zerologon and the all-zero IV

In August 2020, Microsoft patched a vulnerability in the Netlogon protocol that Windows domain controllers use to authenticate machines. Researchers named it **Zerologon**, and it allowed an attacker on the internal network to take over a domain controller, and so the entire Windows domain, in seconds.

The root cause was a mode of operation. Netlogon computed a login credential using AES in a mode called CFB8 with a **fixed IV of all zeros**. With an all-zero IV, there is a 1-in-256 chance, for any key, that encrypting an all-zero input produces an all-zero output. An attacker could simply send an all-zero credential repeatedly; on average, after 256 attempts, it would be accepted as valid. From there, the attacker could reset the domain controller's own machine password.

AES was never weakened. The designers broke the one rule of the mode: the IV must be unpredictable. The flaw went unnoticed for years in one of the most widely deployed authentication systems in the world. It was rated at the maximum severity score and actively exploited by ransomware groups after disclosure.

KEY TAKEAWAYS

Chapter summary

- Modes turn a block cipher into practical encryption by mixing in changing values. IVs and nonces carry most of the risk.
- CBC needs unpredictable IVs and padding; CTR needs unique nonces. Neither provides integrity.
- Unauthenticated encryption leaks through tampering and through how systems react, as padding oracle attacks show.
- AEAD algorithms combine encryption and integrity. AES-GCM and ChaCha20-Poly1305 are the standard choices; associated data binds ciphertext to its context.
- GCM fails catastrophically on nonce reuse. Manage nonces deliberately, cap messages per key, or use misuse-resistant modes such as AES-GCM-SIV.

CHECK YOURSELF

Review questions

1. What is the difference between an IV and a nonce?
2. Explain a padding oracle attack to a developer in four sentences.
3. Why does reusing a nonce in CTR mode reveal the XOR of two plaintexts?
4. What is associated data, and give an example where it prevents an attack.
5. A service encrypts 10 billion records per year with one AES-GCM key and random nonces. What is wrong, and how would you fix it?
6. Why does disk encryption use XTS mode instead of GCM?
7. Which rule of modes of operation did Zerologon break?

Hash Functions, MACs and Key Derivation

Hash functions are the fingerprints of the digital world, MACs are seals that only key holders can apply, and key derivation functions turn one secret into many. Together they appear in nearly every system you will ever touch, from software downloads to password storage to the TLS key schedule. This chapter explains each, the attacks that shaped them, and how to choose correctly.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Explain the three security properties of hash functions and how SHA-1 was broken
- Describe Merkle-Damgård and sponge designs and the length extension attack
- Use HMAC correctly and compare MACs safely
- Choose the right key derivation or password hashing function for a given task

IN THIS CHAPTER

- | | |
|-----|--------------------------------|
| 6.1 | What a hash function is |
| 6.2 | The three security properties |
| 6.3 | The fall of MD5 and SHA-1 |
| 6.4 | How hash functions are built |
| 6.5 | Message authentication codes |
| 6.6 | Key derivation functions |
| 6.7 | Where you will see these tools |

6.1 What a hash function is

A **cryptographic hash function** takes input of any length and produces a fixed-length output, called a **digest** or **hash**. SHA-256, for example, always produces 256 bits (32 bytes), whether you hash one letter or a 10-gigabyte file.

```
SHA-256("hello") = 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
SHA-256("hello.") = 0b0a0e4a7c2a4f2b... (completely different)
```

A tiny change to the input changes about half of the output bits unpredictably. This is called the **avalanche effect**. There is no key: anyone can compute a hash. That makes hashes ideal for checking whether data has changed, but useless on their own for proving who created it.

6.2 The three security properties

A cryptographic hash must resist three kinds of attack. Learn them in order of difficulty for the attacker:

Property	The attacker's task	Generic cost for an n-bit hash	Why it matters
Preimage resistance	Given a hash, find any input that produces it	2^n	Hashes of secrets must not reveal the secret
Second-preimage resistance	Given an input, find a different input with the same hash	2^n	Nobody can swap a signed file for a malicious one with the same hash
Collision resistance	Find any two inputs with the same hash	$2^{(n/2)}$, by the birthday bound	Signatures and certificates sign hashes; a collision lets one signature cover two documents

Collision resistance is the hardest property to keep, because the attacker gets to choose both inputs. It is also the first property to fall when a hash function weakens.

6.3 The fall of MD5 and SHA-1

MD5 (1992) and **SHA-1** (1995) were the standard hashes for two decades. Their histories are a lesson in how cryptographic deprecation works.

- **2004:** researchers found practical MD5 collisions.
- **2008:** a team used MD5 collisions to create a rogue certificate authority certificate that browsers trusted, by getting a real CA to sign a harmless-looking certificate whose MD5 hash collided with a malicious one.
- **2012:** the Flame malware used an MD5 collision to forge a Microsoft code-signing certificate and spread through Windows Update.
- **2017:** Google and CWI Amsterdam announced **SHAttered**, the first real SHA-1 collision: two different PDF files with the same SHA-1 hash, computed with about 2^{63} operations.
- **2020:** the **Shambles** attack made chosen-prefix SHA-1 collisions cost about 45,000 US dollars of cloud computing, and demonstrated an impersonation attack on PGP keys.

Between the first theoretical weakness and practical exploitation, systems had over a decade of warning. Many still did not migrate in time. Today, MD5 and SHA-1 must not be used for signatures, certificates, or any purpose that needs collision resistance. NIST plans to withdraw SHA-1 entirely by 2030.

IN PRACTICE You will still find MD5 and SHA-1 used as non-security checksums (for example, to detect accidental file corruption) and in Git's original object IDs. Accidental corruption cannot exploit collisions, so those uses are tolerable, although Git is moving to SHA-256. Any use where an attacker could choose the input is not.

6.4 How hash functions are built

■ Merkle-Damgård: SHA-2

SHA-2 (SHA-224, SHA-256, SHA-384, SHA-512) uses the **Merkle-Damgård** construction. The message is padded and split into blocks. A compression function absorbs one block at a time into an internal state, and the final state is the hash.

This design has one surprising flaw: the **length extension attack**. Because the hash output *is* the final internal state, anyone who knows `SHA-256(secret || message)` and the length of the secret can compute `SHA-256(secret || message || padding || extra)` without knowing the secret. They simply resume hashing from the published state.

This breaks naive authentication schemes such as `tag = SHA-256(key || message)`, which some APIs used in the 2000s. An attacker could append parameters like `&admin=true` to a signed request and compute a valid tag. The fix is HMAC (section 6.5), or a hash immune to extension: SHA-512/256 (a truncated SHA-512) or SHA-3.

■ Sponges: SHA-3

SHA-3 was chosen through another NIST public competition and standardised in 2015 (FIPS 202). It uses a completely different design called a **sponge**: data is "absorbed" into a large internal state, which is then "squeezed" to produce output. Only part of the state is ever output, so length extension is impossible.

SHA-3 also provides **SHAKE128** and **SHAKE256**, called extendable-output functions (XOFs), which can produce output of any length. The post-quantum standards ML-KEM and ML-DSA use SHAKE heavily (Chapter 26).

Hash	Output	Collision security	Length extension	Notes
SHA-256	256 bits	128 bits	Vulnerable	The default for most uses
SHA-384	384 bits	192 bits	Resistant (truncated)	Required by some high-assurance profiles such as CNSA 2.0
SHA-512	512 bits	256 bits	Vulnerable	Faster than SHA-256 on 64-bit CPUs
SHA3-256	256 bits	128 bits	Resistant	Different design; a hedge if SHA-2 ever weakens
BLAKE2 / BLAKE3	Variable	128 bits or more	Resistant	Very fast; common in software, not FIPS-approved

6.5 Message authentication codes

A **message authentication code (MAC)** is like a hash that requires a key. Only someone holding the key can create a valid tag, and only someone holding the key can check it. MACs provide integrity and authentication between parties who share a secret.

```
tag = MAC(key, message)
verify: MAC(key, received_message) == received_tag
```

■ HMAC

HMAC (RFC 2104, FIPS 198-1) builds a MAC from any hash function by hashing twice with the key mixed in differently each time:

$$\text{HMAC}(K, m) = H((K \text{ XOR } \text{opad}) \parallel H((K \text{ XOR } \text{ipad}) \parallel m))$$

The nested structure defeats length extension, and HMAC remains secure even with hash functions whose collision resistance has weakened. HMAC-SHA256 is the most widely used MAC in the world: it signs API requests, protects session cookies and JWTs (Chapter 14), and forms the core of the TLS 1.3 key schedule.

Other MACs you will meet: **CMAC** (built from AES, common in payment and smart card systems), **GMAC** (the authenticator inside GCM), and **Poly1305** (paired with ChaCha20).

■ Compare tags in constant time

When verifying a MAC, never compare the received tag with the expected tag using ordinary string comparison. Ordinary comparison stops at the first mismatched byte, so the time it takes reveals how many leading bytes were correct. An attacker can discover a valid tag byte by byte by measuring response times. Every language offers a constant-time comparison: `hmac.compare_digest` in Python, `MessageDigest.isEqual` in Java, `crypto.timingSafeEqual` in Node.js, `subtle.ConstantTimeCompare` in Go.

6.6 Key derivation functions

A **key derivation function (KDF)** turns some input secret into one or more cryptographic keys. There are two very different kinds, and confusing them is a common mistake.

■ KDFs for high-entropy secrets: HKDF

When the input is already a strong secret, such as the shared value from a Diffie-Hellman exchange or a master key, you use a fast KDF. The standard is **HKDF** (RFC 5869), built from HMAC in two steps:

1. **Extract:** condense the input, which may have uneven randomness, into a uniform pseudo-random key.
2. **Expand:** produce as many keys as needed, each bound to a distinct **context label**.

```
prk      = HKDF-Extract(salt, shared_secret)
enc_key  = HKDF-Expand(prk, "app v1 encryption key", 32)
mac_key  = HKDF-Expand(prk, "app v1 MAC key", 32)
```

The labels guarantee that keys for different purposes are independent: knowing one reveals nothing about the others. This **key separation** principle is central to key management (Chapter 20). TLS 1.3 derives all of its traffic keys with HKDF and labels such as "c hs traffic" and "s ap traffic".

■ KDFs for passwords: deliberately slow

Passwords are the opposite of high-entropy secrets. Humans choose them, and attackers can guess billions per second using graphics cards. If you store `SHA-256(password)`, an attacker who steals the database can test every common password against every account almost instantly.

Password hashing functions defend against this in three ways:

1. A unique random **salt** per password, so identical passwords produce different hashes and precomputed tables are useless.
2. A tunable **work factor**, so each guess costs real time.
3. **Memory hardness**, so each guess also needs a lot of memory, which blunts the advantage of graphics cards and custom chips.

Function	Memory-hard	Status	Guidance
Argon2id	Yes	Winner of the 2015 Password Hashing Competition; RFC 9106	First choice for new systems
scrypt	Yes	Widely available	Good choice
bcrypt	Partly	Mature and common	Acceptable; limited to 72-byte passwords
PBKDF2-HMAC-SHA256	No	FIPS-approved	Use when FIPS is required, with a high iteration count (OWASP suggests 600,000)
Plain or salted SHA-256	No	Unsuitable	Never use for passwords

6.7 Where you will see these tools

Task	Tool
Check a download was not altered	SHA-256 digest, published over a trusted channel or signed
Sign a certificate or document	Hash with SHA-256 or SHA-384, then sign the hash (Chapter 9)
Authenticate API requests or webhooks	HMAC-SHA256 with a shared secret
Store user passwords	Argon2id, scrypt or bcrypt
Turn a shared secret into session keys	HKDF
Derive per-tenant or per-object keys from a master key	HKDF with distinct labels
Build tamper-evident logs and transparency logs	Hash chains and Merkle trees (Chapter 16)

HANDS-ON LAB

Lab: hashes, MACs and passwords

1. The avalanche effect.

```
echo -n "hello" | openssl dgst -sha256
echo -n "hello." | openssl dgst -sha256
```

2. A length extension attack, safely. The open-source tool `hash_extender` (or the Python package `hashpumpy`) demonstrates it: given `SHA-256(secret || "user=alice")` and the secret's length, it produces a valid hash for `"user=alice" + padding + "&admin=true"`. Then confirm that HMAC is immune:

```
import hmac, hashlib
key = b"server-secret"
tag = hmac.new(key, b"user=alice", hashlib.sha256).hexdigest()
# No extension trick produces a valid HMAC for "user=alice...&admin=true"
```

3. Feel the cost of password hashing.

```
import hashlib, time, os
salt = os.urandom(16)
t = time.time(); hashlib.sha256(b"Summer2026!").digest(); print("sha256", time.time()-t)
t = time.time(); hashlib.scrypt(b"Summer2026!", salt=salt, n=2**15, r=8, p=1);
print("scrypt", time.time()-t)
```

SHA-256 finishes in microseconds; scrypt takes tens of milliseconds and uses 32 MB of memory. A user logging in will not notice the difference. An attacker trying a billion guesses certainly will.

CASE STUDY

Failure story: the hash that forged a CA

In December 2008, at the Chaos Communication Congress, a team of researchers revealed that they had created a certificate authority certificate trusted by every major browser, without the CA's permission.

They exploited three things. A commercial CA still signed certificates using MD5. Its certificates had predictable serial numbers and validity dates. And MD5 was vulnerable to chosen-prefix collisions. The team prepared two certificates with the same MD5 hash: an ordinary website certificate, and a CA certificate with powers to sign any website. They bought the ordinary certificate. The CA's signature on it was equally valid for the colliding CA certificate. Computing the collision took a cluster of 200 PlayStation 3 consoles a few days.

The researchers deliberately set their rogue certificate to expire in the past so it could not be abused, and worked with browsers before disclosure. Within weeks, CAs stopped issuing MD5-signed certificates and began using random serial numbers, a defence that the CA/Browser Forum later made mandatory. Every certificate you see today contains at least 64 bits of randomness in its serial number because of this attack.

KEY TAKEAWAYS

Chapter summary

- Hash functions produce fixed-size fingerprints and must resist preimages, second preimages and collisions. Collision resistance is only half the output size.
- MD5 and SHA-1 are broken for collision resistance. Use SHA-256, SHA-384, or SHA-3.

- SHA-2 is vulnerable to length extension; never use `hash(key || message)` as a MAC. Use HMAC.
- MACs need shared keys and constant-time tag comparison.
- Use HKDF with distinct labels to derive keys from strong secrets, and Argon2id, `scrypt` or `bcrypt` for passwords.

CHECK YOURSELF

Review questions

1. Why is collision resistance for SHA-256 only 128 bits?
2. Explain the length extension attack and two ways to prevent it.
3. Why must MAC verification use constant-time comparison?
4. What is the difference between HKDF and Argon2id, and when would you use each?
5. Why are context labels important when deriving multiple keys from one secret?
6. Which three weaknesses combined in the 2008 rogue CA attack, and which permanent change did it cause in certificates?

PART III

Public-Key Cryptography

How strangers agree on secrets and prove who they are: RSA, Diffie-Hellman, elliptic curves, signatures and the proofs behind them.

In this part

- 7 RSA
- 8 Diffie-Hellman and Elliptic Curves
- 9 Digital Signatures
- 10 Security Definitions and Proofs, Made Practical

RSA

RSA was the first practical public-key system and is still everywhere: in certificates, code signing, smart cards and legacy protocols. It is also the algorithm most often used incorrectly. This chapter builds RSA from the mathematics of Chapter 2, explains why textbook RSA is insecure, and shows the padding schemes, key sizes and implementation rules that make it safe, for now.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Generate, use and break toy RSA keys and explain each step
- Explain why padding is essential and how OAEP and PSS work
- Choose RSA key sizes and recognise when RSA should be replaced
- Recognise famous RSA failures such as Bleichenbacher attacks and ROCA

IN THIS CHAPTER

- | | |
|-----|--|
| 7.1 | The idea |
| 7.2 | Generating a key pair |
| 7.3 | Textbook RSA is broken |
| 7.4 | Padding for encryption: PKCS#1 v1.5 and OAEP |
| 7.5 | Padding for signatures: PKCS#1 v1.5 and PSS |
| 7.6 | Key sizes and lifetime |
| 7.7 | Implementation pitfalls |

7.1 The idea

In 1977, Ron Rivest, Adi Shamir and Leonard Adleman published a system in which anyone could encrypt a message using a public key, but only the holder of the matching private key could decrypt it. The security rests on the factoring problem from Chapter 2: multiplying two large primes is easy, but recovering them from their product is not.

RSA works with ordinary modular exponentiation. A message, represented as a number m , is encrypted by raising it to a public power and decrypted by raising the result to a private power:

```
encrypt:   $c = m^e \bmod n$   
decrypt:   $m = c^d \bmod n$ 
```

The magic is that e and d are chosen so that the two powers cancel each other out mod n , and computing d from e requires knowing the factors of n .

7.2 Generating a key pair

1. Choose two large random primes, p and q , each half the desired key size. For a 3072-bit key, each is 1536 bits.
2. Compute the **modulus** $n = p \times q$. This is public.
3. Compute $\phi(n) = (p - 1)(q - 1)$. This must stay secret, because it reveals the factors.
4. Choose the **public exponent** e . Almost everyone uses $e = 65537$, which makes encryption and signature verification fast.
5. Compute the **private exponent** $d = e^{-1} \bmod \phi(n)$, using the extended Euclidean algorithm.

The **public key** is (n, e) . The **private key** is d , usually stored with p , q and a few precomputed values that speed up decryption using the Chinese Remainder Theorem (CRT), roughly four times faster.

Anyone who factors n can repeat step 5 and obtain d . That is why key size is everything.

7.3 Textbook RSA is broken

The formulas above, used directly, are called **textbook RSA**, and they are insecure in several ways:

- **Deterministic.** Encrypting the same message twice gives the same ciphertext. An attacker can guess likely messages ("yes", "no", "approve") and check by encrypting them with the public key.
- **Malleable.** Multiplying a ciphertext by $2^{e \bmod n}$ produces a valid encryption of $2m$. An attacker can transform encrypted values without decrypting them.
- **Small messages leak.** If m is small and e is small, m^e may be smaller than n , so no modular wrap-around happens and the attacker simply takes an ordinary e -th root.
- **Signatures can be forged.** With textbook signatures ($s = m^d \bmod n$), multiplying two valid signatures gives a valid signature on the product of their messages.

The solution to all of these is **padding**: a structured, randomised encoding applied to the message before exponentiation. Padding is not optional decoration. It is half of the security of RSA.

7.4 Padding for encryption: PKCS#1 v1.5 and OAEP

PKCS#1 v1.5 encryption padding (1993) wraps the message as `00 02 [random non-zero bytes] 00 [message]`. It adds randomness but has a fatal flaw, discovered by Daniel Bleichenbacher in 1998: if a server reveals, by error message or timing, whether a decrypted ciphertext had valid padding, an attacker can decrypt any ciphertext by sending a series of carefully chosen variations. This is the same pattern as the padding oracle in Chapter 5, with a different mathematical engine.

The attack has returned again and again: in 2014 against hardware tokens, in 2017 as **ROBOT** (Return Of Bleichenbacher's Oracle Threat), which affected TLS implementations from major vendors and some of the largest websites in the world, and in 2023 as the **Marvin** attack, which found timing variants in many libraries.

OAEP (Optimal Asymmetric Encryption Padding), standardised in PKCS#1 v2, mixes the message with randomness using hash-based masks in a way that comes with a security proof. Use **RSA-OAEP with SHA-256** whenever RSA encryption is unavoidable.

IN PRACTICE Modern protocols have largely stopped using RSA for encryption at all. TLS 1.3 removed RSA key transport entirely; RSA now appears in TLS only to sign the handshake. If a design proposes RSA encryption for new data, ask why it is not using ECDH or ML-KEM key exchange with symmetric encryption (Chapters 8 and 26).

7.5 Padding for signatures: PKCS#1 v1.5 and PSS

An RSA signature is created by hashing the message, padding the hash, and raising it to the private exponent:

$$s = \text{pad}(\text{hash}(m))^d \bmod n \quad \text{verify: } s^e \bmod n == \text{pad}(\text{hash}(m))$$

- **PKCS#1 v1.5 signature padding** is deterministic and, unlike its encryption counterpart, has no practical attack when verified strictly. It remains the most common RSA signature format in certificates. Loose verification has caused forgeries: in 2006, Bleichenbacher showed that verifiers which did not check the padding fully could be fooled with forged signatures when `e = 3`.
- **RSA-PSS** (Probabilistic Signature Scheme) adds randomness and has a strong security proof. It is required in TLS 1.3 for RSA handshake signatures and is the better choice for new designs.

7.6 Key sizes and lifetime

RSA modulus	Security level	Status
1024 bits	About 80 bits	Broken in practice for well-funded attackers; disallowed
2048 bits	112 bits	Today's minimum; acceptable until about 2030 under NIST guidance
3072 bits	128 bits	Recommended for new keys with long lifetimes
4096 bits	About 140 bits	Common for root CAs; slower

The largest RSA number publicly factored by classical computers is 829 bits (RSA-250, in 2020), which took about 2,700 core-years of computation. 2048-bit RSA is far beyond classical reach. It is not, however, beyond the reach of a large, error-corrected quantum computer running Shor's algorithm (Chapter 25). Estimates of the quantum resources needed keep falling: a 2025 analysis by Google researchers estimated that 2048-bit RSA

could be factored in under a week with fewer than one million noisy qubits, a twentyfold reduction from their 2019 estimate. No such machine exists, but the trend is why governments now set retirement dates for RSA.

CURRENT DEVELOPMENT NIST's draft transition plan proposes that 2048-bit RSA be deprecated after 2030 and all RSA be disallowed after 2035 for federal use. Larger RSA keys do not help against quantum attackers; the replacement is ML-DSA for signatures and ML-KEM for key establishment (Chapter 26).

7.7 Implementation pitfalls

Correct mathematics is not enough. RSA implementations have failed in these ways:

- **Weak prime generation.** In 2012, two research teams scanned the internet's public keys and found that tens of thousands of devices, mostly routers and firewalls generating keys at first boot with poor randomness, shared a prime factor with another device. A simple gcd computation between pairs of keys factored them instantly.
- **Structured primes.** The **ROCA** vulnerability (2017) found that a widely used security chip generated primes with a special structure that made factoring feasible, affecting an estimated hundreds of millions of keys in national ID cards, laptops' trusted platform modules and hardware tokens. Estonia had to replace the keys on 760,000 national ID cards.
- **Timing and fault attacks.** Decryption time can leak bits of d ; a single computing error during CRT decryption can reveal p . Libraries use blinding and verify results before releasing them.
- **Small or shared exponents.** Never use a small private exponent d , and never share a modulus n between users.

The rules for you are short: generate RSA keys only with mature libraries or certified hardware, use OAEP for encryption and PSS (or strictly verified PKCS#1 v1.5) for signatures, use at least 3072 bits for anything long-lived, and plan RSA's replacement.

HANDS-ON LAB

Lab: RSA by hand, then RSA done right

1. Toy RSA and its weaknesses (Python):

```

p, q, e = 1009, 1013, 65537
n, phi = p*q, (p-1)*(q-1)
d = pow(e, -1, phi)
m = 42
c = pow(m, e, n); print(pow(c, d, n))          # 42
# Malleability: double the plaintext without the key
c2 = (c * pow(2, e, n)) % n; print(pow(c2, d, n)) # 84
# Determinism: guess-and-check a "secret" vote
print(pow(1, e, n) == pow(1, e, n))          # same ciphertext every time

```

2. Generate and inspect a real key with OpenSSL.

```

openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:3072 -out rsa.key
openssl pkey -in rsa.key -text -noout | head -20 # modulus, exponent, primes
openssl pkey -in rsa.key -pubout -out rsa.pub

```

3. Sign with PSS and verify.

```

echo "release v2.4.1" > release.txt
openssl dgst -sha256 -sigopt rsa_padding_mode:pss -sign rsa.key -out release.sig release.txt
openssl dgst -sha256 -sigopt rsa_padding_mode:pss -verify rsa.pub -signature release.sig
release.txt

```

Change one character in `release.txt` and verify again. Verification fails.

4. The shared-prime attack in miniature.

```

from math import gcd
n1, n2 = 1009*1013, 1009*1019 # two "devices" that shared a prime
print(gcd(n1, n2))           # 1009: both keys factored instantly

```

CASE STUDY

Failure story: ROCA and the national ID cards

In October 2017, researchers from Masaryk University in the Czech Republic published **ROCA**. A cryptographic library built into chips from a major manufacturer generated RSA primes using a shortcut that made them faster to find. The shortcut left a hidden structure: every key generated by the chip could be recognised by a quick test, and its modulus factored using a specialised method. A 2048-bit key could be factored for an estimated 20,000 to 40,000 US dollars of cloud computing, and 1024-bit keys in hours.

The chip was certified to high security standards and used in national identity cards, trusted platform modules in business laptops, hardware security keys and smart cards. The library's design had never been published, so nobody outside the manufacturer could review it. Kerckhoffs's principle from Chapter 1 is not only about algorithms: secret implementation shortcuts can be just as dangerous as secret algorithms.

Recovery was expensive. Estonia suspended the certificates of 760,000 ID cards and asked citizens to update their keys. Laptop makers shipped firmware updates, and every key generated by an affected chip had to be replaced. The episode is a powerful argument for crypto agility and for knowing exactly which keys came from which hardware.

KEY TAKEAWAYS

Chapter summary

- RSA security rests on factoring. The private exponent d can be computed by anyone who knows the factors of n .
- Textbook RSA is deterministic and malleable. Padding is mandatory: OAEP for encryption, PSS or strictly verified PKCS#1 v1.5 for signatures.
- Bleichenbacher-style padding oracles have recurred for 25 years. Modern protocols avoid RSA encryption altogether.
- Use 3072-bit keys for long-lived uses; 2048-bit is the minimum and is scheduled for deprecation after 2030.
- Bad randomness and secret implementation shortcuts (shared primes, ROCA) have broken millions of RSA keys.

CHECK YOURSELF

Review questions

1. Why can anyone who factors n compute the private key?
2. List three weaknesses of textbook RSA.
3. What is a Bleichenbacher attack, and why does TLS 1.3 avoid it entirely?
4. When would you choose RSA-PSS over PKCS#1 v1.5 signatures?
5. Why does a larger RSA key not protect against quantum computers?
6. How did researchers factor tens of thousands of device keys in 2012 without solving factoring?

Diffie-Hellman and Elliptic Curves

How can two strangers, talking over a channel an attacker can read, agree on a secret key? Diffie-Hellman key exchange answered that question in 1976 and changed cryptography forever. This chapter explains the exchange, its elliptic curve form that secures almost every connection today, the forward secrecy it enables, and the subtle ways it fails when parameters or validation are wrong.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Walk through a Diffie-Hellman exchange and explain why an eavesdropper cannot compute the key
- Explain ephemeral keys and forward secrecy
- Compare finite-field DH, ECDH with NIST curves, and X25519
- Explain man-in-the-middle, small-subgroup and invalid-curve attacks and their defences

IN THIS CHAPTER

- | | |
|-----|------------------------------------|
| 8.1 | The problem Diffie-Hellman solved |
| 8.2 | The exchange, step by step |
| 8.3 | Elliptic curve Diffie-Hellman |
| 8.4 | Ephemeral keys and forward secrecy |
| 8.5 | The missing piece: authentication |
| 8.6 | How Diffie-Hellman fails |
| 8.7 | Diffie-Hellman as a building block |

8.1 The problem Diffie-Hellman solved

Before 1976, two parties could communicate securely only if they had already shared a secret key, delivered by courier, in person, or through some other trusted channel. For a world of billions of devices that meet each other for the first time every second, that does not scale.

Whitfield Diffie and Martin Hellman published a method that lets two parties create a shared secret over a public channel. (It later emerged that researchers at the UK's GCHQ had discovered the same idea secretly a few years earlier.) Their method is called **Diffie-Hellman (DH) key exchange**, and a version of it runs every time your browser opens an HTTPS connection.

8.2 The exchange, step by step

Both parties agree on public parameters: a large prime p and a generator g . Then:

Step	Alice	Public channel	Bob
1	Picks secret a		Picks secret b
2	Computes $A = g^a \bmod p$	A and B are exchanged	Computes $B = g^b \bmod p$
3	Computes $B^a = g^{(ab)}$		Computes $A^b = g^{(ab)}$

Both arrive at the same value, $g^{(ab)} \bmod p$, called the **shared secret**. An eavesdropper sees p , g , A and B . To compute the shared secret, they would need a or b , which means solving a discrete logarithm (Chapter 2).

A famous analogy uses paint. Alice and Bob each mix a secret colour into a shared public colour and swap the mixtures. Each then adds their own secret colour to the other's mixture, and both end up with the same final colour. An observer who sees the two mixtures cannot "unmix" them to find the secret colours.

The shared secret is not used directly as a key. It is fed into a key derivation function such as HKDF (Chapter 6) to produce the actual encryption keys.

8.3 Elliptic curve Diffie-Hellman

The same exchange works in any group where the discrete logarithm is hard. Using elliptic curves (Chapter 2) gives **Elliptic Curve Diffie-Hellman (ECDH)**:

- The public parameter is a curve and a base point G .
- Alice picks a secret number a and publishes the point $A = aG$. Bob publishes $B = bG$.
- Both compute the point abG .

Because the elliptic curve discrete logarithm is much harder, ECDH achieves 128-bit security with 256-bit keys, compared with 3072-bit values for finite-field DH. Exchanges are smaller and much faster.

X25519: the modern default

X25519, defined in RFC 7748, is ECDH on Curve25519, designed by Daniel Bernstein in 2005. It has become the default key exchange in TLS 1.3, SSH, WireGuard, Signal and most modern protocols, for reasons that are about safety as much as speed:

- **Every 32-byte string is a valid public key.** There is no need to validate points, which removes a whole class of attacks (section 8.6).
- **Constant-time by design.** The recommended implementation, called the Montgomery ladder, does the same operations for every key bit.
- **Transparent parameters.** The curve's constants were chosen by a documented, minimal process, which addresses suspicions about how some older curves were generated.

The NIST curves **P-256** and **P-384** remain widely used, are FIPS-approved, and are required by some government profiles. They are secure when implemented carefully, but they demand more care: incoming points must be validated.

Option	Public value size	Security	Strengths	Cautions
Finite-field DH (3072-bit)	384 bytes	128 bits	Simple mathematics	Large and slow; old parameters were weak
ECDH P-256	65 bytes (33 compressed)	128 bits	FIPS-approved, universal hardware support	Must validate points
X25519	32 bytes	About 128 bits	Fast, misuse-resistant, default in TLS 1.3	Not in older FIPS profiles (now allowed under SP 800-186)
ECDH P-384	97 bytes	192 bits	Required by CNSA 1.0	Slower

8.4 Ephemeral keys and forward secrecy

If Alice and Bob use the same DH secrets for every session, then anyone who later steals Alice's secret can decrypt every recorded past session. If instead they generate fresh secrets for each session and delete them afterwards, a later theft reveals nothing about the past. These one-time secrets are called **ephemeral keys**, and the resulting protection is called **forward secrecy** (sometimes "perfect forward secrecy").

Forward secrecy is one of the most important practical security properties in modern protocols:

- **TLS 1.2 with RSA key transport had no forward secrecy.** The browser encrypted the session key with the server's long-term RSA key. Anyone who recorded traffic and later obtained that private key, through a breach, a bug such as Heartbleed, or a legal demand, could decrypt everything ever recorded.
- **TLS 1.3 requires ephemeral (EC)DH** for every handshake. The server's long-term key is used only to sign the handshake, proving identity, never to protect the session key.

WATCH OUT Forward secrecy protects past sessions from future theft of long-term keys. It does not protect against a quantum computer that can break the recorded ephemeral exchange itself. That is why "harvest now, decrypt later" drives the urgent move to hybrid post-quantum key exchange (Chapter 27).

8.5 The missing piece: authentication

Diffie-Hellman alone has a fatal weakness: it does not tell Alice who she is talking to. An attacker in the middle can run one exchange with Alice and a separate exchange with Bob, relaying and reading everything. This is a **man-in-the-middle (MITM) attack**, and plain DH has no defence against it.

Real protocols therefore **authenticate** the exchange: one or both parties sign their DH public value, or the whole handshake transcript, with a long-term signing key whose public half is vouched for by a certificate (Part V) or known in advance (as with SSH host keys). The combination is called **authenticated key exchange**:

- Ephemeral DH provides a fresh shared secret and forward secrecy.
- Signatures with long-term keys prove identity.

Keeping these two roles separate is the design principle behind TLS 1.3, SSH, Noise and Signal.

8.6 How Diffie-Hellman fails

Weak and shared parameters

In 2015, the **Logjam** research showed that many servers still accepted 512-bit "export-grade" DH parameters, a leftover from 1990s US export controls, and that an attacker in the middle could force connections down to them. Worse, millions of servers used the same handful of 1024-bit primes. Because the expensive part of breaking DH can be precomputed once per prime, the researchers estimated that a nation-state could break a single common 1024-bit prime and then passively decrypt a large fraction of VPN and HTTPS traffic. TLS 1.3 now allows only standard, large, well-studied groups (RFC 7919 for finite fields), and only modern curves.

Small-subgroup and invalid-curve attacks

If an implementation does not check that a received public value is valid, an attacker can send a value that lies in a tiny subgroup, or, for elliptic curves, a point that is not on the agreed curve at all. Combining the victim's secret with such a value leaks a few bits of the secret each time, and repeated exchanges reveal it entirely. In 2015, researchers demonstrated invalid-curve attacks that extracted private keys from TLS servers built on several popular libraries.

The defences:

- **Validate every received public key** (on the curve, correct subgroup, not the identity point) for NIST curves and finite-field groups.
- Prefer **X25519**, where every input is safe by construction.
- Use **ephemeral keys**, so that leaked bits relate to a key that is about to be thrown away.

8.7 Diffie-Hellman as a building block

Beyond simple key exchange, DH appears inside many other constructions:

- **ECIES and HPKE**. Public-key encryption built from ECDH plus symmetric encryption. **HPKE** (Hybrid Public Key Encryption, RFC 9180) is the modern standard, used by Encrypted Client Hello and Messaging Layer Security.

- **Signal's X3DH and Double Ratchet** combine several DH exchanges to give messaging both forward secrecy and recovery after compromise (Chapter 13).
- **Key Encapsulation Mechanisms (KEMs)**. A KEM is an interface where one party "encapsulates" a fresh random key to another's public key. DH can be framed as a KEM, and the post-quantum ML-KEM is one (Chapter 26). This shared interface is what makes hybrid classical plus post-quantum key exchange straightforward.

HANDS-ON LAB

Lab: exchange keys yourself

1. A toy exchange in Python.

```
import secrets
p = 2**127 - 1          # a prime (far too small for real use)
g = 3
a, b = secrets.randbelow(p), secrets.randbelow(p)
A, B = pow(g, a, p), pow(g, b, p)
print(pow(B, a, p) == pow(A, b, p))  # True: same shared secret
```

2. X25519 with OpenSSL.

```
openssl genpkey -algorithm X25519 -out alice.key
openssl genpkey -algorithm X25519 -out bob.key
openssl pkey -in alice.key -pubout -out alice.pub
openssl pkey -in bob.key -pubout -out bob.pub
openssl pkeyutl -derive -inkey alice.key -peerkey bob.pub | xxd -p
openssl pkeyutl -derive -inkey bob.key -peerkey alice.pub | xxd -p
```

The two outputs are identical: both sides derived the same 32-byte secret without ever sending it.

3. See what your browser negotiates. Visit any major website, open the browser's developer tools, choose the Security tab, and look at the key exchange. Since late 2024, Chrome, Edge and Firefox negotiate **X25519MLKEM768** with most large sites: classic X25519 combined with a post-quantum KEM. Chapter 27 explains this hybrid.

CASE STUDY

Failure story: Logjam and the shared primes

In May 2015, a team of 14 researchers published "Imperfect Forward Secrecy: How Diffie-Hellman Fails in Practice". They found two problems.

First, 8.4 percent of the top one million HTTPS websites still supported 512-bit export-grade DH, and a flaw in TLS 1.2 let an attacker in the middle downgrade normal connections to it. The researchers broke 512-bit groups in about 90 seconds per connection after a week of precomputation per prime.

Second, and more troubling, the expensive precomputation for the number field sieve depends only on the prime, not on individual keys. Because a few standard 1024-bit primes were hard-coded into widely used software, the researchers estimated that breaking a single common prime, at a cost of hundreds of millions of US dollars in custom hardware, would allow passive decryption of connections to about 18 percent of the top one million HTTPS sites and a large share of VPN servers. They noted that this matched descriptions in leaked intelligence documents of large-scale VPN decryption capabilities.

Browsers raised minimum DH sizes immediately, and TLS 1.3 later removed custom DH groups altogether. The lesson reaches beyond DH: shared parameters concentrate risk, and "nobody would spend that much" is not a threat model when the prize is everyone's traffic.

KEY TAKEAWAYS

Chapter summary

- Diffie-Hellman lets two parties create a shared secret over a public channel, based on the hardness of discrete logarithms. The secret is turned into keys with a KDF.
- ECDH gives the same exchange with much smaller, faster keys. X25519 is the modern default because it is fast and hard to misuse.
- Ephemeral keys give forward secrecy: stealing a long-term key later does not expose past sessions. TLS 1.3 requires it.
- DH alone is open to man-in-the-middle attacks. Authenticated key exchange adds signatures.
- Weak or shared parameters (Logjam) and unvalidated public keys (invalid-curve attacks) have broken real deployments.

CHECK YOURSELF

Review questions

1. In your own words, why can an eavesdropper not compute the Diffie-Hellman shared secret?
2. What is forward secrecy, and why did TLS 1.2 with RSA key transport lack it?
3. Describe a man-in-the-middle attack on unauthenticated DH, and how protocols prevent it.
4. Give two reasons X25519 is considered safer to implement than ECDH on P-256.
5. Why did shared 1024-bit primes make large-scale decryption economically plausible?
6. What is a KEM, and why does the KEM interface matter for post-quantum migration?

Digital Signatures

A digital signature lets anyone confirm that a message came from the holder of a particular private key and has not been changed since. Signatures secure software updates, certificates, documents, code, transactions and the TLS handshake itself. This chapter explains how the major signature schemes work, why ECDSA is so unforgiving, and how to choose and operate signature keys safely.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Explain what signatures guarantee and what they do not
- Compare RSA, ECDSA and EdDSA signatures and their failure modes
- Explain why ECDSA nonces must never repeat or leak and how deterministic signing fixes this
- Apply signatures correctly to software, documents and protocols

IN THIS CHAPTER

- | | |
|-----|------------------------------|
| 9.1 | What a signature guarantees |
| 9.2 | Hash-then-sign |
| 9.3 | RSA signatures |
| 9.4 | ECDSA |
| 9.5 | EdDSA and Ed25519 |
| 9.6 | Signatures in the real world |
| 9.7 | Operating signature keys |

9.1 What a signature guarantees

A **digital signature** scheme has three operations:

```
keypair    = KeyGen()
signature  = Sign(private_key, message)
valid      = Verify(public_key, message, signature)  # true or false
```

A valid signature proves three things:

1. **Authenticity.** It was produced by someone holding the private key.
2. **Integrity.** The message has not changed since it was signed. Changing a single bit makes verification fail.
3. **Non-repudiation.** Because only the private key holder could have produced it, the signer cannot plausibly deny it later, assuming the key was properly protected.

Signatures differ from MACs (Chapter 6) in one crucial way. A MAC uses a shared secret, so either party could have created the tag. A signature uses a private key held by one party, and anyone with the public key can verify it. This makes signatures suitable for publishing: one signature on a software update can be verified by a billion devices that hold only the public key.

What a signature does not guarantee

- **That the public key belongs to who you think.** A signature proves that the holder of *some* key signed. Linking that key to an identity requires a certificate or another trusted source (Part V).
- **That the signer understood or intended the content.** Signing systems that show users one thing and sign another have been abused.
- **When the signature was made.** Without a trusted timestamp, a signature does not prove it was created before a key was compromised or expired. Long-lived signatures, such as code signing, use **timestamping authorities** for this.
- **Confidentiality.** Signed messages are readable by anyone.

9.2 Hash-then-sign

Signature algorithms work on short, fixed-size values. Every practical scheme first hashes the message, then signs the hash. Two consequences follow:

- The signature is only as strong as the hash. If an attacker can find two messages with the same hash, a signature on one is a signature on the other. This is exactly how MD5 collisions produced a rogue CA certificate (Chapter 6).
- Large files can be signed cheaply. Signing a 4-gigabyte disk image costs the same as signing a sentence, plus the time to hash it.

9.3 RSA signatures

Chapter 7 covered RSA signatures: pad the hash with PKCS#1 v1.5 or PSS, then exponentiate with the private key. RSA signatures are large (384 bytes for a 3072-bit key), slow to create, but very fast to verify, which suits certificates verified many times. They remain the most common signature in the web PKI and in code signing, but are scheduled for retirement with the rest of classical public-key cryptography.

9.4 ECDSA

The **Elliptic Curve Digital Signature Algorithm (ECDSA)** is the elliptic curve version of the older DSA standard. It gives 128-bit security with 64-byte signatures on P-256, and is used in TLS certificates, code signing, smart cards, cryptocurrency and countless devices.

Signing works like this, with private key d and public key $Q = dG$:

1. Hash the message to get z .
2. Choose a secret, single-use random number k , called the **nonce**.
3. Compute the point kG and take its x-coordinate, mod the curve order, as r .
4. Compute $s = k^{-1}(z + r \cdot d) \bmod n$.
5. The signature is the pair (r, s) .

Step 2 is the dangerous one. The nonce k must be:

- **Secret.** If k is ever revealed, the private key follows immediately: $d = (s \cdot k - z)/r$.
- **Never repeated.** If the same k signs two different messages, the two equations can be solved together to find k , and then d . This is how the PlayStation 3 master key was recovered (Chapter 3).
- **Unbiased.** Even if a few bits of k are predictable, or leak through timing, lattice techniques combine many signatures to recover the key. The 2019 **Minerva** and

TPM-Fail attacks recovered keys from smart cards and trusted platform modules this way, using timing leaks of just a few bits per signature.

WATCH OUT ECDSA is unusual among cryptographic tools in that a small flaw in one signature, not in the key, can reveal the long-term private key. Any ECDSA implementation that draws k from a random number generator inherits that generator's every weakness.

Deterministic ECDSA

RFC 6979 removes the dependence on randomness by deriving k from the private key and the message hash using HMAC. The same message always gives the same signature, k is unpredictable to anyone without the private key, and a broken random number generator can no longer leak the key. Many libraries now use deterministic or "hedged" nonces (deterministic plus some fresh randomness, which also helps against fault attacks).

9.5 EdDSA and Ed25519

EdDSA, and its most common instance **Ed25519** (RFC 8032), was designed to avoid ECDSA's sharp edges:

- **Deterministic by design.** The nonce is derived from a hash of a secret key component and the message. There is no random number to get wrong.
- **Complete, constant-time formulas** on Edwards curves, with no special cases for an attacker to trigger.
- **Fast and small:** 32-byte public keys, 64-byte signatures, and verification fast enough to check thousands of signatures per second per core.

Ed25519 is the default for SSH keys, is used by Signal, Tor, WireGuard and many software signing systems, and was added to FIPS 186-5 in 2023, making it acceptable in US federal systems.

Scheme	Signature size	Security	Nonce risk	Typical use
RSA-3072 PSS	384 bytes	128 bits	None (PSS randomness is not key-critical)	Certificates, code signing, legacy
ECDSA P-256	64 bytes	128 bits	Severe unless deterministic	TLS certificates, smart cards, devices
Ed25519	64 bytes	About 128 bits	None by design	SSH, modern protocols, software signing

Scheme	Signature size	Security	Nonce risk	Typical use
ML-DSA-65	3,309 bytes	Category 3 (post-quantum)	Hedged by design	Post-quantum signatures (Chapter 26)

9.6 Signatures in the real world

Software and code signing

Operating systems and package managers verify signatures before installing software. Code signing keys are among the most valuable keys any organisation holds: a stolen code signing key lets malware run as trusted software. The 2020 compromise of a network monitoring vendor's build system, which inserted a backdoor into signed updates delivered to about 18,000 customers, showed that the signature only proves what the build system produced, not that the build was clean. Modern supply-chain practice adds **reproducible builds**, signed build provenance, and transparency logs.

Sigstore, an open-source project launched in 2021, popularised **keyless signing**: a developer proves their identity through an existing login, receives a certificate valid for about ten minutes, signs, and the signature is recorded in a public transparency log. There is no long-lived signing key to steal. Major open-source package ecosystems have adopted it for publishing provenance.

Documents and long-term signatures

Signed contracts and records may need to be verified decades later, long after the signing certificate expired and perhaps after the algorithm weakened. Standards such as PAdES (for PDFs) solve this with trusted timestamps and by periodically adding fresh timestamps over the old signatures, so that the evidence chain is always protected by a current algorithm.

Protocols

In TLS 1.3, the server signs a hash of the entire handshake transcript with its certificate key, proving it holds the private key and binding the key exchange to its identity. In SSH, the server signs the exchange with its host key. These signatures are created millions of times per second across the internet, which is why fast schemes like ECDSA and Ed25519 dominate there.

9.7 Operating signature keys

Signature keys need different handling from encryption keys:

- **Value scales with what trusts them.** A root CA key, a firmware signing key or a package repository key may be trusted by millions of devices. Protect such keys in hardware security modules, with multi-person controls (Chapter 23).
- **Separate keys by purpose.** Never use one key for both signing and encryption, and never reuse a TLS server key for code signing.
- **Plan for rotation.** Devices that trust a single hard-coded signing key cannot be fixed if that key leaks. Ship devices with the ability to trust new keys, signed by the old ones.
- **Log every use.** For high-value keys, every signature should be recorded and reviewable, so a misuse is detected.
- **Use timestamps** for signatures that must outlive the signing certificate.

HANDS-ON LAB

Lab: sign, verify and break ECDSA

1. Ed25519 and ECDSA with OpenSSL.

```
openssl genpkey -algorithm ED25519 -out ed.key
openssl pkey -in ed.key -pubout -out ed.pub
echo "firmware v7" > fw.bin
openssl pkeyutl -sign -inkey ed.key -rawin -in fw.bin -out fw.sig
openssl pkeyutl -verify -pubin -inkey ed.pub -rawin -in fw.bin -sigfile fw.sig
```

2. Recover an ECDSA key from a repeated nonce (Python, with the `ecdsa` package: `pip install ecdsa`):

```

from ecdsa import SigningKey, NIST256p
from ecdsa.util import sigdecode_string
import hashlib

sk = SigningKey.generate(curve=NIST256p); n = NIST256p.order
k = 123456789                                # the fatal mistake: a fixed nonce
def sign(m):
    sig = sk.sign(m, k=k, hashfunc=hashlib.sha256)
    return sigdecode_string(sig, n), int.from_bytes(hashlib.sha256(m).digest(), "big")
(r, s1), z1 = sign(b"message one")
(_, s2), z2 = sign(b"message two")
k_found = ((z1 - z2) * pow(s1 - s2, -1, n)) % n
d = ((s1 * k_found - z1) * pow(r, -1, n)) % n
print(d == sk.privkey.secret_multiplier)      # True: private key recovered

```

Two signatures, a few lines of algebra, and the private key is gone. This is why deterministic nonces matter.

CASE STUDY

Failure story: a stolen signing key and a crash dump

In July 2023, Microsoft disclosed that a threat actor it named Storm-0558 had accessed the email accounts of about 25 organisations, including US government agencies. The attackers forged authentication tokens accepted by Microsoft's cloud email service. They could do this because they had obtained a **consumer account signing key**, created in 2016, that should only have been able to sign tokens for personal accounts.

Two failures combined. First, the private signing key left its protected environment: Microsoft's investigation suggested it ended up in a crash dump that was moved into a less-protected debugging environment and later reached a compromised engineer's account, although Microsoft later said it could not confirm this path with certainty. Second, a validation flaw meant enterprise email systems accepted tokens signed by the consumer key, so one key could forge access far beyond its intended scope.

The US Cyber Safety Review Board's 2024 report called the intrusion preventable and criticised the long-lived key, which should have been rotated years earlier. The lessons map directly onto this chapter: keep signing keys in hardware where they cannot be copied into dumps, scope keys narrowly and verify scope strictly, rotate keys on a schedule, and keep an inventory of which keys are trusted by what.

KEY TAKEAWAYS

Chapter summary

- Signatures provide authenticity, integrity and non-repudiation, verifiable by anyone with the public key. They do not provide confidentiality, identity binding or timing on their own.
- All practical schemes hash then sign, so signatures are only as strong as their hash.
- ECDSA leaks its private key if a nonce is repeated, revealed or slightly biased. Use deterministic nonces (RFC 6979) or Ed25519.
- Ed25519 is fast, compact and safe by design, and is now FIPS-approved.
- Signing keys are among the most valuable secrets an organisation holds: protect them in hardware, scope them narrowly, rotate them and log their use.

CHECK YOURSELF

Review questions

1. What is the key difference between a MAC and a digital signature, and why does it matter for software updates?
2. Why is a signature only as strong as the hash it signs?
3. Explain, without equations, why a repeated ECDSA nonce reveals the private key.
4. Give two design features that make Ed25519 safer than ECDSA.
5. What problem does a trusted timestamp solve for long-lived signatures?
6. In the Storm-0558 case, which two independent failures let one key compromise enterprise email?

Security Definitions and Proofs, Made Practical

When a cryptographer says an algorithm is "proven secure", the claim has a precise meaning that is easy to misread. This chapter teaches you to read security claims like a professional: what the standard definitions say, how security proofs work as games and reductions, what assumptions hide inside them, and why proven-secure systems still get broken.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Describe IND-CPA, IND-CCA, EUF-CMA and related definitions in plain words
- Explain security games and reductions
- Explain the random oracle model and its limits
- Read a vendor or paper security claim and identify its assumptions and gaps

IN THIS CHAPTER

- | | |
|------|---------------------------------------|
| 10.1 | Why definitions matter |
| 10.2 | Security as a game |
| 10.3 | Definitions for encryption |
| 10.4 | Definitions for signatures and MACs |
| 10.5 | Definitions for building blocks |
| 10.6 | Reductions: how proofs work |
| 10.7 | The random oracle model |
| 10.8 | Why proven-secure systems still fail |
| 10.9 | Reading a security claim: a checklist |

10.1 Why definitions matter

For most of history, ciphers were judged by intuition: a design looked complicated, so people assumed it was strong. Many were broken. Modern cryptography, from the 1980s onward, insisted on two things before calling anything secure:

1. **A precise definition** of what "secure" means: what the attacker can do, and what counts as a win.
2. **Evidence** that no efficient attacker can win, usually a **proof** that winning would require solving a problem believed to be hard.

You do not need to write proofs. But you will constantly read claims like "IND-CCA2 secure", "EUF-CMA secure", "provably secure in the random oracle model" or "secure under the Module-LWE assumption". This chapter teaches you what those phrases promise and what they leave out.

10.2 Security as a game

Cryptographers define security as a **game** between a **challenger**, who holds the secret key, and an **adversary**, who is any efficient algorithm. The game spells out exactly which questions the adversary may ask and what the adversary must achieve to win. A scheme is secure if every efficient adversary wins with probability only negligibly better than guessing.

This framing is powerful because it does not list specific attacks. It covers **every** attack, including ones nobody has invented yet, as long as the attacker fits the model.

10.3 Definitions for encryption

IND-CPA: indistinguishability under chosen-plaintext attack

The game:

1. The adversary may ask for the encryption of any messages it likes.
2. It then picks two messages, m_0 and m_1 , of equal length.
3. The challenger secretly flips a coin and encrypts one of them.
4. The adversary may ask for more encryptions, then guesses which message was encrypted.

The scheme is **IND-CPA secure** if no efficient adversary guesses correctly noticeably more than half the time. In plain words: **ciphertexts reveal nothing about which message was encrypted, even to an attacker who can encrypt messages of their choice.**

One immediate consequence: deterministic encryption can never be IND-CPA secure, because the adversary simply encrypts m_0 itself and compares. This is the formal reason textbook RSA and ECB mode fail (Chapters 4 and 7), and why encryption needs IVs, nonces or randomness.

■ IND-CCA: adding chosen-ciphertext attacks

IND-CCA2 (often just "IND-CCA") adds a decryption oracle: the adversary may also ask for the decryption of any ciphertext except the challenge itself. This models attackers who can submit modified ciphertexts and observe the result, which is exactly the padding oracle and Bleichenbacher attacks of Chapters 5 and 7.

IND-CCA2 is the standard required for public-key encryption and key encapsulation used in real protocols. ML-KEM, for example, is designed to be IND-CCA2 secure.

■ Authenticated encryption: INT-CTXT

For AEAD (Chapter 5), confidentiality is paired with **integrity of ciphertexts (INT-CTXT)**: the adversary must not be able to produce any new ciphertext that decrypts successfully. A scheme that is IND-CPA and INT-CTXT secure is automatically IND-CCA secure, which is the formal reason Encrypt-then-MAC and AEAD work.

10.4 Definitions for signatures and MACs

EUF-CMA (existential unforgeability under chosen-message attack) is the standard for signatures and MACs:

1. The adversary may ask for signatures on any messages it chooses.
2. It wins if it outputs a valid signature on **any** message it did not ask about.

"Existential" means the forged message can be anything, even gibberish. This is deliberately strict: if an attacker can forge even meaningless messages, a real attacker will find meaningful ones. A stronger variant, **SUF-CMA** (strong unforgeability), also forbids producing a new signature on a message already signed. Some protocols, such as those that use signatures as identifiers, need the stronger property.

10.5 Definitions for building blocks

Primitive	Security notion	Plain meaning
Block cipher	PRP (pseudo-random permutation)	Indistinguishable from a random shuffle of blocks
MAC, PRF, KDF	PRF (pseudo-random function)	Outputs look random to anyone without the key
Hash function	Collision, preimage, second-preimage resistance	No efficient way to find these (Chapter 6)
Key exchange	Key indistinguishability	The agreed key looks random to an eavesdropper, even after other session keys leak
Random generator	Next-bit unpredictability	No efficient way to predict the next output bit

10.6 Reductions: how proofs work

Security proofs are almost always **reductions**. A reduction says:

If there were an efficient adversary that breaks scheme S , we could use it as a subroutine to build an efficient algorithm that solves hard problem P .

Since we believe P is hard, no such adversary should exist. For example, a proof might show that breaking a KEM implies solving the Module-LWE problem, or that forging HMAC implies breaking the underlying compression function as a PRF.

Three details of reductions matter in practice:

- **The assumption is the foundation.** A proof is only as good as the problem it reduces to. Proofs based on factoring or discrete logarithms become worthless against quantum computers, even though the proofs themselves are correct.
- **Tightness.** A reduction may lose a factor: "breaking S with probability ϵ implies solving P with probability $\epsilon / 2^{40}$ ". A loose reduction means parameters must be larger than they look to keep the promised security. Some deployed schemes have notably loose proofs.
- **The model.** The proof covers only attackers who behave as the game allows. An attacker who measures timing, injects faults, or exploits a parsing bug is outside the model.

10.7 The random oracle model

Many efficient schemes, including RSA-OAEP, RSA-PSS, Ed25519 and parts of ML-KEM, are proven secure in the **random oracle model (ROM)**. The proof pretends that a hash function is a perfect random function: a magic box that returns truly random output for every new input.

Real hash functions are not random oracles, and in 1998 researchers constructed (artificial) schemes that are secure in the ROM but insecure with any real hash function. So a ROM proof is weaker than a proof in the **standard model**, which makes no such idealisation. In practice, ROM proofs have an excellent track record for natural designs: no widely used scheme with a ROM proof has been broken through the gap between hashes and random oracles. Treat ROM proofs as strong evidence, not a guarantee.

A related idealisation, the **quantum random oracle model (QROM)**, lets quantum attackers query the hash in superposition. Post-quantum schemes need proofs in this model, which is technically harder and one reason post-quantum security analysis took years.

10.8 Why proven-secure systems still fail

Proofs are about mathematical models. Real systems break for reasons outside the model:

Gap	Example
Side channels	Timing leaks of a few nonce bits broke ECDSA in smart cards (Chapter 9)
Implementation bugs	Heartbleed leaked memory, including keys, from a correct protocol (Chapter 12)
Misuse	Nonce reuse destroys GCM's proven security (Chapter 5)
Wrong definition	A scheme proven IND-CPA used where IND-CCA was needed
Composition	Two individually secure components combined insecurely (MAC-then-encrypt in old TLS)
Broken assumption	Quantum computers break the problems behind RSA and ECC proofs
Model mismatch	Protocol proofs that ignored real-world details such as session resumption

This is why mature engineering combines proofs with public scrutiny, formal verification of protocols and code, careful implementation, and conservative parameters.

10.9 Reading a security claim: a checklist

When a vendor, paper or colleague says something is secure, ask:

- ☐ **Secure in what sense?** Which definition: IND-CPA, IND-CCA2, EUF-CMA, something else? Is it the right one for the use?
- ☐ **Under what assumption?** Factoring, discrete log, lattices, an idealised hash? Does it survive quantum attackers?
- ☐ **Proved, or argued?** Is there a published proof, ideally peer-reviewed or machine-checked?
- ☐ **What is outside the model?** Side channels, faults, misuse, key management.
- ☐ **What parameters?** Do the chosen key sizes actually give the claimed security, given the reduction's tightness?
- ☐ **Who has attacked it?** Years of public cryptanalysis count for more than any marketing claim.

CASE STUDY

Failure story: when the proof assumed away the attack

TLS 1.2 and earlier used **MAC-then-encrypt**: compute a MAC over the plaintext, append it, add padding, then encrypt with CBC. Analyses in the early 2000s gave this composition security arguments under certain assumptions. Implementations, however, had to check padding and MAC in some order, and the time taken differed depending on where the check failed.

In 2013, Nadhem AlFardan and Kenny Paterson published **Lucky Thirteen**, a timing attack that recovered plaintext from TLS connections by measuring differences of about a microsecond in how long servers took to reject malformed records. The difference came from how many hash compression blocks were processed, which depended on the padding length. Every major TLS library was affected and had to be patched with painstaking constant-time code.

The mathematics had not been wrong. The model had not included time. TLS 1.3 removed the problem at its root by allowing only AEAD ciphers, whose security definitions cover tampering directly and whose implementations naturally reject altered records before revealing anything.

KEY TAKEAWAYS

Chapter summary

- Modern cryptography defines security as a game against any efficient adversary, then proves no such adversary can win, assuming a hard problem.
- IND-CPA means ciphertexts reveal nothing; IND-CCA2 adds resistance to tampering-and-observing; EUF-CMA means no forgeries, even of meaningless messages.
- Proofs are reductions to assumptions. Check the assumption, the tightness, and the model.
- Random oracle proofs idealise hash functions; they are strong evidence with an excellent record, not a guarantee.
- Real breaks happen outside the model: side channels, bugs, misuse, composition and broken assumptions.

CHECK YOURSELF

Review questions

1. Why can no deterministic encryption scheme be IND-CPA secure?
2. Which real attacks does the IND-CCA2 definition capture that IND-CPA does not?
3. Why is EUF-CMA defined to include forgeries of meaningless messages?
4. What does it mean for a reduction to be "loose", and why does it matter for key sizes?
5. Name three ways a scheme with a valid security proof can still be broken in practice.
6. Apply the checklist in section 10.9 to the claim "our product uses military-grade 256-bit encryption".

PART IV

Protocols

How primitives are assembled into the secure channels and identity systems the world runs on, and how those designs fail.

In this part

- 11 Building a Secure Channel with TLS 1.3

- 12 Deploying TLS Well

- 13 Beyond TLS: SSH, VPNs and Secure Messaging

- 14 Identity Protocols: OAuth, OpenID Connect, JWT and Passkeys

Building a Secure Channel with TLS 1.3

Transport Layer Security protects most of the traffic on the internet, from web browsing to API calls to email between servers. TLS 1.3, published in 2018, is the cleanest and safest version ever designed. This chapter walks through a TLS 1.3 connection message by message, explains the key schedule, and covers resumption, 0-RTT, Encrypted Client Hello and the post-quantum key exchange now used by default.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Describe each message of the TLS 1.3 handshake and what it achieves
- Explain how the key schedule derives traffic keys with HKDF
- Explain session resumption and the replay risk of 0-RTT data
- Describe Encrypted Client Hello and hybrid post-quantum key exchange in TLS

IN THIS CHAPTER

- | | |
|------|--|
| 11.1 | What a secure channel must provide |
| 11.2 | A short history of TLS |
| 11.3 | The TLS 1.3 handshake |
| 11.4 | The key schedule |
| 11.5 | Resumption and 0-RTT |
| 11.6 | Hiding the server name: Encrypted Client Hello |
| 11.7 | Post-quantum key exchange in TLS |
| 11.8 | Mutual TLS |

11.1 What a secure channel must provide

A **secure channel** gives two parties, usually a client and a server, three things over an untrusted network:

1. **Confidentiality:** nobody else can read the data.
2. **Integrity:** nobody can change, reorder, drop or replay records without detection.
3. **Authentication:** at least the server, and optionally the client, proves its identity.

TLS provides all three by combining the tools from Parts II and III: ephemeral (EC)DH for key agreement, signatures and certificates for authentication, HKDF for key derivation, and AEAD for protecting data.

TLS has two layers:

- The **handshake protocol** negotiates parameters, authenticates the server (and optionally the client), and establishes shared keys.
- The **record protocol** splits application data into records and protects each one with AEAD, using a sequence number as part of the nonce so that records cannot be reordered or replayed.

11.2 A short history of TLS

Version	Year	Status
SSL 2.0 and 3.0	1995 to 1996	Broken (POODLE, DROWN); prohibited
TLS 1.0 and 1.1	1999, 2006	Deprecated by RFC 8996 in 2021; removed from browsers in 2020
TLS 1.2	2008	Still widely supported; secure only with modern settings
TLS 1.3	2018 (RFC 8446)	The current standard; used by the large majority of web traffic

TLS 1.3 was designed after two decades of attacks on earlier versions. Its designers removed everything that had caused trouble: RSA key transport (no forward secrecy, Bleichenbacher attacks), CBC mode and MAC-then-encrypt (padding oracles, Lucky Thirteen), static DH, compression (CRIME), renegotiation, weak hashes, and custom DH groups (Logjam). What remains is small enough to analyse formally, and TLS 1.3 was the first major protocol to be designed alongside machine-checked security proofs.

11.3 The TLS 1.3 handshake

A full TLS 1.3 handshake completes in **one round trip**, compared with two in TLS 1.2.

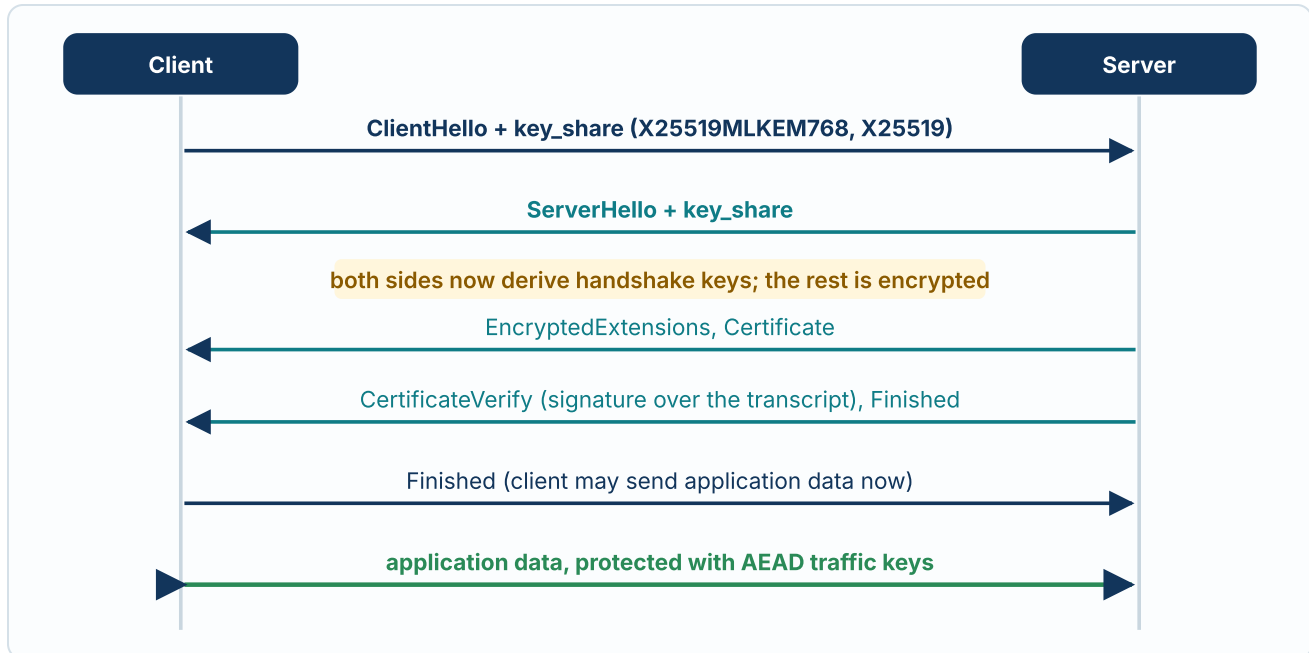


Figure 11.1 The TLS 1.3 full handshake. Everything after the ServerHello is encrypted, including the server's certificate.

ClientHello. The client sends a random value, the TLS versions and cipher suites it supports, and, crucially, one or more **key shares**: ephemeral public keys for the key exchange groups it guesses the server will accept. It also sends extensions, including **Server Name Indication (SNI)** naming the website it wants, and **ALPN**, which names the application protocol (such as HTTP/2).

ServerHello. The server picks a cipher suite and one of the client's key shares, and replies with its own ephemeral key share. At this moment, both sides can compute the shared secret, and everything that follows is encrypted.

EncryptedExtensions, Certificate, CertificateVerify. The server sends its certificate chain, then a **CertificateVerify** message: a signature, using its certificate's private key, over a hash of every handshake message so far. This proves the server holds the private key and binds its identity to this exact key exchange, defeating man-in-the-middle attacks.

Finished. Each side sends a MAC over the whole transcript using a key derived from the handshake secret. This confirms that both sides saw identical messages, so an attacker cannot have silently removed or altered anything, such as downgrading the offered algorithms.

If the server does not support any of the groups the client guessed, it sends a **HelloRetryRequest** naming a group it does support, and the client tries again, costing an extra round trip.

Cipher suites in TLS 1.3

TLS 1.3 cipher suites name only the AEAD and the hash; key exchange and signatures are negotiated separately. There are just five, and three are used in practice:

Cipher suite	Use
TLS_AES_128_GCM_SHA256	Mandatory; most common
TLS_AES_256_GCM_SHA384	Where AES-256 is required
TLS_CHACHA20_POLY1305_SHA256	Devices without AES hardware

11.4 The key schedule

TLS 1.3 derives every key from the shared secret through a **key schedule** built entirely on HKDF (Chapter 6). It runs in three stages, each mixing in more secret material and the transcript hash so far:

1. **Early secret.** From a pre-shared key if resuming a session, otherwise from zeros.
2. **Handshake secret.** Mixes in the (EC)DH or KEM shared secret. Produces keys that encrypt the rest of the handshake.
3. **Master secret.** Produces the **application traffic keys**, separate for each direction.

Each key is derived with a distinct label ("c hs traffic", "s ap traffic", "exp master" and so on) and the transcript hash, so keys for different purposes and different connections are independent. Keys can also be **updated** during long connections with a KeyUpdate message, limiting how much data any single key protects.

IN PRACTICE For debugging, most TLS libraries can write the derived secrets to a **key log file** (the `SSLKEYLOGFILE` environment variable in browsers and curl). Wireshark uses it to decrypt captured traffic. Never enable this on production systems: anyone with the file can read the traffic.

11.5 Resumption and 0-RTT

Full handshakes cost public-key operations and a round trip. TLS 1.3 lets clients **resume** a previous session:

- After a handshake, the server sends one or more **session tickets** containing a pre-shared key (PSK) for future connections.
- On reconnection, the client presents the ticket. The server can skip certificate verification, and the handshake can still include a fresh ECDH exchange to preserve forward secrecy (the recommended "psk_dhe_ke" mode).

0-RTT early data and replay

TLS 1.3 also allows a resuming client to send application data in its very first message, before the handshake completes. This **0-RTT** (zero round trip) data makes repeat visits faster, but it has a serious weakness: **it can be replayed**. An attacker who captures the client's first flight can send it to the server again, and the server may process the request twice.

The rule is simple: **only allow 0-RTT for requests that are safe to repeat**, such as fetching a public page. Never accept 0-RTT for payments, state changes or anything non-idempotent. Many servers disable 0-RTT entirely, and that is a sensible default.

11.6 Hiding the server name: Encrypted Client Hello

Even with TLS 1.3, one important piece of information remains visible to network observers: the **SNI**, which names the website the user is visiting. Network operators, censors and attackers can see it.

Encrypted Client Hello (ECH) fixes this. The server publishes an ECH public key in DNS (in an HTTPS resource record). The client encrypts its real ClientHello, including the SNI, using HPKE (Chapter 8), and wraps it inside an outer ClientHello that names only a shared front-end, such as a content delivery network. Observers see that you connected to the CDN, not which of its thousands of sites you visited.

CURRENT DEVELOPMENT ECH was published as RFC 9849 in March 2026, after years of drafts. Major browsers have supported it since 2023 and 2024, and large CDNs enable it for hosted sites. It depends on encrypted DNS: if DNS lookups are visible, the site name leaks there instead.

11.7 Post-quantum key exchange in TLS

The ephemeral ECDH exchange in TLS is vulnerable to "harvest now, decrypt later": an attacker who records traffic today could decrypt it once a large quantum computer exists. TLS 1.3's design makes the fix straightforward, because the key exchange is just a named group carried in the key share.

The new group **X25519MLKEM768** combines classical X25519 with the post-quantum KEM ML-KEM-768 (Chapter 26). Both shared secrets are concatenated and fed into the key schedule, so the connection is secure as long as **either** algorithm holds. This **hybrid** design protects against quantum attackers while hedging against any undiscovered weakness in the newer algorithm.

CURRENT DEVELOPMENT X25519MLKEM768 has been the default key exchange in Chrome and Edge since version 131 and in Firefox since version 132 (late 2024). Apple added it across iOS 26 and macOS in 2025. By early 2026, major CDNs reported that around two-thirds of human web traffic reaching them was protected by hybrid post-quantum key exchange. Server-side and internal adoption lag far behind, which is the migration challenge Chapter 27 addresses.

The hybrid key share is larger: about 1,216 bytes from the client instead of 32. This occasionally breaks old middleboxes that cannot handle a ClientHello spanning more than one network packet, a class of bug called **protocol ossification**.

11.8 Mutual TLS

By default only the server authenticates. In **mutual TLS (mTLS)**, the server also requests a certificate from the client, and the client proves possession of its private key with its own CertificateVerify message. mTLS is widely used between services inside organisations, in service meshes, for device authentication and for high-security API access. It requires a way to issue and rotate client certificates at scale, usually from a private CA (Chapter 18).

WATCH OUT Public certificate authorities have been removing the client authentication purpose from public TLS certificates, following browser root program rules that public TLS hierarchies serve websites only. Systems that use public certificates as client certificates must move to private PKI. Check your CA's current timeline.

HANDS-ON LAB

Lab: watch a TLS 1.3 handshake

1. Inspect a live handshake.

```
openssl s_client -connect example.com:443 -servername example.com -tls1_3 </dev/null
2>/dev/null \
    | grep -E "Protocol|Cipher|Server Temp Key|Peer signature|Verification"
```

You will see the protocol version, the AEAD cipher suite, the key exchange group, and the signature algorithm.

2. Request post-quantum key exchange. OpenSSL 3.5 and later support ML-KEM:

```
openssl s_client -connect cloudflare.com:443 -groups X25519MLKEM768 </dev/null 2>/dev/null |
grep -i "temp key\|group"
```

3. Decrypt your own traffic. Set a key log file, make a request, and open the capture in Wireshark:

```
export SSLKEYLOGFILE=$HOME/tlskeys.log
curl -s https://example.com > /dev/null
```

In Wireshark, set Preferences, Protocols, TLS, "(Pre)-Master-Secret log filename" to that file while capturing. The encrypted records become readable, and you can match every message in Figure 11.1. Delete the key log afterwards.

CASE STUDY

Failure story: the downgrade that TLS 1.3 was built to stop

In 2016, researchers published **DROWN**. Many servers that supported only modern TLS for their websites still had **SSLv2**, a protocol from 1995, enabled on another service sharing the same RSA key, such as an old mail server configuration. SSLv2's weaknesses let an attacker use that server as a Bleichenbacher oracle (Chapter 7) to decrypt recorded *modern TLS* connections that used RSA key exchange with the same key. At disclosure, about a third of HTTPS servers were vulnerable, including many large sites.

DROWN combined three things TLS 1.3 removed by design: RSA key transport (no forward secrecy and an oracle target), the reuse of one key across protocols, and support for ancient protocol versions. TLS 1.3 also signs the full transcript and includes special values in the server random that let clients detect downgrade attempts. Its lesson for operators remains: disable old protocols everywhere a key is used, not only on the main website, and never share keys across services.

KEY TAKEAWAYS

Chapter summary

- TLS provides confidentiality, integrity and authentication through a handshake protocol and an AEAD record protocol.
- TLS 1.3 completes a full handshake in one round trip, encrypts everything after the ServerHello, signs the whole transcript, and removes the legacy features behind two decades of attacks.
- An HKDF key schedule derives independent, labelled keys at each stage.
- Session resumption speeds reconnection; 0-RTT data can be replayed and must be limited to safe requests.
- ECH hides the destination site; hybrid X25519MLKEM768 key exchange protects against harvest-now-decrypt-later and is already the browser default.

CHECK YOURSELF

Review questions

1. What does the CertificateVerify message prove, and why does it sign the whole transcript?
2. Why does TLS 1.3 need only one round trip for a full handshake?
3. Why is 0-RTT data dangerous for a payment API?
4. What information does ECH hide, and what does it depend on to be effective?
5. Explain why a hybrid key exchange is secure as long as either component is secure.
6. Which three properties of older TLS deployments did DROWN exploit?

Deploying TLS Well

A correct protocol can still be deployed badly. Most real TLS problems are configuration mistakes, missing chains, legacy settings left on for one old client, or web features that undo the protection TLS provides. This chapter turns TLS theory into an operational checklist: protocol and cipher choices, certificates and chains, HTTP security headers, testing, and the history of attacks that explains each rule.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Write a modern TLS configuration and justify each setting
- Diagnose chain, name and protocol errors from tool output
- Apply HSTS, secure cookies and other HTTPS protections
- Test and monitor TLS endpoints continuously

IN THIS CHAPTER

- | | |
|------|--|
| 12.1 | The goal: a boring, modern configuration |
| 12.2 | Protocol and cipher choices |
| 12.3 | Certificates on the server |
| 12.4 | HTTPS beyond the handshake |
| 12.5 | A history of TLS attacks, and the rule each one taught |
| 12.6 | Testing and monitoring |

12.1 The goal: a boring, modern configuration

A good TLS configuration is boring. It offers a small set of modern options, has no legacy fallbacks, and changes rarely. Most organisations should adopt a published profile rather than invent their own. The Mozilla Server Side TLS guidelines are the most widely used, with three levels:

Profile	Protocols	Who it serves	When to use
Modern	TLS 1.3 only	Clients from roughly the last five years	New services, APIs, internal systems
Intermediate	TLS 1.2 and 1.3, AEAD ciphers with forward secrecy	Almost all clients	The default for public websites
Old	Down to TLS 1.0	Very old clients	Only with a documented, time-limited exception

12.2 Protocol and cipher choices

For the intermediate profile, the TLS 1.2 cipher suites should all:

- Use **ephemeral key exchange** (ECDHE), for forward secrecy.
- Use **AEAD** (AES-GCM or ChaCha20-Poly1305), never CBC.
- Avoid RSA key transport, static DH, 3DES, RC4, export suites, and NULL or anonymous suites.

A representative TLS 1.2 list:

```
ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:
ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:
ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305
```

TLS 1.3 suites need no tuning; all of them are good. Two settings deserve attention:

- **Key exchange groups.** Offer `X25519MLKEM768` first where your software supports it (OpenSSL 3.5 and later, recent BoringSSL, Go 1.24 and later, and others), then `X25519`, `secp256r1`, `secp384r1`.
- **Server cipher preference.** Letting the server choose the order is no longer critical with an all-strong list, but it prevents clients from picking a weaker option if any slip in.

■ An example configuration

For nginx, an intermediate profile looks like this:

```
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305;
ssl_ecdh_curve X25519MLKEM768:X25519:prime256v1:secp384r1;
ssl_session_tickets off;      # or rotate ticket keys frequently
ssl_certificate      /etc/tls/fullchain.pem;  # leaf + intermediates
ssl_certificate_key  /etc/tls/privkey.pem;
add_header Strict-Transport-Security "max-age=63072000; includeSubDomains" always;
```

Session tickets are disabled here because their encryption keys, if never rotated, can undermine forward secrecy: anyone who steals the ticket key can decrypt resumed sessions. If you enable tickets, rotate their keys at least daily across all servers.

12.3 Certificates on the server

Many TLS failures are certificate failures. Know the four classic ones.

■ 1. The incomplete chain

A server must send its own certificate **and** the intermediate CA certificates that link it to a trusted root (Chapter 15). Browsers sometimes cope with a missing intermediate by fetching it or using a cache, which hides the problem during testing. Mobile apps, API clients, IoT devices and older systems do not, and fail with "unable to get local issuer certificate". Always configure the **full chain** (leaf plus intermediates, not the root).

■ 2. The name mismatch

The hostname the client connects to must appear in the certificate's **Subject Alternative Name** list. Connecting by IP address, an internal alias, or a new subdomain not in the list fails. Wildcards cover one label only: `*.example.com` covers `shop.example.com` but not `example.com` or `eu.shop.example.com`.

■ 3. The expired certificate

Covered at length in Chapter 19. The fix is automation, monitoring and early renewal, not calendar reminders.

4. The wrong key type for the client

Some old clients only accept RSA certificates. Servers can hold both an ECDSA and an RSA certificate for the same names and choose per client. ECDSA P-256 is the better default: smaller, faster handshakes.

12.4 HTTPS beyond the handshake

TLS protects the connection, but web applications can still undo that protection. Four mechanisms matter.

HTTP Strict Transport Security (HSTS). A response header that tells the browser to use only HTTPS for this site for a set period. It stops **SSL stripping** attacks, where an attacker on the network intercepts the first plain-HTTP request and keeps the user on HTTP. With `includeSubDomains` and a long `max-age`, a site can apply to be **preloaded** into browsers, so even the first visit is protected. Roll it out carefully: once preloaded, every subdomain must support HTTPS, and removal takes months.

Secure cookies. Session cookies must carry the `Secure` flag (sent only over HTTPS), `HttpOnly` (unreadable by JavaScript) and an appropriate `SameSite` value. The `__Host-` name prefix enforces these properties.

Mixed content. An HTTPS page that loads scripts or styles over plain HTTP gives an attacker a way to inject code. Modern browsers block active mixed content and upgrade many passive resources automatically. The `Content-Security-Policy: upgrade-insecure-requests` header helps migrate old pages.

Certificate Authority Authorization (CAA). A DNS record listing which CAs may issue certificates for your domain (Chapter 16). It is a cheap, valuable protection against mis-issuance.

12.5 A history of TLS attacks, and the rule each one taught

Year	Attack	What it exploited	Rule it taught
2009	SSL stripping	First request over plain HTTP	Use HSTS and preloading
2011	BEAST	Predictable CBC IVs in TLS 1.0	Retire TLS 1.0; prefer AEAD
2012	CRIME, later BREACH	Compression leaking secret lengths	Disable TLS compression; be careful compressing secrets in HTTP

Year	Attack	What it exploited	Rule it taught
2013	Lucky Thirteen	Timing in MAC-then-encrypt CBC	Use AEAD only
2014	Heartbleed	A memory-read bug in OpenSSL's heartbeat feature	Patch fast; rotate keys after memory disclosure
2014	POODLE	SSL 3.0 padding	Disable SSL 3.0 everywhere
2015	FREAK, Logjam	Downgrade to export-grade crypto	Remove legacy options entirely
2016	Sweet32	64-bit block ciphers (3DES)	Remove 3DES
2016	DROWN	SSLv2 on any service sharing a key	Never share keys across services
2017	ROBOT	Bleichenbacher oracles in RSA key exchange	Remove RSA key transport

Notice the pattern: almost every attack exploited an option that was kept "for compatibility". The single best rule in TLS deployment is to **remove what you do not need**.

12.6 Testing and monitoring

Test every public endpoint when it is built and continuously afterwards.

Tool	What it does
Qualys SSL Labs Server Test	Free web-based grading (A+ to F) of public servers; the best known external check
testssl.sh	Open-source command-line scanner; works on internal hosts and in pipelines
sslyze	Python scanner suited to automation
openssl s_client	Manual inspection of handshakes and chains
Certificate Transparency monitors	Alert when certificates are issued for your domains (Chapter 16)

Integrate a scanner into your deployment pipeline so that a configuration regression fails the build, and run scheduled external scans so drift is caught.

Getting an A+ on SSL Labs

The SSL Labs grade is a useful, widely recognised benchmark. An A+ typically requires: TLS 1.2 and 1.3 only, forward-secret AEAD suites only, a complete chain from a trusted CA, a key of adequate size, no known vulnerabilities, and HSTS with a long `max-age`. Treat the grade as a floor, not a finish line: it does not see your internal services, client-side behaviour or certificate operations.

IN PRACTICE TLS often terminates not on the application server but on a load balancer, reverse proxy, CDN or service mesh. Every termination point needs its own configuration review, and traffic between the terminator and the backend needs its own protection. "We use TLS" should always be followed by "where does it terminate, and what protects the next hop?"

HANDS-ON LAB

Lab: grade and fix a server

1. Scan a public site. Run `testssl.sh` (from `testssl.sh` on GitHub) against a site you own or are permitted to test:

```
./testssl.sh --fast https://your-site.example
```

Read the sections on protocols, cipher order, forward secrecy, HSTS, and vulnerabilities.

2. Build a deliberately weak local server and fix it. Using Docker and nginx, start with a configuration that allows TLS 1.0, CBC ciphers and no HSTS, scan it with `testssl.sh`, then apply the configuration in section 12.2 and scan again. Record each finding that disappears.

3. Break and fix a chain. Configure the server with only the leaf certificate, then connect with:

```
openssl s_client -connect localhost:443 -servername localhost </dev/null | grep -i verify
```

Observe the verification error, add the intermediate to `fullchain.pem`, and confirm `Verify return code: 0 (ok)`.

CASE STUDY

Failure story: Heartbleed

In April 2014, a flaw named **Heartbleed** was disclosed in OpenSSL, which then protected roughly two-thirds of the web's HTTPS servers. The TLS heartbeat extension let one side send a message and ask for it to be echoed back, stating the message's length. OpenSSL trusted the stated length without checking it against the actual message. An attacker could send one byte and claim 64 kilobytes, and the server would return 64 kilobytes of whatever happened to be in its memory: session cookies, passwords, and in some cases the server's private TLS key.

The cryptography was never involved. A missing bounds check in a rarely used feature exposed everything the cryptography protected, and the attack left no trace in normal logs. Recovery required patching, then **replacing private keys and certificates** on hundreds of thousands of servers, because nobody could prove their keys had not been read. Many organisations discovered they had no inventory of where their certificates were installed, which turned a patch into months of work.

Heartbleed also exposed how underfunded critical open-source infrastructure was: OpenSSL had a handful of part-time maintainers. The response led to the Core Infrastructure Initiative, later the Open Source Security Foundation, and to new TLS libraries such as BoringSSL and LibreSSL.

KEY TAKEAWAYS

Chapter summary

- Adopt a published profile (Mozilla intermediate or modern) instead of inventing your own. Offer TLS 1.2 and 1.3 with forward-secret AEAD suites only, and hybrid post-quantum groups where supported.
- Configure full chains, correct names, and automated renewal. Prefer ECDSA certificates.
- Protect the web layer with HSTS, secure cookies, mixed-content controls and CAA.
- Nearly every historical TLS attack exploited a legacy option kept for compatibility. Remove what you do not need.
- Test in pipelines and continuously, and remember every TLS termination point.

CHECK YOURSELF

Review questions

1. Why should session ticket keys be rotated, and what happens to forward secrecy if they are not?
2. A mobile app fails to connect but browsers work fine. What is the most likely certificate problem?
3. What attack does HSTS prevent, and what risk comes with preloading?
4. Pick three attacks from section 12.5 and explain which configuration setting prevents each.
5. Why is an SSL Labs A+ grade not proof that an organisation's TLS is in good shape?
6. After Heartbleed, why did organisations need to replace certificates, not just patch OpenSSL?

Beyond TLS: SSH, VPNs and Secure Messaging

TLS is not the only secure channel you will meet. Administrators live in SSH, networks are joined by IPsec and WireGuard tunnels, and billions of people rely on end-to-end encrypted messaging. Each protocol makes different trade-offs about trust and keys. This chapter explains how each works, how their key management differs, and how each is moving to post-quantum security.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Explain SSH's trust model and replace static SSH keys with SSH certificates
- Compare IPsec and WireGuard designs and their key management
- Explain how the Signal protocol achieves forward secrecy and post-compromise security
- Describe MLS for group messaging and post-quantum upgrades in each protocol

IN THIS CHAPTER

- | | |
|------|---|
| 13.1 | SSH: secure remote access |
| 13.2 | IPsec: network-layer protection |
| 13.3 | WireGuard: radical simplicity |
| 13.4 | End-to-end encrypted messaging: the Signal protocol |
| 13.5 | Group messaging: MLS |
| 13.6 | Comparing the protocols |

13.1 SSH: secure remote access

SSH (Secure Shell) gives administrators encrypted remote access to servers, and moves files and automation traffic. It runs over TCP port 22 and, like TLS, combines an ephemeral key exchange, server authentication, and symmetric encryption. Its trust model is very different, though.

Host keys and trust on first use

Every SSH server has a **host key**. The first time you connect, your client shows the key's fingerprint and asks whether to trust it. If you accept, the key is saved in

`~/.ssh/known_hosts`, and future connections warn loudly if it changes. This model is called **trust on first use (TOFU)**.

TOFU is convenient but weak in practice. Almost nobody checks the fingerprint on first connection, and in large fleets where servers are rebuilt often, users learn to click through "host key changed" warnings, which are exactly the warnings that should stop a man-in-the-middle.

User authentication with keys

Users usually authenticate with a key pair: the public key is placed in

`~/.ssh/authorized_keys` on each server, and the client proves possession of the private key. Ed25519 is the recommended key type.

At organisational scale, static SSH keys become a serious problem:

- Keys are copied to many servers and never removed when people leave.
- Private keys sit unencrypted on laptops and build servers.
- Nobody can answer "who has access to this server?" because the answer is scattered across thousands of `authorized_keys` files.

Studies of large enterprises have repeatedly found more SSH keys than employees, many granting root access, many orphaned.

SSH certificates: the scalable answer

OpenSSH supports its own simple **certificates** (not X.509). An organisation runs an **SSH certificate authority**:

- Servers trust the CA's public key for user logins (`TrustedUserCAKeys`), instead of listing individual user keys.

- Users authenticate to an identity system (with single sign-on and multi-factor authentication) and receive a **short-lived certificate**, often valid for 8 to 16 hours, stating their username and allowed principals.
- Likewise, hosts receive host certificates, and clients trust the host CA, which eliminates TOFU prompts.

When someone leaves, their certificates simply expire. There are no keys to hunt down. This is the same pattern as short-lived X.509 certificates (Chapter 19), and the same principle runs through this book: **short lifetimes replace revocation**.

CURRENT DEVELOPMENT OpenSSH added hybrid post-quantum key exchange in 2022 (sntrup761×25519-sha512, default from version 9.0), and version 10.0 in April 2025 made **mlkem768×25519-sha256** the default. Recent versions warn when a connection uses a key exchange that is not post-quantum. SSH sessions are therefore already protected against harvest-now-decrypt-later on up-to-date systems. Host and user signature keys are a separate, later migration.

13.2 IPsec: network-layer protection

IPsec protects traffic at the IP layer, so every application benefits without changes. It is the long-standing standard for site-to-site VPNs between offices and clouds, and is built into most routers and firewalls.

IPsec has two parts:

- **IKEv2** (Internet Key Exchange, RFC 7296) authenticates the peers, with certificates or pre-shared keys, and negotiates keys using Diffie-Hellman.
- **ESP** (Encapsulating Security Payload) encrypts and authenticates the packets, today usually with AES-GCM.

IPsec is powerful but notoriously complex: many options, many negotiable algorithms, and configuration that must match on both ends. Interoperability between vendors often forces administrators to enable weaker options, and pre-shared keys, which are easy to set up, are often weak and shared across many tunnels.

IN PRACTICE Review IPsec tunnels like TLS endpoints: IKEv2 only (not IKEv1), AES-GCM, DH groups of at least 2048-bit finite field or P-256 and above, certificates instead of shared keys for anything important, and lifetimes that force regular rekeying. RFC 9370 defines how IKEv2 adds post-quantum KEMs alongside the classic exchange, and vendors are rolling it out.

13.3 WireGuard: radical simplicity

WireGuard, created by Jason Donenfeld and merged into the Linux kernel in 2020, takes the opposite approach to IPsec. It has about 4,000 lines of code, compared with hundreds of thousands for typical IPsec stacks, and **no algorithm negotiation at all**. It uses one fixed set of modern primitives:

Purpose	WireGuard uses
Key exchange	X25519
Encryption	ChaCha20-Poly1305
Hashing	BLAKE2s
Handshake framework	The Noise protocol framework (Noise_IK)

Peers are identified purely by their public keys, which are configured in advance, much like SSH `authorized_keys`. The handshake gives forward secrecy and rekeys every two minutes.

The lack of negotiation is both a strength and a weakness. There is nothing to downgrade, and the small code base has been formally verified. But if one of its primitives were ever broken, every deployment would need a new protocol version rather than a configuration change. This is a deliberate rejection of algorithm agility in favour of simplicity, and it is a useful design to compare with the agility requirements of Chapter 24. WireGuard also offers an optional pre-shared symmetric key, mixed into the handshake, which some deployments use as a stop-gap protection against future quantum attacks on X25519.

13.4 End-to-end encrypted messaging: the Signal protocol

In TLS, the server can read your data: encryption stops at the server. **End-to-end encryption (E2EE)** means only the communicating users can read messages; the service that relays them cannot. The **Signal protocol**, developed by Moxie Marlinspike and Trevor Perrin, is the foundation of E2EE in Signal, WhatsApp, Google Messages and others, protecting several billion users.

Messaging has a hard problem that TLS does not: the other person may be offline when you start a conversation, and conversations last for years on devices that may be stolen or compromised. Signal solves this with two components.

■ X3DH, now PQXDH: starting a conversation offline

Each user uploads a bundle of public keys to the server: a long-term identity key, a signed medium-term "prekey", and a batch of one-time prekeys. To start a conversation, Alice downloads Bob's bundle and combines several Diffie-Hellman exchanges between her keys and his to derive a shared secret, without Bob being online. This is **X3DH** (Extended Triple Diffie-Hellman).

In 2023, Signal upgraded this to **PQXDH**, which adds an ML-KEM encapsulation to the exchange, so that new conversations are protected against harvest-now-decrypt-later.

■ The Double Ratchet: healing after compromise

Once a conversation starts, the **Double Ratchet** derives a new key for every message:

- A **symmetric ratchet** runs each key through a KDF to produce the next. Old keys are deleted, so stealing today's key does not reveal yesterday's messages (**forward secrecy**).
- A **Diffie-Hellman ratchet** mixes in a fresh DH exchange whenever the conversation changes direction. Even if an attacker steals a device's current keys, the next fresh DH exchange, which the attacker did not observe, locks them out again. This is called **post-compromise security** or "self-healing".

CURRENT DEVELOPMENT In October 2025, Signal announced a "triple ratchet" design that adds a post-quantum ratchet (named SPQR) alongside the Double Ratchet, so ongoing conversations, not only their start, gain quantum resistance. Apple's iMessage moved to a comparable post-quantum design, PQ3, in 2024.

■ What E2EE does not protect

E2EE protects message content in transit and on the server. It does not hide **metadata** (who talks to whom, when, how often) unless the service takes extra steps, it does not protect messages on a compromised phone, and it depends on users verifying each other's keys (the "safety number") to rule out a server inserting itself as a man-in-the-middle. **Key transparency** systems, now deployed by several large messaging services, publish users' public keys in auditable logs so that such substitution would be detected automatically.

13.5 Group messaging: MLS

Signal's pairwise design becomes expensive for large groups: a message to 1,000 members means 1,000 encryptions. **Messaging Layer Security (MLS)**, standardised as RFC 9420 in 2023, solves this with a tree of keys (TreeKEM) so that adding, removing or updating a member costs work proportional to the logarithm of the group size. MLS keeps forward secrecy and post-compromise security for groups of up to tens of thousands of members, and is being adopted by enterprise messaging platforms and by the mobile industry's RCS standard.

13.6 Comparing the protocols

Protocol	Who is trusted to vouch for keys	Algorithm agility	Forward secrecy	Post-quantum status (2026)
TLS 1.3	Certificate authorities	Negotiated	Yes	Hybrid key exchange deployed widely
SSH	TOFU, or an SSH CA	Negotiated	Yes	Hybrid ML-KEM default in OpenSSH 10
IPsec / IKEv2	Certificates or pre-shared keys	Negotiated	Yes, if configured	Standard defined (RFC 9370); rolling out
WireGuard	Pre-configured peer keys	None	Yes	Optional PSK as stop-gap
Signal	The service, verified by safety numbers or key transparency	Versioned	Yes, plus self-healing	PQXDH and post-quantum ratchet deployed
MLS	Credentials from an identity service	Cipher suites	Yes, plus self-healing	Post-quantum cipher suites in development

HANDS-ON LAB

Lab: an SSH certificate authority

1. Create a user CA and sign a short-lived certificate.

```
ssh-keygen -t ed25519 -f user_ca -C "lab user CA" -N ""
ssh-keygen -t ed25519 -f alice -N ""
ssh-keygen -s user_ca -I alice@lab -n alice -V +8h alice.pub
ssh-keygen -L -f alice-cert.pub      # shows principals and the 8-hour validity
```

2. Trust the CA on a test server (a container or VM you control):

```
# /etc/ssh/sshd_config
TrustedUserCAKeys /etc/ssh/user_ca.pub
```

Restart `sshd`, then connect with `ssh -i alice alice@server`. No `authorized_keys` entry is needed. Wait for the certificate to expire, or issue one with `-V +1m`, and watch access end on its own.

3. Check your post-quantum key exchange.

```
ssh -v user@server 2>&1 | grep "kex: algorithm"
```

On OpenSSH 10 at both ends, you should see `mlkem768x25519-sha256`.

CASE STUDY

Failure story: the SSH keys nobody owned

Beginning around 2013, the inventor of SSH, Tatu Ylönen, and other researchers published findings from key audits at large banks, governments and enterprises. The pattern was consistent: organisations held hundreds of thousands to millions of SSH keys, the large majority unused or of unknown purpose, and a significant share granting privileged access, including keys belonging to people who had long left. Nobody had ever designed this sprawl. It accumulated one convenient copy at a time.

Attackers exploit exactly this. Malware families and intrusion groups routinely harvest SSH private keys from compromised machines and use the `known_hosts` and `authorized_keys` files as maps to move laterally through networks. Unlike passwords, stolen SSH keys usually bypass multi-factor authentication and are rarely rotated.

The remedy is the one in the lab: replace scattered static keys with short-lived certificates issued after strong authentication, and treat any remaining static keys as inventoried, owned credentials with expiry dates, just like the certificates and secrets in Parts V and

VI.

KEY TAKEAWAYS

Chapter summary

- SSH uses trust on first use and static keys by default, which scale badly. SSH certificates with short lifetimes fix both, and OpenSSH now uses hybrid post-quantum key exchange by default.
- IPsec is powerful and complex; configure IKEv2 with AEAD, strong groups and certificates. WireGuard trades agility for simplicity and verifiability.
- The Signal protocol combines an asynchronous start (now PQXDH) with the Double Ratchet for forward secrecy and post-compromise security, and has added a post-quantum ratchet.
- MLS brings these properties efficiently to large groups.
- End-to-end encryption does not hide metadata or protect compromised devices; key verification and key transparency close the man-in-the-middle gap.

CHECK YOURSELF

Review questions

1. What is trust on first use, and why does it fail in large server fleets?
2. How do SSH certificates solve the problem of departing employees' keys?
3. Give one advantage and one disadvantage of WireGuard's lack of algorithm negotiation.
4. Explain the difference between forward secrecy and post-compromise security.
5. Why does group messaging need a different design from pairwise messaging?
6. What does end-to-end encryption not protect, and how does key transparency help?

Identity Protocols: OAuth, OpenID Connect, JWT and Passkeys

Most of the cryptography a typical engineer touches day to day is hidden inside identity systems: the tokens that let apps call APIs, the single sign-on that logs employees into hundreds of services, and the passkeys that are replacing passwords. This chapter explains the cryptography inside these systems, the signing keys they depend on, and the mistakes that turn a token into a skeleton key.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Explain the roles in OAuth 2.0 and OpenID Connect and what each token proves
- Validate JWTs correctly and recognise alg confusion and other token attacks
- Explain how signing keys are published, rotated and protected in identity systems
- Describe how passkeys use public-key cryptography to resist phishing

IN THIS CHAPTER

- 14.1** Authentication, authorisation and federation
- 14.2** OAuth 2.0: delegated authorisation
- 14.3** OpenID Connect: authentication on top of OAuth
- 14.4** JSON Web Tokens
- 14.5** SAML
- 14.6** Protecting identity signing keys
- 14.7** Passkeys: public-key cryptography for everyone

14.1 Authentication, authorisation and federation

Three words are often mixed up:

- **Authentication** answers "who are you?" (logging in).
- **Authorisation** answers "what may you do?" (permissions).
- **Federation** lets one system trust another's answer, so a user who logged in once can use many services (single sign-on).

Identity protocols carry these answers between systems as **signed tokens**. The signature is what lets a service trust a token without calling back to the issuer every time. That makes the issuer's signing key one of the most powerful keys in any organisation: anyone who holds it can mint any identity.

14.2 OAuth 2.0: delegated authorisation

OAuth 2.0 (RFC 6749) lets a user grant an application limited access to their resources without giving it their password. The roles:

Role	Example
Resource owner	The user
Client	The application that wants access
Authorisation server	The system that authenticates the user and issues tokens
Resource server	The API holding the data

The recommended flow for most applications is the **authorisation code flow with PKCE**:

1. The client sends the user to the authorisation server with a random **code challenge** (a hash of a secret **code verifier**).
2. The user logs in and approves access.
3. The authorisation server redirects back with a short-lived **authorisation code**.
4. The client exchanges the code, plus the original code verifier, for an **access token** (and often a **refresh token**).

PKCE (Proof Key for Code Exchange, RFC 7636) ensures that a stolen authorisation code is useless without the verifier, which never left the client. It is now required for all clients in the OAuth 2.1 consolidation of best practices.

Access tokens are usually **bearer tokens**: whoever presents one gets access, like cash. That makes them valuable to steal. Defences include short lifetimes (minutes), narrow **scopes**, and **sender-constrained tokens** that are bound to a key the client holds, using mTLS (RFC 8705) or **DPoP** (RFC 9449), where the client signs each request with a private key, so a stolen token alone is not enough.

14.3 OpenID Connect: authentication on top of OAuth

OAuth on its own is about access, not identity. **OpenID Connect (OIDC)** adds an **ID token**: a signed statement from the identity provider saying "this user authenticated, at this time, for this client". Enterprise single sign-on, "Sign in with..." buttons, and workload identity in cloud platforms all use OIDC.

The identity provider publishes its signing public keys at a well-known URL as a **JSON Web Key Set (JWKS)**. Relying services fetch the JWKS, cache it, and verify ID token signatures locally. Each key has a **key ID (kid)**, and tokens name which key signed them, which makes rotation possible: the provider publishes a new key, starts signing with it, and later removes the old one once all tokens signed by it have expired.

14.4 JSON Web Tokens

Both access tokens and ID tokens are commonly **JSON Web Tokens (JWT)**, RFC 7519. A JWT has three base64url-encoded parts separated by dots:

```
header.payload.signature
{"alg":"ES256","kid":"2026-09"} .
{"iss":"https://id.example","sub":"u123","aud":"api","exp":1791500000} . <signature>
```

The **header** names the signing algorithm and key. The **payload** holds **claims**: issuer (`iss`), subject (`sub`), audience (`aud`), expiry (`exp`) and others. The **signature** covers the header and payload. A JWT is signed, not encrypted: anyone can read the payload. (Encrypted JWTs, called JWE, exist but are less common.)

How to validate a JWT

Every check matters. Missing any one has caused real breaches.

1. **Fix the algorithm on your side.** Accept only the algorithm you expect for this issuer, such as ES256 or RS256. Never let the token's header choose.
2. **Verify the signature** with the issuer's key, selected by `kid` from a JWKS you fetched over TLS from a configured URL.
3. **Check the issuer** (`iss`) is the one you trust.
4. **Check the audience** (`aud`) is your service. Otherwise a token issued for another service can be replayed against yours.
5. **Check time claims** (`exp`, `nbf`, `iat`) with a small allowance for clock skew.
6. **Check token type and purpose**, so an ID token is not accepted as an access token.

Classic JWT attacks

Attack	How it works	Prevention
<code>alg: none</code>	Attacker sets the algorithm to "none" and removes the signature; naive libraries accept it	Reject "none"; fix the algorithm (check 1)
Algorithm confusion	Attacker changes RS256 to HS256 and signs with the server's public key as an HMAC secret; a library that picks the algorithm from the header verifies it	Bind each key to one algorithm
<code>kid</code> or <code>jku</code> injection	Attacker points the header at a key they control, or injects paths into <code>kid</code>	Only use keys from configured JWKS URLs
Missing audience check	Token for service A accepted by service B	Always check <code>aud</code>
Weak HMAC secret	Short shared secrets for HS256 brute-forced offline	Use asymmetric signatures, or 256-bit random secrets

IN PRACTICE Prefer asymmetric signing (ES256 or EdDSA) for tokens consumed by multiple services. With HMAC (HS256), every service that verifies tokens holds the secret and can therefore also mint them.

14.5 SAML

SAML 2.0 is the older, XML-based federation standard, still common for enterprise single sign-on. The identity provider signs XML assertions with **XML Digital Signature**. XML signatures are notoriously tricky: **signature wrapping attacks** move signed elements

around in the document so that the signature still verifies but the application reads a different, unsigned element. Several major SAML libraries have had such flaws. If you must accept SAML, use a mature, well-maintained library and keep it patched.

14.6 Protecting identity signing keys

Whoever holds an identity provider's signing key can impersonate any user to every federated service, often without triggering normal login alerts. Two incidents make the point:

- In the **2020 software supply-chain intrusion** that affected many US agencies and companies, attackers who reached on-premises federation servers stole token-signing certificates and forged SAML assertions, a technique researchers named **Golden SAML**. Forged tokens gave access to cloud email and documents while bypassing multi-factor authentication.
- In the **2023 Storm-0558** incident (Chapter 9), a stolen consumer signing key, combined with a validation flaw, let attackers forge tokens for enterprise mailboxes.

Treat identity signing keys like root CA keys: keep them in HSMs or managed key services where they cannot be exported, rotate them on a schedule, publish only what verifiers need, scope keys tightly by tenant and purpose, and monitor for tokens that do not match normal issuance patterns.

14.7 Passkeys: public-key cryptography for everyone

Passwords are phishable: a convincing fake login page collects them, and many forms of multi-factor authentication (codes by text message or authenticator apps) can be phished in real time too. **FIDO2** and its web API, **WebAuthn**, replace passwords with public-key cryptography, and **passkeys** are the consumer-friendly form of FIDO2 credentials.

How it works:

1. **Registration.** Your device creates a new key pair for this specific website. The public key goes to the website; the private key stays on your device or in your platform's password manager, protected by the device's secure hardware and unlocked with your fingerprint, face or PIN.
2. **Login.** The website sends a random challenge. Your device signs it, together with the website's origin (its exact domain), and the site verifies the signature with the stored public key.

Two properties make passkeys **phishing-resistant**:

- The signature includes the origin. A key created for `bank.example` will never sign for `bank-example.login.site`. The browser enforces this, not the user.
- There is no shared secret to steal from the server. A breached website loses only public keys.

CURRENT DEVELOPMENT Passkeys are now supported by all major operating systems, browsers and password managers, and can sync across a user's devices. Large consumer platforms report that passkey sign-ins are both faster and far more successful than passwords, and many organisations now require phishing-resistant authentication, such as FIDO2 security keys or passkeys, for administrators. US federal guidance has required phishing-resistant multi-factor authentication for agency staff since 2022.

HANDS-ON LAB

Lab: build and break a JWT

1. Create and verify a signed JWT (Python, `pip install pyjwt cryptography`):

```
import jwt, time
from cryptography.hazmat.primitives.asymmetric import ec
key = ec.generate_private_key(ec.SECP256R1())
tok = jwt.encode({"iss": "https://id.lab", "aud": "api", "sub": "u1",
                  "exp": int(time.time()) + 300}, key, algorithm="ES256",
                  headers={"kid": "lab-1"})
print(jwt.decode(tok, key.public_key(), algorithms=["ES256"],
                  audience="api", issuer="https://id.lab"))
```

2. Try the "none" attack. Edit the token: change the header to `{"alg": "none"}`, change `sub` to `admin`, and drop the signature (keep the trailing dot). Pass it to `jwt.decode` with `algorithms=["ES256"]`. It is rejected, because the algorithm is fixed by the verifier, not the token.

3. Try the audience attack. Encode a token with `aud: "other-service"` and decode with `audience="api"`. It is rejected. Now remove the `audience` argument and see the library refuse to skip the check silently: good libraries force you to make the decision explicitly.

4. Inspect a JWKS. Fetch a real identity provider's public keys, for example `https://www.googleapis.com/oauth2/v3/certs`, and note the `kid`, `alg` and key parameters. Fetch it again weeks later to observe rotation.

CASE STUDY

Failure story: the token that trusted its own header

In 2015, security researcher Tim McLean reviewed popular JWT libraries across several languages and found two critical flaws in many of them. Some libraries accepted tokens with `"alg": "none"` as valid, even when the application expected signed tokens, so anyone could forge any identity by deleting the signature. Others allowed **algorithm confusion**: when a server expected RSA signatures, an attacker could change the header to HS256 and create an HMAC using the server's **public** RSA key, which the library then used as the HMAC secret and accepted.

The underlying mistake was the same in both: the library let attacker-controlled data, the token header, decide how the token would be verified. Libraries were patched to require callers to specify allowed algorithms, and the JWT best-practice RFC 8725 was published in 2020 to codify the lessons. Variants still appear in new libraries and custom code, which is why "fix the algorithm on your side" is the first rule of token validation.

KEY TAKEAWAYS

Chapter summary

- Identity protocols carry authentication and authorisation as signed tokens; the issuer's signing key can mint any identity.
- OAuth 2.0 delegates access; use the authorisation code flow with PKCE, short-lived scoped tokens, and sender-constrained tokens where possible. OIDC adds signed ID tokens and publishes keys via JWKS.
- Validate JWTs completely: fixed algorithm, signature, issuer, audience, time and type. Never let the token's header choose the verification method.
- Identity signing keys deserve root-CA-level protection, as Golden SAML and Storm-0558 showed.
- Passkeys apply public-key signatures bound to the website's origin, making login phishing-resistant and server breaches far less damaging.

CHECK YOURSELF

Review questions

1. What problem does PKCE solve in the authorisation code flow?
2. Why are bearer tokens risky, and name two ways to constrain them.
3. List the six checks for validating a JWT and give an attack each one prevents.
4. Why is asymmetric token signing preferable to HMAC when many services verify tokens?
5. What is Golden SAML, and what key does it depend on?
6. Explain why a passkey cannot be phished by a look-alike website.

PART V

Certificates and PKI

Certificates, the trust hierarchies behind them, revocation, private certificate authorities and lifecycle automation at scale.

In this part

- 15 X.509 Certificates Explained

- 16 The Web PKI: Certificate Authorities, Root Programs and Transparency

- 17 Path Validation and Revocation

- 18 Building and Running a Private CA

- 19 Certificate Lifecycle Automation at Scale

X.509 Certificates Explained

A certificate is the document that binds a public key to a name, signed by someone the reader already trusts. Nearly every trust decision on the internet, and inside most organisations, depends on reading certificates correctly. This chapter takes a certificate apart field by field, explains chains and extensions, and shows how to read, create and debug certificates with everyday tools.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Read every important field and extension in an X.509 certificate
- Explain how certificate chains link a server's certificate to a trusted root
- Recognise the common file formats and convert between them
- Design certificate profiles for servers, clients and CAs

IN THIS CHAPTER

- | | |
|------|---------------------------------------|
| 15.1 | The problem certificates solve |
| 15.2 | Anatomy of a certificate |
| 15.3 | Extensions: where the real rules live |
| 15.4 | Chains of trust |
| 15.5 | Encodings and file formats |
| 15.6 | Certificate types by purpose |

15.1 The problem certificates solve

Public-key cryptography lets you verify a signature or encrypt to a key. It does not tell you **whose** key it is. If an attacker can make you believe their public key belongs to your bank, every guarantee collapses.

A **certificate** solves this with a signed statement: "this public key belongs to `www.bank.example`, signed by a certificate authority". If you already trust the CA's public key, you can check the signature and trust the binding. A certificate is like a passport: the passport office (CA) vouches that the photo (public key) belongs to the named person (subject), and border staff (clients) trust the passport office, not the traveller.

Certificates are **public**. Anyone can see and copy them. They are useless to an attacker without the matching **private key**, which never appears in the certificate.

The dominant certificate format is **X.509 version 3**, profiled for the internet in **RFC 5280**.

15.2 Anatomy of a certificate

A certificate has three parts: the **to-be-signed** data (all the fields below), the **signature algorithm**, and the **signature** by the issuing CA.

Field	What it holds	Example or notes
Version	Always 3 for modern certificates	Extensions require version 3
Serial number	A unique number assigned by the issuing CA	Must include at least 64 random bits for public certificates (Chapter 6)
Signature algorithm	How the CA signed	<code>ecdsa-with-SHA384</code> , <code>sha256WithRSAEncryption</code>
Issuer	The name of the CA that signed it	<code>C=US, O=Example CA, CN=Example TLS Issuing CA 2</code>
Validity	Not Before and Not After dates	Public TLS certificates: at most 200 days from March 2026
Subject	Who the certificate is about	Often only a Common Name, sometimes organisation details
Subject public key info	The public key and its algorithm	ECDSA P-256, RSA 3072, and soon ML-DSA
Extensions	Everything else that matters (section 15.3)	Names, usages, constraints, policy, revocation pointers

Names in the Subject and Issuer are **Distinguished Names (DNs)**, built from attributes such as **C** (country), **O** (organisation), **OU** (organisational unit) and **CN** (common name). Public CAs stopped including **OU** in TLS certificates in 2022, because it was rarely validated properly.

15.3 Extensions: where the real rules live

Extensions turn a bare key-to-name binding into a precise set of permissions. Each extension can be marked **critical**, meaning a client that does not understand it must reject the certificate.

Extension	What it does	Typical value for a TLS server	Typical value for a CA
Subject Alternative Name (SAN)	The names (DNS, IP, email, URI) the certificate is valid for	DNS:shop.example.com, DNS:www.shop.example.com	Not used
Basic Constraints	Whether this is a CA, and how many CAs may sit below it	CA:FALSE	CA:TRUE, pathlen:0 for an issuing CA
Key Usage	Which operations the key may perform	digitalSignature	keyCertSign, cRLSign
Extended Key Usage (EKU)	Which applications it may be used for	serverAuth	Often restricted on intermediates
Authority Key Identifier / Subject Key Identifier	Fingerprints that link a certificate to its issuer's key	Present	Present
CRL Distribution Points	Where to download the revocation list	URL	URL
Authority Information Access	Where to fetch the issuer's certificate, and an OCSP responder if any	URL	URL
Certificate Policies	Identifiers of the rules under which it was issued	DV, OV or EV policy OIDs	Policy OIDs
Name Constraints	Limits which names a CA may issue for	Not used	permitted: .example.com
SCT list	Proof that the certificate was logged in Certificate Transparency	Embedded SCTs	Not used

Three of these deserve emphasis.

The SAN is the identity. Browsers and modern clients check only the SAN, not the Common Name, when matching a hostname. A certificate with a CN but no SAN entry for the name fails.

Basic Constraints is the most important safety switch in PKI. It decides whether a certificate can sign other certificates. Bugs that ignored it, such as a 2002 flaw in Internet Explorer that accepted ordinary website certificates as CAs, let anyone with a cheap certificate impersonate any website.

Name Constraints makes private CAs safer. A CA constrained to `.example.com` cannot issue a trusted certificate for any other domain, even if its key is stolen. It is underused and worth adopting for internal CAs (Chapter 18).

15.4 Chains of trust

Certificate authorities do not sign server certificates directly with their most valuable key. Instead, they form a hierarchy:

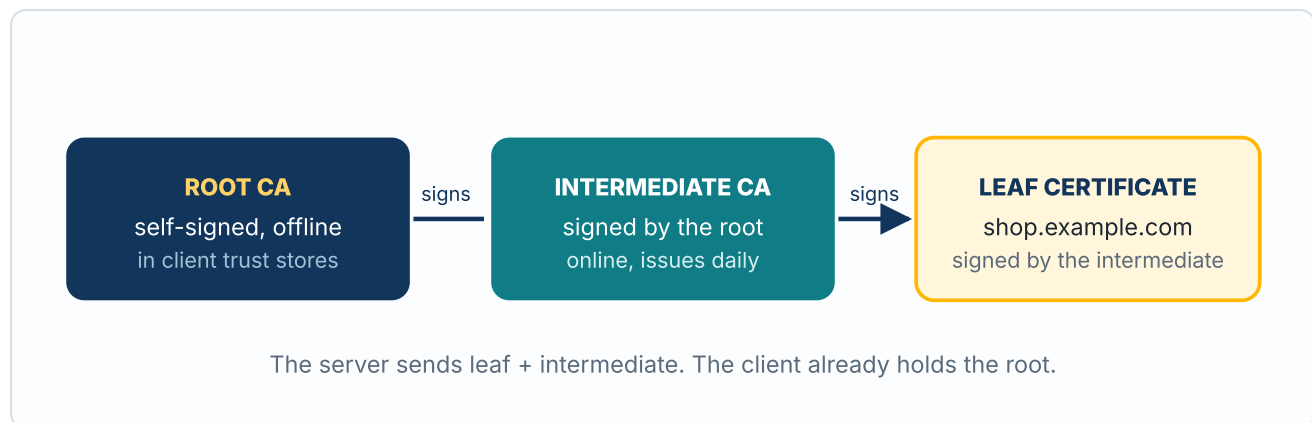


Figure 15.1 A three-level chain. Validation walks from the leaf up to a root the client already trusts.

- The **root CA** certificate is self-signed. Its trust comes not from the signature but from its presence in the client's **trust store**, the list of roots shipped by the operating system or browser. Root keys are kept offline in HSMs and used only to sign intermediates and revocation lists.
- **Intermediate CAs** are signed by the root and do the day-to-day issuing. If an intermediate is compromised, it can be revoked and replaced without changing every client's trust store.
- **Leaf** (end-entity) certificates are what servers, clients and devices present.

To validate a leaf, the client builds a **chain** (also called a path) from the leaf to a trusted root, checking at each link that the signature verifies, the issuer is allowed to be a CA, the dates are valid, constraints are respected and nothing is revoked. Chapter 17 covers path validation in detail.

Cross-signing

A new CA's root takes years to reach enough trust stores. To bridge the gap, an established CA can **cross-sign** the new CA's intermediate or root, so the same key has two certificates: one chaining to the new root, one to the old. Clients with either root can build a valid chain. Let's Encrypt launched in 2015 this way, cross-signed by an established CA, and the expiry of that cross-signature in September 2021 broke connections on many old devices that did not have the newer root.

15.5 Encodings and file formats

Certificate files confuse everyone at first because the same content appears in many wrappers.

Format	What it is	Typical extensions	How to recognise it
DER	The binary encoding of a certificate (ASN.1 rules)	<code>.der</code> , <code>.cer</code>	Binary; unreadable in a text editor
PEM	DER in base64 between header lines	<code>.pem</code> , <code>.crt</code> , <code>.cer</code> , <code>.key</code>	Starts with <code>-----BEGIN CERTIFICATE-----</code>
PKCS#12 / PFX	A password-protected bundle of private key plus certificate chain	<code>.p12</code> , <code>.pfx</code>	Binary; used by Windows and Java
PKCS#7 / P7B	A bundle of certificates without keys	<code>.p7b</code> , <code>.p7c</code>	Used for chains on Windows
PKCS#8	A private key, optionally encrypted	<code>.key</code> , <code>.pem</code>	<code>-----BEGIN PRIVATE KEY-----</code> or <code>ENCRYPTED PRIVATE KEY</code>
CSR (PKCS#10)	A certificate signing request	<code>.csr</code>	<code>-----BEGIN CERTIFICATE REQUEST-----</code>

A **certificate signing request (CSR)** contains the public key and requested names, signed with the private key to prove possession. The applicant sends the CSR to the CA; the private key never leaves the applicant.

15.6 Certificate types by purpose

Type	Key usage and EKU	Validated how	Notes
TLS server (public)	digitalSignature; serverAuth	Domain control (DV), plus organisation (OV) or extended (EV) checks	Governed by CA/Browser Forum rules (Chapter 16)
TLS client	digitalSignature; clientAuth	By the private CA's enrolment process	Moving to private PKI as public CAs drop clientAuth
Code signing	digitalSignature; codeSigning	Organisation validation; keys must be in hardware since 2023	Paired with timestamps
S/MIME email	digitalSignature, keyEncipherment; emailProtection	Mailbox control and identity	Governed by its own baseline requirements since 2023
Document signing	digitalSignature, nonRepudiation	Identity checks; qualified certificates under eIDAS in the EU	Long-term validation needed
CA certificates	keyCertSign, cRLSign; Basic Constraints CA:TRUE	Audits	Pathlen and name constraints limit scope

HANDS-ON LAB

Lab: take certificates apart

1. Fetch and read a real certificate.

```
openssl s_client -connect www.wikipedia.org:443 -servername www.wikipedia.org -showcerts
</dev/null 2>/dev/null > chain.txt
openssl x509 -in chain.txt -noout -text | less
```

Find the serial number, issuer, validity dates, SAN list, key usage, EKU, CRL distribution point, authority information access and the embedded SCTs.

2. Create a CSR and inspect it.

```
openssl req -new -newkey ec -pkeyopt ec_paramgen_curve:P-256 -nodes \  
-keyout lab.key -out lab.csr -subj "/CN=lab.example.test" \  
-addext "subjectAltName=DNS:lab.example.test,DNS:www.lab.example.test" \  
openssl req -in lab.csr -noout -text -verify
```

3. Convert between formats.

```
openssl x509 -in cert.pem -outform DER -out cert.der           # PEM to DER  
openssl pkcs12 -export -inkey lab.key -in cert.pem -out lab.p12 # bundle key + cert  
openssl pkcs12 -in lab.p12 -noout -info                      # inspect a bundle
```

4. Check that a key and certificate match. Compare the public key hashes:

```
openssl x509 -in cert.pem -noout -pubkey | openssl sha256  
openssl pkey -in lab.key -pubout | openssl sha256
```

Identical hashes mean they are a pair. This single check solves a surprising number of deployment mysteries.

CASE STUDY

Failure story: the root that expired

On 30 May 2020, a widely used root certificate, the AddTrust External CA Root, reached its expiry date. Modern browsers were unaffected because they had long since learned other paths to the same intermediate. But many servers still sent an old cross-signed chain ending at AddTrust, and many older clients and libraries, including some versions of OpenSSL and GnuTLS used by payment systems, APIs and embedded devices, stopped at the first path they found instead of searching for an alternative. Those clients began failing with "certificate has expired", even though the server's own certificate was valid.

Over the following days, outages hit payment processors, monitoring services and many internal integrations. The fix was to update the chain the servers sent, removing the expired cross-sign, and to update client libraries. A similar event followed in September 2021 when an older root expired and affected devices that had not received updated trust stores.

The lesson: a certificate's validity depends on the whole chain, not just the leaf. Inventory the chains you serve, know which roots your clients trust, and treat root and intermediate expiries as planned events.

KEY TAKEAWAYS

Chapter summary

- A certificate binds a public key to a name with a CA's signature. It is public; the private key is the secret.
- The key fields are serial, issuer, validity, subject and public key; the real rules live in extensions, especially SAN, Basic Constraints, Key Usage, EKU and Name Constraints.
- Chains link leaves to roots through intermediates; trust in the root comes from the client's trust store, not its signature.
- PEM, DER, PKCS#12, PKCS#7, PKCS#8 and CSRs are wrappers for the same few objects; know how to convert and inspect them.
- Certificates expire at every level of the chain. Root and intermediate expiries can break old clients.

CHECK YOURSELF

Review questions

1. Why is a certificate safe to publish?
2. Which extension do modern clients use to match a hostname, and what happens if it is missing?
3. Why is Basic Constraints called the most important safety switch in PKI?
4. Why do CAs use intermediates instead of signing leaves with the root key?
5. What is cross-signing, and what problem did its expiry cause in 2021?
6. Describe how to confirm that a private key and certificate belong together.

The Web PKI: Certificate Authorities, Root Programs and Transparency

Every browser trusts a few dozen organisations to vouch for every website on earth. This chapter explains who those organisations are, who decides they can be trusted, what rules they follow, how Certificate Transparency watches them, and how the system punishes failure. It closes with the changes reshaping the web PKI right now: shorter lifetimes, stricter validation, and the removal of untrustworthy CAs.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Explain the roles of CAs, root programs, the CA/Browser Forum and auditors
- Describe DV, OV and EV validation and how domain control is proven
- Explain how Certificate Transparency and CAA protect domain owners
- Summarise the current lifetime, validation and distrust changes and their operational impact

IN THIS CHAPTER

- | | |
|------|---|
| 16.1 | The players |
| 16.2 | Root programs hold the real power |
| 16.3 | The Baseline Requirements and validation levels |
| 16.4 | Shorter lifetimes: the 47-day future |
| 16.5 | Certificate Transparency |
| 16.6 | CAA: telling CAs who may issue |
| 16.7 | How users see all this |

16.1 The players

The **web public key infrastructure (web PKI)** is the system that lets any browser verify any website's certificate. Five groups make it work:

Player	Role
Certificate authorities (CAs)	Verify applicants and issue certificates. Some are commercial companies; Let's Encrypt is a non-profit that issues most of the web's certificates for free
Root programs	Run by browser and operating system makers (Apple, Google Chrome, Microsoft, Mozilla). Decide which CA roots go into their trust stores, and remove them
The CA/Browser Forum	An industry body of CAs and browser makers that writes the Baseline Requirements , the common rulebook for publicly trusted TLS certificates
Auditors	Independent firms that audit CAs annually against WebTrust or ETSI criteria
Domain owners and site operators	Request certificates, prove domain control, and watch for mis-issuance

The central fact of the web PKI is uncomfortable: **any trusted CA can issue a certificate for any domain**. Your bank's site is only as safe as the weakest CA in your browser's trust store. Most of the system's machinery exists to manage that risk.

16.2 Root programs hold the real power

A CA is "publicly trusted" only because root programs include its root certificates. Each program publishes its own policy, layered on top of the Baseline Requirements, and can remove a CA whose behaviour falls short. Removal, called **distrust**, is the strongest sanction in the system, and it has been used repeatedly:

Year	CA	Reason	Outcome
2011	DigiNotar	Hacked; fraudulent certificates for major sites used to spy on users	Removed within days; the company went bankrupt
2015 to 2016	CNNIC, WoSign and StartCom	Unauthorised intermediate; backdated certificates and concealed ownership	Distrusted
2017 to 2018	Symantec (and its brands)	Years of mis-issuance and inadequate oversight of partners	Gradually distrusted; business sold
2019 to 2020	DarkMatter	Concerns about trustworthiness, never admitted	Inclusion refused

Year	CA	Reason	Outcome
2024	Entrust	A long pattern of compliance failures and slow incident responses	Chrome, Mozilla and others distrusted new certificates from November 2024
2025	Chunghwa Telecom and Netlock	Compliance failures and unmet improvement commitments	Distrusted by Chrome from August 2025

Modern distrusts are usually **gradual**: browsers stop trusting certificates issued after a cut-off date, while existing certificates remain valid until they expire. This protects users while giving site operators months to switch CAs.

IN PRACTICE Every organisation using public certificates should be able to answer: "If our CA were distrusted tomorrow, how long would it take to move every public certificate to another CA?" Organisations that relied on Entrust in 2024 had a few months. Those with automation and a tested second CA switched in days.

16.3 The Baseline Requirements and validation levels

The Baseline Requirements set rules on key sizes, algorithms, certificate contents, validity periods, revocation, audits and, above all, **validation**: how a CA confirms an applicant should get a certificate.

Level	What the CA verifies	What the certificate shows	Status
DV (Domain Validated)	That the applicant controls the domain	Only the domain name	The large majority of web certificates
OV (Organisation Validated)	Domain control plus that the organisation legally exists	Organisation name and location in the subject	Common for businesses
EV (Extended Validation)	Domain control plus extensive organisation checks	Detailed organisation information	Browsers removed the special EV indicator around 2019, because studies found users did not notice it

Proving domain control

The approved **domain control validation (DCV)** methods include:

- **DNS**: publish a random value in a DNS TXT record (or a CNAME, depending on the method).

- **HTTP:** serve a random value at a specific path on the website.
- **TLS-ALPN:** answer a special TLS handshake on port 443.
- **Email:** click a link sent to a constructed address such as admin@ the domain.
Constructed-email and WHOIS-based methods have been progressively restricted after abuse.

CURRENT DEVELOPMENT Since March 2025, CAs must check domain control from **multiple network perspectives** in different regions (Multi-Perspective Issuance Corroboration, adopted in Ballot SC-067). This defends against attackers who hijack routing (BGP) to one CA's validation server for a few minutes, a technique demonstrated by researchers and used in real cryptocurrency thefts.

16.4 Shorter lifetimes: the 47-day future

Certificate lifetimes have shrunk steadily: from five years before 2015, to three, then two, then 398 days in 2020. In April 2025, the CA/Browser Forum approved **Ballot SC-081**, which shortens them further on a schedule:

Certificates issued on or after	Maximum validity	Maximum reuse of a domain validation
15 March 2026	200 days	200 days
15 March 2027	100 days	100 days
15 March 2029	47 days	10 days

Why shorten lifetimes? Because revocation works poorly (Chapter 17). A certificate that leaks or is mis-issued stays dangerous until it expires. Shorter lifetimes cap that window, and they force automation, which also makes it possible to replace certificates quickly when something goes wrong.

The validation reuse column matters as much as the lifetime. Today, a CA may validate a domain once and issue against that validation for months. By 2029, nearly every renewal will need fresh domain validation, so organisations must automate DCV through DNS APIs or delegated validation records, not only certificate installation.

Some CAs go further already. Let's Encrypt began offering optional **six-day certificates** in 2025, which need no revocation infrastructure at all, and uses them for its new IP address certificates.

16.5 Certificate Transparency

How would you know if a CA, through error or compromise, issued a certificate for your domain to someone else? Before 2013, you mostly would not. DigiNotar's fraudulent certificates were discovered only because one Chrome user's browser had a hard-coded expectation for Google's keys.

Certificate Transparency (CT), created at Google and specified in RFC 6962, makes all public certificates visible:

1. Before issuing, the CA submits the certificate to several public **CT logs**, independently operated append-only ledgers built as Merkle trees, so any tampering with history is detectable.
2. Each log returns a **Signed Certificate Timestamp (SCT)**, a promise to include the certificate.
3. The CA embeds the SCTs in the certificate. Browsers such as Chrome and Safari reject public certificates without enough valid SCTs.
4. **Monitors** watch the logs. Anyone can search them, for example through public search tools such as crt.sh, and domain owners can subscribe to alerts for new certificates naming their domains.

CT turned mis-issuance from invisible to detectable, and it has caught many incidents, from test certificates issued for well-known domains to CAs violating the rules. It also has a side effect worth knowing: every hostname in a public certificate becomes public. Internal hostnames such as `payroll-test.example.com` should not appear in public certificates; use a private CA or wildcards for those.

CURRENT DEVELOPMENT The CT ecosystem is moving to a newer log design called **static CT** (the "Sunlight" architecture and the static-ct-api specification), which serves logs as static files that are cheaper and more reliable to operate at today's volume of billions of certificates. Chrome began accepting these logs in 2025.

16.6 CAA: telling CAs who may issue

A **CAA** (Certification Authority Authorization) record is a DNS record that lists which CAs may issue certificates for a domain. CAs must check it before issuing.

```
example.com. CAA 0 issue "letsencrypt.org"
example.com. CAA 0 issue "ca-two.example"
example.com. CAA 0 issuewild ";"
example.com. CAA 0 iodef "mailto:security@example.com"
```

This example allows two CAs, forbids wildcard certificates entirely, and asks CAs to report refused requests to a security mailbox. Newer CAA parameters can also restrict issuance to a specific CA account or validation method, which makes it harder for an attacker who controls the website, but not your CA account, to obtain a certificate.

WATCH OUT If you use two CAs so that one can take over when the other fails (Chapter 19), CAA must list **both**. A CAA record that lists only your primary CA silently blocks your backup at the moment you need it.

16.7 How users see all this

Users see almost none of it. Browsers removed the padlock's prominence and the EV indicator, and now mainly warn when something is wrong. The web PKI works when it is invisible: certificates issued automatically, logged publicly, validated from multiple vantage points, short-lived, and from CAs under constant scrutiny.

HANDS-ON LAB

Lab: watch the web PKI at work

1. Search Certificate Transparency. Open `https://crt.sh` and search for a domain you own or a well-known one. Note how many certificates exist, which CAs issued them, and how quickly they are renewed. Search for `%.yourdomain.example` to find every subdomain that ever had a public certificate.

2. Read SCTs in a certificate.

```
echo | openssl s_client -connect www.mozilla.org:443 -servername www.mozilla.org 2>/dev/null \
| openssl x509 -noout -text | grep -A12 "CT Precertificate SCTs"
```

3. Check CAA records.

```
dig +short CAA google.com
dig +short CAA github.com
```

4. Count the days. For three sites you use, print the validity period and compare it with the SC-081 schedule:

```
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null \
| openssl x509 -noout -dates
```

CASE STUDY

Failure story: DigiNotar

In the summer of 2011, attackers broke into DigiNotar, a Dutch certificate authority whose roots were trusted by every major browser and by the Dutch government's own PKI. They issued over 500 fraudulent certificates, including one for `*.google.com`. That certificate was used to intercept the Gmail traffic of an estimated 300,000 users, mostly in Iran, by sitting in the middle of their connections with a certificate their browsers fully trusted.

The attack was discovered when Chrome, which had Google's expected keys hard-coded (an early form of pinning), refused the fraudulent certificate, and a user reported the error. DigiNotar's own logs had been tampered with, so the company could not even say which certificates had been issued. Within days, every major browser removed its roots. DigiNotar was declared bankrupt within weeks, and the Dutch government had to replace certificates across its own services.

DigiNotar shaped the modern web PKI. It is the reason Certificate Transparency exists, the reason root programs now enforce strict incident reporting, and the reason "any CA can issue for any domain" is treated as a risk to be monitored rather than a fact to be accepted.

KEY TAKEAWAYS

Chapter summary

- Any publicly trusted CA can issue for any domain; root programs, the Baseline Requirements, audits and transparency manage that risk.

- Root programs can and do distrust CAs. Organisations should be able to switch CAs quickly.
- DV certificates dominate; domain control is now checked from multiple network perspectives.
- Maximum public certificate lifetime is 200 days, falling to 100 in 2027 and 47 in 2029, with domain validation reuse falling to 10 days. Automate issuance and validation.
- Certificate Transparency makes every public certificate visible and monitorable; CAA restricts which CAs may issue.

CHECK YOURSELF

Review questions

1. Why is "any CA can issue for any domain" the central risk of the web PKI?
2. What is the difference between a root program and the CA/Browser Forum?
3. Why do modern distrusters apply to certificates issued after a cut-off date?
4. What attack does multi-perspective domain validation defend against?
5. Explain how Certificate Transparency would have exposed DigiNotar's fraudulent certificates within hours.
6. Write a CAA record set for a domain that uses two CAs and forbids wildcards.

Path Validation and Revocation

Receiving a certificate is easy; deciding whether to trust it is hard. This chapter explains how clients build and validate certificate chains, why revocation has been called the broken part of PKI, how CRLs, OCSP and stapling work, and why the industry is replacing revocation checks with short-lived certificates and browser-pushed revocation lists.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Describe the steps of certificate path building and validation
- Explain CRLs, OCSP, OCSP stapling and their weaknesses
- Explain soft-fail versus hard-fail and why browsers chose soft-fail
- Describe the move to CRLite, CRLSets and short-lived certificates and its impact on operations

IN THIS CHAPTER

- | | |
|------|--|
| 17.1 | From certificate to trust decision |
| 17.2 | Path building |
| 17.3 | Path validation |
| 17.4 | Revocation: the hard part |
| 17.5 | Soft-fail: why revocation rarely protects anyone |
| 17.6 | What browsers do instead |
| 17.7 | Revocation inside organisations |
| 17.8 | The right order for revocation |

17.1 From certificate to trust decision

When a client receives a certificate chain, it must answer one question: should I trust that this public key belongs to this name, for this purpose, right now? Answering takes two steps:

1. **Path building:** find a chain from the leaf certificate to a root in the trust store.
2. **Path validation:** check every certificate in that chain against a long list of rules.

17.2 Path building

Servers are supposed to send the leaf plus its intermediates, in order. In reality, chains arrive incomplete, out of order, with extra certificates, or with expired cross-signs. Because several valid paths can exist (cross-signing, Chapter 15), clients must search:

- Match each certificate's **Issuer** name and **Authority Key Identifier** against candidate issuers.
- Look for candidates in the certificates the server sent, in a local cache, in the trust store, and, in some clients, by downloading from the **Authority Information Access (AIA)** URL in the certificate.
- If a path fails (for example, it ends at an expired root), backtrack and try another.

Good path builders, as in modern browsers and current versions of major libraries, search all possibilities. Simpler ones take the first path and fail if it is bad, which caused the 2020 and 2021 root expiry outages described in Chapter 15. This difference is why the same server can work in a browser and fail in a mobile app or a Java service.

17.3 Path validation

Once a candidate path exists, the client checks it using the algorithm in RFC 5280, section 6. For each certificate, from the root down to the leaf:

Check	What fails it
Signature verifies with the issuer's public key	A forged or corrupted certificate
Current time is within Not Before and Not After	An expired or not-yet-valid certificate, or a wrong system clock

Check	What fails it
Issuer is allowed to be a CA (Basic Constraints CA:TRUE)	A leaf certificate pretending to be a CA
Path length constraints are respected	Too many CAs below a constrained intermediate
Name constraints are respected	An internal CA issuing for a domain outside its permitted space
Key usage permits certificate signing for CAs	Misused keys
Not revoked	A compromised or withdrawn certificate (sections 17.4 to 17.7)
Policy constraints are met	Rare outside government PKIs

Then, for the leaf only:

- The **hostname** must match a SAN entry.
- The **Extended Key Usage** must allow the purpose, such as serverAuth.
- For public TLS, browsers also check **Certificate Transparency** proofs (Chapter 16).

WATCH OUT Writing your own certificate validation is one of the most reliably dangerous things a developer can do. A 2012 study titled "The Most Dangerous Code in the World" found that many non-browser applications, including payment SDKs and cloud clients, disabled or broke validation, often through a single misused library option. Use your platform's validation, and never ship code that turns verification off "temporarily".

17.4 Revocation: the hard part

Sometimes a certificate must stop being trusted before it expires, usually because its private key was exposed or it was issued wrongly. **Revocation** is how the CA announces this. It has three classic mechanisms, and all of them have serious problems.

Certificate Revocation Lists (CRLs)

A **CRL** is a signed list of the serial numbers of revoked, unexpired certificates, published by the CA at the URL in the certificate's CRL Distribution Points extension and refreshed on a schedule (for example, every few days).

- **Strengths:** simple, cacheable, and private: downloading the whole list does not reveal which site you are visiting.

- **Weaknesses:** lists can grow to megabytes, so downloading them on every connection is impractical; and a newly revoked certificate stays trusted until clients fetch the next list.

OCSP

The **Online Certificate Status Protocol** (RFC 6960) lets a client ask the CA's responder about one certificate and receive a signed "good", "revoked" or "unknown" answer.

- **Strengths:** small, current answers.
- **Weaknesses:** an extra network request on every new connection, slowing page loads; a **privacy leak**, because the CA learns which sites each user visits; and a single point of failure, because if the responder is down, clients must decide what to do.

OCSP stapling

With **OCSP stapling**, the server fetches its own OCSP response periodically and attaches ("staples") it to the TLS handshake. The client gets fresh, signed status without contacting the CA. It fixes the speed and privacy problems, but only if every server enables it, and an attacker with a stolen key can simply not staple. The **Must-Staple** certificate extension was designed to require stapling, but it saw little adoption.

17.5 Soft-fail: why revocation rarely protects anyone

What should a browser do if it cannot reach the OCSP responder? If it refuses the connection (**hard-fail**), every responder outage breaks large parts of the web. So browsers **soft-fail**: if the check fails, they proceed as if the certificate were good.

Now consider the attacker. Someone able to intercept your connection with a stolen certificate is, by definition, in a position to block your OCSP request too. Soft-fail therefore protects you only from attackers who are not able to carry out the attack. Security researcher Adam Langley summarised it in 2014: soft-fail revocation checking is like a seat belt that snaps when you crash.

This is why Chrome stopped doing online OCSP checks for most certificates years ago.

17.6 What browsers do instead

Browsers moved revocation from the network moment to a push model:

- **CRLSets (Chrome)** and **disallowed lists (other platforms)** are curated lists of high-priority revocations, such as compromised intermediates and major incidents, pushed to browsers through their update mechanism.
- **CRLite (Firefox)** compresses the revocation status of all publicly trusted certificates, gathered from CRLs and CT logs, into a compact data structure using cascading Bloom filters, and pushes small updates to browsers several times a day. Firefox can then check revocation locally, privately and with hard-fail semantics.
- **Short-lived certificates** make revocation less necessary: if a certificate lives a few days, a leaked key is useful only briefly.

CURRENT DEVELOPMENT The industry is retiring OCSP. In 2023 the CA/Browser Forum made OCSP optional for public CAs and required CRLs instead (Ballot SC-063). Let's Encrypt, the largest CA, ended its OCSP service in August 2025 and now publishes revocation only through CRLs. Combined with lifetimes falling to 47 days, revocation for the public web is becoming a matter of CRLs aggregated by browsers, plus short validity, rather than live per-connection checks.

17.7 Revocation inside organisations

Inside private PKIs (Chapter 18), the picture is different and often better:

- You control both the CA and the clients, so you can configure **hard-fail** checks where they matter, such as for VPN or mTLS clients.
- CRLs for a private CA are usually small and can be cached and refreshed predictably.
- You can use very short lifetimes (hours or days) for workloads and devices, which is the most robust approach of all.

Whatever the mechanism, operations matter: CRL and OCSP endpoints must be highly available, CRLs must be re-signed before their **Next Update** time even if nothing changed, and the CA's revocation signing must be monitored. An expired CRL can cause hard-failing clients to reject every certificate the CA ever issued.

17.8 The right order for revocation

Revoking a certificate that is still installed takes the service down for every client that checks. The safe order is always:

1. **Replace** the certificate on every installation, with a new key.
2. **Verify** each installation now serves the replacement.

- 3. Revoke** the old certificate, with the correct reason code (key compromise, superseded, cessation of operation, affiliation changed).

With a key compromise, follow the same order, only faster. This is one of many reasons an accurate inventory of where every certificate is installed (Chapter 19) is essential.

HANDS-ON LAB

Lab: validate chains and check revocation

- 1. Validate a chain manually.** Save a server's certificates (as in Chapter 15's lab) into `leaf.pem` and `intermediate.pem`, then:

```
openssl verify -CAfile /etc/ssl/certs/ca-certificates.crt -untrusted intermediate.pem
leaf.pem
```

Remove `-untrusted intermediate.pem` and observe the failure: this is what a client sees from a server with an incomplete chain.

- 2. Fetch and read a CRL.**

```
openssl x509 -in leaf.pem -noout -ext crlDistributionPoints
curl -s -o ca.crl <URL-from-the-output>
openssl crl -in ca.crl -inform DER -noout -text | head -30
```

Note the **This Update** and **Next Update** times and count the revoked entries.

- 3. Query OCSP, where still offered.** Many CAs still run OCSP responders for now:

```
openssl x509 -in leaf.pem -noout -ocsp_uri
openssl ocsp -issuer intermediate.pem -cert leaf.pem -url <OCSP-URI> -resp_text | head -20
```

- 4. Test hostname and time failures.** Use `openssl verify` with `-verify_hostname` wrong.example and with `-atime` set to a date after expiry, and read the errors. These are the two most common real-world validation failures.

CASE STUDY

Failure story: the most dangerous code in the world

In 2012, researchers from Stanford and the University of Texas published "The Most Dangerous Code in the World: Validating SSL Certificates in Non-Browser Software". They tested widely used software that made TLS connections outside browsers: payment gateway libraries from major merchants, cloud storage clients, popular HTTP libraries, and mobile app frameworks.

They found that certificate validation was broken in a striking number of them. Some libraries checked the chain but not the hostname, so any valid certificate for any site was accepted. Others had confusing APIs where a parameter that looked like "verify hostname = true" actually meant something else. Many applications had simply disabled validation, often with a code comment about getting tests to pass. In every case, an attacker on the network could intercept supposedly secure connections, including payment traffic, with a self-made certificate.

The cryptography was sound; the checks around it were missing. The paper drove fixes across many libraries and is still a useful checklist: verify the chain, verify the hostname, never disable verification, and test that your code rejects bad certificates, not only that it accepts good ones.

KEY TAKEAWAYS

Chapter summary

- Trust decisions require path building, which may need backtracking, and path validation against RFC 5280 rules, plus hostname and purpose checks.
- CRLs are private but slow to update; OCSP is current but slow, leaky and fragile; stapling helps but is optional.
- Browsers soft-fail on revocation, which protects little, so they moved to pushed lists (CRLSets, CRLite) and rely on short lifetimes.
- OCSP is being retired from the public web; CRLs plus short-lived certificates are the future.
- Private PKIs can use hard-fail and very short lifetimes. Always replace before revoking.

CHECK YOURSELF

Review questions

1. Why can the same server work in a browser but fail in an older Java or mobile client?
2. List five checks performed during path validation.
3. Compare CRLs and OCSP on speed, privacy and freshness.
4. Explain soft-fail, and why it offers little protection against a real attacker.
5. How does CRLite let Firefox check revocation privately?
6. Why should a compromised certificate be replaced before it is revoked?

Building and Running a Private CA

Public certificate authorities serve the open internet. Everything else, from internal services and service meshes to employee devices, VPN clients and IoT fleets, is usually better served by a private CA that the organisation controls. This chapter shows how to design a private PKI, build a two-tier hierarchy step by step, protect its keys, and operate it without it becoming the next outage.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Decide when to use public versus private certificates
- Design a CA hierarchy with appropriate tiers, lifetimes and constraints
- Build an offline root and online issuing CA with open-source tools
- Write the policies, ceremonies and operational controls a private CA needs

IN THIS CHAPTER

- | | |
|------|------------------------------------|
| 18.1 | Why run a private CA |
| 18.2 | Designing the hierarchy |
| 18.3 | Policy documents |
| 18.4 | Protecting CA keys |
| 18.5 | Issuance, automation and enrolment |
| 18.6 | Operating the CA |

18.1 Why run a private CA

Use **public** certificates when clients you do not control must trust you: customer browsers, partner systems, public APIs. Use **private** certificates when you control the clients, or can configure them to trust your root:

Need	Why a private CA fits
Service-to-service mTLS	Thousands of short-lived certificates; internal names; client authentication
Internal web applications	Hostnames that must not appear in public CT logs
VPN and Wi-Fi client authentication	Device and user certificates with your own identity data
IoT and manufacturing	Device identities with lifetimes and rules public CAs do not offer
Kubernetes and service meshes	Automated issuance per workload, lifetimes in hours
Code signing for internal tools	Control over who can sign what

A private CA gives you control over lifetimes, names, extensions and issuance speed, and keeps internal hostnames out of public logs. It also gives you all the responsibility. If your private CA is compromised, an attacker can impersonate any internal service that trusts it, and if its CRL expires or its intermediate expires unnoticed, everything that depends on it can fail at once.

18.2 Designing the hierarchy

Most organisations need a **two-tier** hierarchy; large or regulated ones sometimes use three.

Tier	Role	Key storage	Online?	Typical lifetime
Root CA	Signs intermediates and its own CRL only	HSM, offline, in a safe	Powered on only for ceremonies, perhaps twice a year	10 to 20 years
Policy CA (optional middle tier)	Separates hierarchies by policy, business unit or region	HSM, offline	Rarely	5 to 10 years
Issuing CA	Issues leaf certificates every day	Network HSM or managed key service	Yes	2 to 5 years

Tier	Role	Key storage	Online?	Typical lifetime
Leaf certificates	Used by servers, workloads, devices, users	On the endpoint, ideally in hardware	n/a	Hours to months

Design rules that save pain later:

- **Separate issuing CAs by purpose.** For example: one for TLS servers, one for workload mTLS, one for devices, one for users. A problem in one, such as a compromise or a bad profile, does not affect the others.
- **Constrain intermediates.** Use `pathlen:0` so issuing CAs cannot create further CAs, Name Constraints so they can only issue for your domains, and EKU constraints so a device CA cannot issue code signing certificates.
- **Mind the lifetime ladder.** A CA cannot issue certificates valid beyond its own expiry. Renew issuing CAs well before their remaining life drops below the longest leaf lifetime you issue.
- **Choose algorithms deliberately.** ECDSA P-384 for CAs and P-256 for leaves is a common modern choice. Plan for post-quantum: a new root generated today may still be in service when ML-DSA is required (Chapter 27).

18.3 Policy documents

A trustworthy CA writes down its rules:

- A **Certificate Policy (CP)** says what the certificates may be used for and what assurance they carry.
- A **Certification Practice Statement (CPS)** says how the CA actually operates: identity checks, key protection, personnel controls, logging, revocation and disaster recovery.

The public web PKI requires these documents in a standard RFC 3647 structure. For a private CA, a shorter document following the same headings is enough, and it pays off in audits, incidents and handovers.

18.4 Protecting CA keys

The CA's private keys are the most sensitive part of the system. Standard controls:

- **Hardware.** Generate and keep CA keys inside a hardware security module (Chapter 23), where they cannot be exported in the clear. For a small internal CA, a FIPS-validated USB HSM or a managed cloud key service with non-exportable keys is a big improvement over a key file on disk.
- **Offline root.** Keep the root's HSM disconnected from any network and physically secured. Bring it out only during scheduled, documented ceremonies.
- **Multi-person control.** Require several people to activate the root, using M-of-N quorums of smart cards (for example, any 3 of 5 key holders).
- **Backups.** Back up CA keys in encrypted, split form to a second secure location. A root key lost to a hardware failure with no backup means rebuilding trust everywhere.

■ The key ceremony

A **key ceremony** is the formal, scripted event in which a root key is generated or used. A good ceremony has:

1. A written **script** listing every command and check, approved in advance.
2. Named roles: ceremony administrator, key custodians, an independent witness, and often an auditor.
3. A clean, verified computer that has never been and will never be networked.
4. **Video recording** and a signed log of every step, including hardware serial numbers and tamper-evident bag numbers.
5. Secure storage of all materials at the end, with sign-off by every participant.

It sounds theatrical. Its purpose is evidence: years later, you can prove the root key was created correctly, never left the HSM, and was only used for authorised purposes.

18.5 Issuance, automation and enrolment

An issuing CA must decide who may receive which certificate. Common enrolment methods:

Method	Best for
ACME (Chapter 19) with internal validation	Servers and workloads; the same automation as public certificates
EST or SCEP	Network devices, mobile device management, Wi-Fi certificates
Platform-native issuers	Kubernetes cert-manager, service mesh certificate authorities, workload identity systems such as SPIFFE/SPIRE

Method	Best for
Manual CSR approval	Rare, high-value certificates only

Open-source software that implements private CAs includes **step-ca** (Smallstep), **EJBCA** Community, the PKI engine in **OpenBao** (an open-source secrets manager), **cfssl**, and Microsoft **Active Directory Certificate Services** for Windows estates. Every major cloud also offers a managed private CA service.

WATCH OUT Windows estates often run Active Directory Certificate Services with permissive certificate templates. Researchers showed in 2021 ("Certified Pre-Owned") that misconfigured templates let ordinary domain users request certificates that authenticate as domain administrators. The attack paths, catalogued as ESC1 through ESC15 and counting, are now standard in penetration tests. Audit every template's enrolment rights and subject-name settings.

18.6 Operating the CA

Day-to-day operation is where private CAs fail. Plan for:

- **CRL publishing:** re-sign and publish CRLs before their Next Update time, from highly available locations. Monitor for stale CRLs.
- **CA certificate renewal:** track the expiry of every CA certificate in the hierarchy, and practise renewal before it is urgent.
- **Trust distribution:** decide how roots reach clients: operating system trust stores through device management, container base images, Java keystores, language-specific bundles. Each one is a separate place to update.
- **Logging and monitoring:** record every issuance and revocation; alert on unusual volumes or names.
- **Audit:** an annual review of the CP/CPS against what actually happens.
- **Disaster recovery:** a tested plan to restore the CA from backups, including HSM key restoration.

HANDS-ON LAB

Lab: build a two-tier private CA with OpenSSL

This lab builds a root and an issuing CA with OpenSSL so you see every moving part. In production, generate keys in an HSM and use a CA product; the concepts are identical.

1. Root CA (on a machine you will treat as offline).

```
mkdir -p pki/root && cd pki/root
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-384 -out root.key
openssl req -x509 -new -key root.key -sha384 -days 7300 -out root.crt \
  -subj "/O=Lab Corp/CN=Lab Corp Root CA 2026" \
  -addext "basicConstraints=critical,CA:TRUE" \
  -addext "keyUsage=critical,keyCertSign,cRLSign"
```

2. Issuing CA, constrained. Create its key and CSR, then sign it with the root:

```
cd .. && mkdir issuing && cd issuing
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-384 -out issuing.key
openssl req -new -key issuing.key -out issuing.csr -subj "/O=Lab Corp/CN=Lab Corp TLS Issuing CA 1"
cat > issuing.ext <<'EOF'
basicConstraints=critical,CA:TRUE,pathlen:0
keyUsage=critical,keyCertSign,cRLSign
extendedKeyUsage=serverAuth,clientAuth
nameConstraints=critical,permitted;DNS:.lab.internal
EOF
openssl x509 -req -in issuing.csr -CA ../root/root.crt -CAkey ../root/root.key \
  -CAcreateserial -days 1825 -sha384 -extfile issuing.ext -out issuing.crt
```

3. A leaf certificate for a service.

```
openssl req -new -newkey ec -pkeyopt ec_paramgen_curve:P-256 -nodes \
  -keyout api.key -out api.csr -subj "/CN=api.lab.internal"
printf
"subjectAltName=DNS:api.lab.internal\nextendedKeyUsage=serverAuth\nbasicConstraints=CA:FALSE\n" > leaf.ext
openssl x509 -req -in api.csr -CA issuing.crt -CAkey issuing.key -CAcreateserial \
  -days 30 -sha256 -extfile leaf.ext -out api.crt
openssl verify -CAfile ../root/root.crt -untrusted issuing.crt api.crt # OK
```

4. Test the name constraint. Issue a leaf for `api.example.com` from the same issuing CA and verify it. OpenSSL reports a name constraint violation: even a stolen issuing CA key cannot create trusted certificates outside `.lab.internal`.

5. Rebuild it the automated way. Install **step-ca**, initialise a CA with `step ca init`, and issue a certificate with `step ca certificate`. Compare the effort, and note that step-ca offers ACME out of the box (Chapter 19).

CASE STUDY

Failure story: the internal CA that expired on a Saturday

This pattern has been reported, in various forms, by many organisations, and makes the point of this chapter better than any single incident. An organisation builds an internal issuing CA with a five-year certificate. The engineer who built it leaves. Nobody records the CA's expiry date in any monitoring system, because monitoring watches leaf certificates, not CAs. Five years later, on a weekend, the issuing CA certificate expires.

Every certificate it ever issued, for service-to-service authentication, VPN access and internal websites, instantly becomes invalid, because a chain is only valid while every certificate in it is valid. Services cannot authenticate to each other, staff cannot connect to the VPN to fix anything, and the people who could re-sign the CA need to find the root key ceremony materials, which have not been touched in years.

Prevention is simple and unglamorous: track CA certificates in the same inventory as leaves, alert at least a year before CA expiry, keep ceremony scripts current, practise CA renewal on a schedule, and make sure more than one person knows how.

KEY TAKEAWAYS

Chapter summary

- Use private CAs for internal services, workloads, devices and users; they give control over lifetimes, names and issuance, and keep internal names private.
- A two-tier hierarchy with an offline root and purpose-specific, constrained issuing CAs suits most organisations.
- Protect CA keys in HSMs, with offline roots, M-of-N control, backups and documented ceremonies.
- Automate enrolment with ACME, EST/SCEP or platform-native issuers; audit Windows certificate templates.
- Operations decide success: CRLs, CA renewals, trust distribution, logging and disaster recovery.

CHECK YOURSELF

Review questions

1. Give three situations where a private CA is better than a public one, and one where it is not.
2. Why should issuing CAs be separated by purpose?
3. How do pathlen and Name Constraints limit the damage of a stolen issuing CA key?
4. What is the purpose of recording and witnessing a key ceremony?
5. Explain the "lifetime ladder" and how it can cause an outage.
6. Why did the expiry of one internal issuing CA break VPN access as well as services?

Certificate Lifecycle Automation at Scale

An expired certificate is the most avoidable outage in computing, and one of the most common. As public certificate lifetimes fall toward 47 days, manual renewal stops being merely risky and becomes impossible. This chapter explains how organisations discover, govern, issue, deploy, renew and replace certificates automatically, using ACME, inventory, policy and certificate lifecycle management platforms.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Explain how ACME and ACME Renewal Information automate issuance and renewal
- Design discovery, inventory and ownership for an organisation's certificates
- Choose provisioning models with private keys in mind and design safe renewal pipelines
- Plan CA failover and measure a certificate automation programme

IN THIS CHAPTER

- | | |
|-------|--|
| 19.1 | Why certificates cause outages |
| 19.2 | ACME: the protocol that made automation normal |
| 19.3 | The six jobs of certificate lifecycle management |
| 19.4 | Discovery and inventory |
| 19.5 | Policy |
| 19.6 | Provisioning, and where the private key lives |
| 19.7 | Renewal and escalation |
| 19.8 | Two CAs, and mass replacement |
| 19.9 | Access control and audit |
| 19.10 | Measuring the programme |

19.1 Why certificates cause outages

Every certificate has a fixed expiry date, and when it passes, every client that checks it refuses to connect. There is no grace period. Well-known examples include a mobile network outage affecting millions of users in 2018 caused by an expired certificate in network equipment software, a global collaboration-tool outage in 2020, and countless smaller incidents that never make the news.

In almost every case, the certificate was easy to replace. The real failures were:

- **Nobody knew it existed.** It was created years earlier by someone who left.
- **Nobody owned it.** Expiry warnings went to an unmonitored mailbox.
- **Renewal was manual.** Someone had to remember, request, install and restart.
- **Nobody checked the result.** A renewed certificate sat on disk while the service kept presenting the old one.

Shrinking lifetimes multiply the problem. Organisations typically renew when about a third of a certificate's life remains. For 1,000 public certificates, that means:

Maximum lifetime	Renew roughly every	Renewals per year for 1,000 certificates
398 days (until March 2026)	265 days	1,400
200 days (now)	133 days	2,700
100 days (from March 2027)	67 days	5,500
47 days (from March 2029)	31 days	11,800

At 47 days, those 1,000 certificates need about 45 renewals every working day. Automation is the only answer.

19.2 ACME: the protocol that made automation normal

The **Automatic Certificate Management Environment (ACME)**, RFC 8555, was created by the Internet Security Research Group for Let's Encrypt, which launched in 2015 and now issues certificates for hundreds of millions of websites. ACME lets software obtain and renew certificates with no human involved:

1. The **ACME client** creates an account key and registers with the CA's ACME server.
2. It places an **order** for the names it wants.

3. The server returns **challenges** to prove control of each name: - **HTTP-01**: serve a token at `http://<name>/.well-known/acme-challenge/<token>`. - **DNS-01**: publish a TXT record at `_acme-challenge.<name>`. The only method that supports wildcards and works for hosts not reachable from the internet. - **TLS-ALPN-01**: answer a special TLS handshake on port 443.
4. The server validates the challenges (from multiple network perspectives, Chapter 16).
5. The client sends a CSR, and downloads the certificate.

Clients exist for every environment: Certbot and lego for servers, built-in ACME in web servers such as Caddy, cert-manager in Kubernetes, and ACME support in many load balancers, CDNs and appliances. Most public CAs and many private CA products now offer ACME endpoints, often with **External Account Binding (EAB)**, which ties ACME accounts to a customer account at a commercial CA.

■ Delegating DNS validation safely

DNS-01 requires automation to write DNS records, and giving a script power over your whole DNS zone is risky. The common safer pattern is **delegation**: create a CNAME from `_acme-challenge.shop.example.com` to a record in a small, separate zone that only the ACME automation can write. The automation can answer challenges but cannot change any real records. This becomes essential as domain validation reuse falls to 10 days in 2029.

■ ACME Renewal Information (ARI)

ARI, standardised in 2025 as RFC 9773, lets the CA tell each client when to renew. Normally it suggests a window spread over time to avoid load spikes. If the CA must revoke certificates early, because of a validation bug, a mis-issuance or a key compromise, it can move the window forward, and ARI-aware clients renew within hours without anyone being paged. ARI turns mass revocation events from emergencies into routine.

19.3 The six jobs of certificate lifecycle management

ACME automates issuance for clients that support it. A large organisation also has certificates on appliances, legacy systems, cloud services and devices, issued by several CAs. **Certificate lifecycle management (CLM)** is the discipline, and often the product category, that covers all of them. It has six jobs:

1. **Discover** every certificate, wherever it is.
2. **Govern** with policy: allowed CAs, key types, names and lifetimes.
3. **Issue** by talking to public and private CAs.

4. **Provision:** install, bind and verify on the target.
5. **Control access:** who may request and approve what.
6. **Renew and revoke,** safely and on time.

Commercial CLM platforms, offered by specialist security vendors, by public CAs, and by cloud providers, combine these jobs with dashboards, workflows and integrations. Open-source tools and in-house automation can cover them too. The rest of this chapter describes the jobs in a product-neutral way, so the concepts apply whatever tools you use.

19.4 Discovery and inventory

You cannot renew a certificate you do not know about. No single discovery method finds everything:

Method	Finds	Misses
Network scanning	Certificates served on scanned IPs and ports, including unknown systems	Unscanned ports and networks; sites that need the right hostname (SNI) to answer
Integration discovery (reading a load balancer's, CDN's or cluster's certificate store through its API)	Everything stored there, including unused certificates, and how each is bound	Systems not integrated
CA account import	Everything bought through your CA accounts	Where certificates are installed; other CAs
Certificate Transparency monitoring	Every public certificate for your domains, from any CA	Private certificates; locations
Host agents or file scanning	Certificate and key files and keystores on disk	Hosts without agents

Combine several. Then turn raw findings into an **inventory** with two kinds of record, kept separate:

- **Certificate:** one per unique certificate (identified by fingerprint), with names, issuer, key type, expiry and owner.
- **Installation:** one per place the certificate is used, with the target, location, binding and last-seen time.

A dashboard that says "12,000 certificates" should always prompt the question: unique certificates, or installations? The numbers can differ by a factor of five.

Ownership is the hardest part

Every certificate needs an owning **team** (not a person, who may leave) and an escalation contact. Match certificate names to an application catalogue, use the requester recorded by the CA, use the owner of the hosting system, and route everything unresolved to an **unowned queue** that someone works through every week. Ownership coverage is the first metric to drive up, because every later control depends on someone receiving the alert.

19.5 Policy

Policy is enforced twice: **prevent** at request time (refuse a request that breaks the rules) and **detect** after discovery (flag existing certificates that break them). A practical baseline:

Setting	Typical rule
Allowed CAs	Named public CAs (at least two) and the internal CA
Key types	ECDSA P-256 preferred; RSA 2048 minimum, 3072 for long-lived keys
Names	Only domains the organisation owns; no internal names in public certificates
Wildcards	Only with approval; each one spreads one key across many servers
Lifetime	Public: just under the current maximum. Internal: shorter, such as 30 to 90 days, to force automation early
Key reuse	Never; every renewal creates a new key
Key location	Created on the target wherever possible

Layer the rules: a company baseline, then environment rules (internet-facing, internal, non-production), then application rules, each layer only tightening the one above. Exceptions must be written down with an owner and an end date.

19.6 Provisioning, and where the private key lives

Provisioning means getting the certificate, chain and key onto the target and making the target use them. Common models:

Model	Typical targets
Push through the target's management API	Load balancers, CDNs, firewalls, cloud certificate stores
Agent on the host	Servers, Windows and Java applications
Pipeline or CLI request at deploy time	Applications with mature CI/CD
Standard protocol (ACME, EST, SCEP)	Web servers, devices, anything with a built-in client
Through a platform issuer	Kubernetes cert-manager, service meshes, secrets managers with PKI engines

For every integration, ask the most important question in certificate operations: **where is the private key created, and who can read it afterwards?**

- **On the target** (best): the target creates the key and a CSR; only the certificate travels.
- **Centrally, then pushed** (sometimes necessary, for targets that cannot create CSRs or high-availability pairs that must share a key): store it encrypted, ideally under an HSM-protected key, send it only over encrypted management channels, and log every export.
- **Generated by someone else and uploaded by hand:** treat as an exception.

■ Install, bind, verify

Installing is not finishing. A complete deployment:

1. **Installs** the certificate and chain.
2. **Binds** it: updates the TLS profile, listener, route or ingress to use it.
3. **Synchronises** clustered or high-availability peers.
4. **Verifies from the outside:** connects as a client and confirms the new certificate, full chain and correct names are served.

Without the last step, automation reports success while users still receive the old certificate.

19.7 Renewal and escalation

Set the renewal point as a **percentage of lifetime** (for example, when one third remains), not a fixed number of days, so it adapts automatically as lifetimes shrink. Follow ARI where the CA supports it.

When renewal fails, the old certificate is still valid, so escalate in steps rather than all at once:

Time before expiry (200-day certificate)	Action	Who is told
About 66 days	Renewal starts	Nobody unless it fails
60 to 30 days	Automatic retries	Owning team
30 days	Still failing	Team and its manager; ticket raised
14 days	Still failing	Certificate platform on-call takes over
7 days	Still failing	Raised as an incident

Scale the steps to the lifetime. The rule is that no certificate reaches expiry with only an unread email as warning.

Agree with change management that automated renewals with unchanged names and policy are **standard changes**, pre-approved and automatically logged. If every renewal needs a change ticket and a meeting, automation stops being automatic.

19.8 Two CAs, and mass replacement

Several events force many certificates to change at once: a CA revoking certificates after a validation bug (one large CA had to revoke over 80,000 in 2024 within days), a CA being distrusted (Entrust in 2024), a CA outage, or a weak algorithm being banned. Prepare for all of them:

- ☐ Accounts with **at least two public CAs**, both configured in your automation, with templates that can switch CA with one change.
- ☐ **CAA records** listing both CAs on every domain.
- ☐ Domains and organisation details validated (or automatically validatable) at both.
- ☐ No clients **pinning** one CA's intermediate or root, or a documented exception for each that does.
- ☐ A **quarterly test**: issue and install a certificate from the backup CA.
- ☐ A **runbook**: select affected certificates by CA, intermediate or key type; prioritise internet-facing services; renew in batches within CA rate limits; track to zero; chase manual leftovers first.

19.9 Access control and audit

Whoever controls certificate automation can obtain trusted certificates for your domains and install them on your most exposed systems. Apply the same discipline as for any privileged system: single sign-on and group-based access, separate credentials for each pipeline and cluster, approval for first issuance of sensitive names but not for routine renewals, protected CA API credentials, and an audit log streamed to your security monitoring with alerts for key exports, CA credential changes and issuance outside policy.

19.10 Measuring the programme

Report a handful of metrics monthly, with trends:

Metric	Definition
Ownership coverage	Certificates with an owning team ÷ all certificates
Automation coverage	Certificates renewed without human action ÷ all certificates
Policy compliance	Certificates with no violations ÷ all certificates
Renewal success rate	Renewals completed automatically ÷ all renewals
Warning time	Days before expiry that a human was alerted, for failed renewals
Expiry incidents	Outages or failures caused by expired certificates (target: zero)
Out-of-band issuance	Public certificates for your domains found in CT but not issued through your process

Always report public and private certificates separately, and keep a named list of the certificates that cannot yet be automated. Those few are usually the ones that cause the next outage.

HANDS-ON LAB

Lab: automate a certificate end to end

1. Run a local ACME CA. Start step-ca with ACME enabled (or Pebble, Let's Encrypt's test ACME server):

```
docker run -d --name ca -p 9000:9000 -e "DOCKER_STEP_CA_INIT_NAME=Lab" \
  -e "DOCKER_STEP_CA_INIT_DNS_NAMES=localhost" -e "DOCKER_STEP_CA_INIT_ACME=true"
smallstep/step-ca
```

2. Issue with an ACME client. Use `lego` or `Certbot` against

`https://localhost:9000/acme/acme/directory`, with HTTP-01 on a local web server, and observe the order, challenge and finalisation steps in the client's verbose output.

3. Build a five-line inventory. Write a script that connects to a list of

`host:port:servername` entries, records subject, issuer, expiry and SHA-256 fingerprint to CSV, and flags anything expiring within 30 percent of its lifetime. Run it with and without SNI and compare.

4. Break verification on purpose. Renew a certificate on disk but do not reload the web server. Run your inventory script: the old expiry still appears. Reload, rerun, and see the new one. You have just reproduced the most common "successful renewal" outage.

CASE STUDY

Failure story: the certificate that switched off a security tool

In 2017, attackers stole personal data on about 147 million people from Equifax, a US credit bureau. They entered through an unpatched web application. They remained undetected for 76 days because a network monitoring device that inspected encrypted traffic leaving the affected systems had an **expired certificate**, and had had one for 19 months. With the certificate expired, the device could not decrypt traffic, so it silently stopped inspecting it. Nobody noticed, because no user-facing service broke.

On 29 July 2017, staff renewed the certificate, and the device almost immediately showed suspicious traffic, triggering the investigation. A US House of Representatives committee report later found that the company had allowed more than 300 security certificates to expire, including 79 used for monitoring business-critical domains, and concluded that a certificate management process with defined roles and responsibilities would have kept the monitoring device working.

Two lessons stand out. Certificates on security tools matter as much as those on websites, because when they expire, protection fails silently. And an outage is the lucky failure: the dangerous expiries are the ones nobody complains about. Count expired certificates in use, not only outages.

KEY TAKEAWAYS

Chapter summary

- Expired certificates cause outages and silently disable security controls. Lifetimes falling to 47 days make manual renewal impossible.
- ACME automates issuance and renewal; delegate DNS validation safely; use ARI so CAs can bring renewals forward.
- Lifecycle management means discover, govern, issue, provision, control access, and renew or revoke.
- Combine discovery methods, separate certificates from installations, and make ownership the first metric.
- Create keys on the target where possible; install, bind, sync and verify; renew at a percentage of lifetime and escalate in steps.
- Keep two tested CAs with CAA allowing both, and practise mass replacement.

CHECK YOURSELF

Review questions

1. Why does DNS-01 matter more as domain validation reuse falls to 10 days?
2. What does ACME Renewal Information let a CA do, and why does it matter during a mass revocation?
3. Why must inventories separate certificates from installations?
4. Why is "where is the private key created?" the first question for every integration?
5. Explain why renewal points should be a percentage of lifetime.
6. List four things that must be true for a backup CA to take over successfully.
7. Why did an expired certificate at Equifax lead to a breach rather than an outage?

PART VI

Keys, Secrets and Hardware

Managing keys and secrets across their whole life, the hardware that protects them, and the architecture and governance around it all.

In this part

- 20 Key Management Fundamentals
- 21 Key Management Services, BYOK and HYOK
- 22 Secrets Management and Workload Identity
- 23 HSMs, TPMs and Confidential Computing
- 24 Cryptographic Architecture, Agility and Governance

Key Management Fundamentals

Strong algorithms protect data only as well as their keys are managed. Key management is the discipline of generating, storing, distributing, using, rotating, backing up and destroying keys, and of proving that all of it was done correctly. This chapter lays out the key lifecycle, key hierarchies and envelope encryption, the rules for rotation and cryptoperiods, and the inventory that makes everything else possible.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Describe each stage of the key lifecycle and the controls each needs
- Design a key hierarchy using envelope encryption and key derivation
- Set cryptoperiods and rotation strategies with reasons
- Build a key inventory and assign ownership and policy to every key

IN THIS CHAPTER

20.1 Why key management is the real work

20.2 Types of keys

20.3 The key lifecycle

20.4 Key hierarchies and envelope encryption

20.5 Cryptoperiods and rotation

20.6 The key inventory

20.7 Key management policy and roles

20.1 Why key management is the real work

Kerckhoffs's principle (Chapter 1) says security should rest only on the key. The flip side is that **every weakness in how keys are handled becomes a weakness in the system.**

Real breaches rarely break AES. They find keys:

- in source code repositories and container images,
- in configuration files, environment variables and logs,
- in memory and crash dumps,
- in backups stored next to the data they protect,
- with administrators and services that should never have had them,
- long after they should have been rotated or destroyed.

NIST's **SP 800-57** (Recommendation for Key Management) is the most widely used reference for the discipline. This chapter follows its concepts in plain language.

20.2 Types of keys

Keys are classified by algorithm family and by purpose. Purpose matters most, because **each key should have exactly one purpose.**

Key type	Purpose	Example
Data encryption key (DEK)	Encrypts application data	A per-file AES-256 key
Key encryption key (KEK)	Encrypts (wraps) other keys	A master key in a key management service
Signing key	Creates digital signatures	Code signing, token signing, CA keys
Verification key	Checks signatures	Public keys in trust stores and JWKS
Key agreement key	Establishes shared secrets	Static or ephemeral ECDH, ML-KEM keys
MAC key	Creates and checks MACs	HMAC keys for API signing
Key derivation key	Derives other keys	An HKDF master secret

Using one key for two purposes, for example signing and encryption, or two different protocols, has repeatedly enabled cross-protocol attacks, as DROWN showed (Chapter 11). Separate keys are cheap; untangling shared ones after an incident is not.

20.3 The key lifecycle

Every key passes through the same states. SP 800-57 names them; the controls below are what matter in practice.

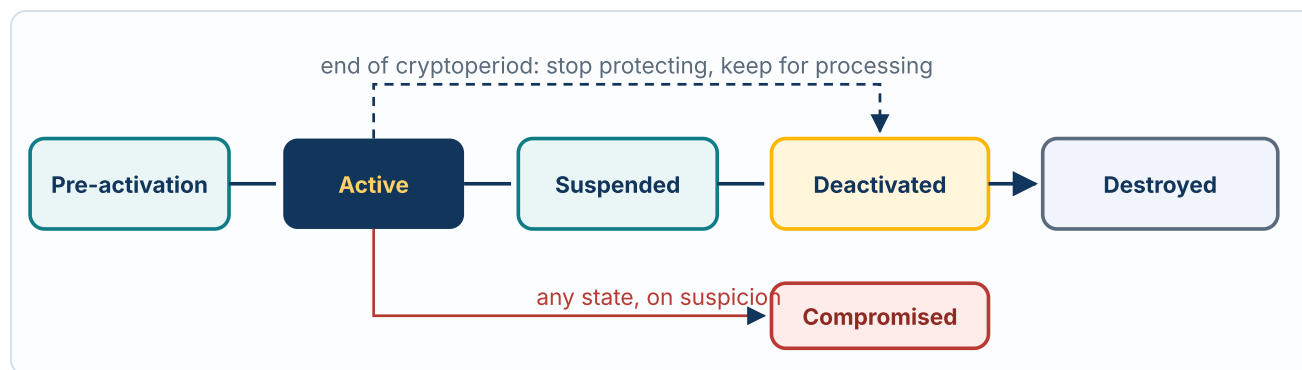


Figure 20.1 Key states, simplified from NIST SP 800-57. A deactivated key may still decrypt or verify old data, but never protects new data.

Stage	What happens	Controls that matter
Generation	The key is created	Approved algorithm and size; strong randomness (Chapter 3); generated inside the system that will protect it (an HSM or KMS)
Registration and distribution	The key is recorded, given metadata, and delivered to users	Inventory entry with owner, purpose and expiry; keys delivered wrapped, never in plain text
Storage	The key rests	Hardware protection for high-value keys; wrapped by a KEK otherwise; never alongside the data it protects
Use	The key performs its purpose	Access control per operation; logging; usage limits (for example, GCM message counts)
Rotation	A new key replaces it for new data	Scheduled cryptoperiods and event-driven rotation
Archival and backup	Copies kept for recovery	Encrypted, split or HSM-protected backups in a separate location; tested restores
Deactivation	The key stops protecting new data but may decrypt or verify old data	Enforced by the key system, not by convention
Destruction	All copies are removed	Including backups and caches; destruction recorded; data under the key becomes unrecoverable (crypto-shredding)

Crypto-shredding deserves a note: destroying a key makes everything encrypted under it permanently unreadable. Organisations use it to delete data from backups that cannot be edited, or to satisfy data-deletion requirements, by giving each customer or dataset its own key and destroying the key when the data must go.

20.4 Key hierarchies and envelope encryption

Encrypting large amounts of data directly with a master key in an HSM would be slow, and would expose the master key to heavy use. Instead, systems use **envelope encryption**:

1. Generate a fresh **data encryption key (DEK)** for each object (file, record, disk, backup).
2. Encrypt the data locally with the DEK, using AEAD.
3. Send the DEK to the key management system to be **wrapped** (encrypted) under a **key encryption key (KEK)** that never leaves the KMS or HSM.
4. Store the wrapped DEK next to the encrypted data, and discard the plain DEK from memory.

To decrypt, send the wrapped DEK to the KMS, receive the plain DEK (if authorised), and decrypt the data locally.

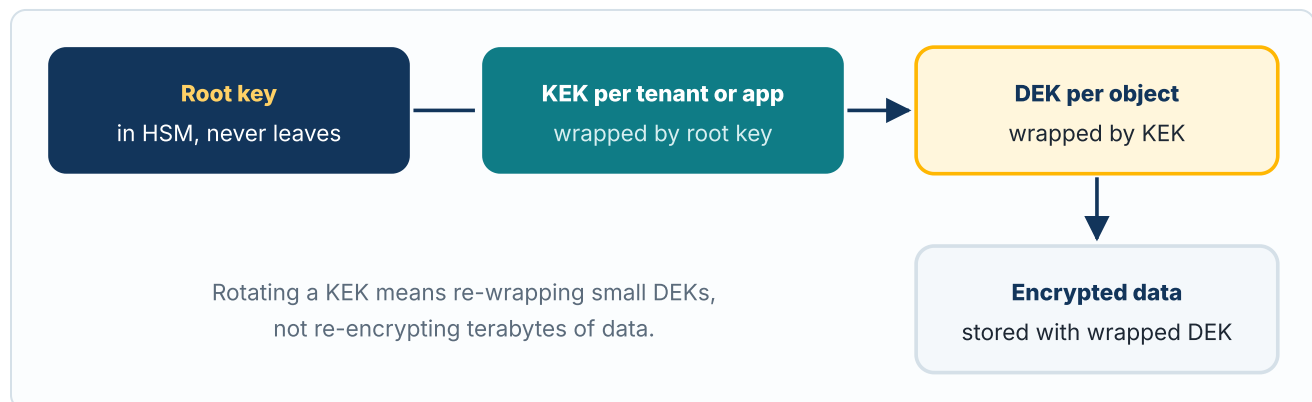


Figure 20.2 A three-level key hierarchy with envelope encryption.

The benefits are large:

- **Performance:** bulk encryption happens locally at full speed; the KMS handles only tiny wrapping operations.
- **Containment:** each object has its own DEK, so a leaked DEK exposes one object.

- **Cheap rotation:** rotating a KEK means re-wrapping DEKs, not re-encrypting the data.
- **Access control and audit:** every unwrap is an authorised, logged KMS call, so you can see who decrypted what and revoke access centrally.
- **Separation of duties:** storage administrators hold encrypted data; KMS administrators hold keys; neither alone can read the data.

Derived keys

An alternative to storing many DEKs is to **derive** them with HKDF from a master key and a context label such as a tenant ID or object ID (Chapter 6). Derivation avoids storing wrapped keys but makes rotation of the master key harder, because every derived key changes. Many systems combine the approaches: a wrapped per-tenant key from which per-object keys are derived.

20.5 Cryptoperiods and rotation

A **cryptoperiod** is the time a key is allowed to be used. Limiting it limits how much data any one key protects, how long a stolen key stays useful, and how much material an attacker can gather against one key.

SP 800-57 suggests cryptoperiods by key type. Typical practice:

Key	Typical cryptoperiod
Symmetric data encryption key	Up to 2 years for protecting; can decrypt longer
Key encryption key	1 to 2 years
TLS server key	Lifetime of its certificate; new key every renewal
Code signing key	1 to 3 years, with timestamps extending signature validity
CA keys	Root 10 to 20 years; issuing CA 2 to 5 years
API secrets and tokens	Days to months; ideally replaced with short-lived credentials

Rotation comes in two kinds:

- **Scheduled rotation** at the end of each cryptoperiod. Fully automated in a good system: new data uses the new key version, old versions remain available for decryption.
- **Event-driven rotation** after a suspected compromise, a staff departure with access, or a vulnerability. This is the one that tests whether your systems can actually rotate.

If rotation has never been practised, it will fail when you need it most.

WATCH OUT Rotation of a KEK protects new data immediately but does not re-protect data wrapped under the old version. If an old KEK is believed compromised, you must re-wrap (or re-encrypt) everything it protected, then destroy it.

20.6 The key inventory

You cannot manage keys you do not know about. A **key inventory** records, for every key:

- An identifier and version.
- Algorithm, size and purpose.
- Where it lives (which HSM, KMS, application or file) and whether it can be exported.
- Owner (a team) and the systems that use it.
- Creation date, cryptoperiod, and next rotation date.
- What it protects (data classification, regulatory scope).
- Its current lifecycle state.

The inventory is also the foundation for the post-quantum migration. A **cryptographic bill of materials (CBOM)** extends the idea to every use of cryptography in an application, including algorithms in libraries and protocols, not only managed keys (Chapter 27).

20.7 Key management policy and roles

A short written policy, approved by security leadership, should state at least: approved algorithms and sizes; where each class of key must be stored; cryptoperiods; who may create, use, export, rotate and destroy keys; logging and review; backup and recovery; and how compromise is handled.

Separate the roles. The people who administer the key management system should not be the people who own the data, and destroying or exporting high-value keys should require two people. This **separation of duties** and **dual control** is a requirement in standards such as PCI DSS for payment keys.

HANDS-ON LAB

Lab: envelope encryption in practice

This Python lab simulates a KMS with a local master key. Replace the "KMS" functions with calls to a real key management service later.

```
import os
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.keywrap import aes_key_wrap, aes_key_unwrap

KEK = AESGCM.generate_key(bit_length=256) # lives in the KMS in real life
def kms_wrap(dek): return aes_key_wrap(KEK, dek)
def kms_unwrap(wdek): return aes_key_unwrap(KEK, wdek)

def encrypt_object(data, object_id):
    dek = AESGCM.generate_key(bit_length=256)
    nonce = os.urandom(12)
    ct = AESGCM(dek).encrypt(nonce, data, object_id) # object ID as associated data
    return {"wdek": kms_wrap(dek), "nonce": nonce, "ct": ct, "id": object_id}

def decrypt_object(blob):
    dek = kms_unwrap(blob["wdek"])
    return AESGCM(dek).decrypt(blob["nonce"], blob["ct"], blob["id"])

b = encrypt_object(b"customer record 42", b"rec-42")
print(decrypt_object(b))
```

Now extend it:

- 1. Rotate the KEK.** Create a new KEK, unwrap each stored DEK with the old one, re-wrap with the new one, and confirm decryption still works. Note that the data ciphertext never changed.
- 2. Crypto-shred.** Delete one object's wrapped DEK and show its data is unrecoverable.
- 3. Context binding.** Try decrypting `rec-42`'s ciphertext with the ID `rec-43` and observe the failure: associated data stops records being swapped.

CASE STUDY

Failure story: the keys in the repository

In January 2023, a widely used continuous integration service disclosed that an attacker had stolen customer secrets: environment variables, tokens and keys that customers had stored for their build pipelines. The intrusion began with malware on one engineer's laptop that stole a session token, giving the attacker access to production systems, including the data and the encryption keys needed to decrypt it. The service told all customers to rotate every secret they had ever stored with it.

The incident showed two things. Encryption at rest helped little, because the attacker reached a place where keys and data were both accessible. And many customers discovered they could not rotate quickly: secrets had been copied into dozens of places, had no recorded owner, or were long-lived cloud credentials used by systems nobody remembered. Organisations with an inventory, short-lived credentials and practised rotation recovered in hours; others took weeks.

Envelope encryption with keys in a separate, access-controlled KMS, short-lived credentials instead of stored secrets (Chapter 22), and a live inventory are the direct answers.

KEY TAKEAWAYS

Chapter summary

- Breaches find keys more often than they break algorithms. Key management is the core of applied cryptography.
- Give every key one purpose, and manage it through generation, storage, use, rotation, backup, deactivation and destruction.
- Envelope encryption gives speed, containment, cheap rotation, central access control and audit.
- Set cryptoperiods; automate scheduled rotation and practise event-driven rotation.
- Keep a key inventory with owners and states; it is also the foundation of post-quantum migration.

CHECK YOURSELF

Review questions

1. Why should each key have exactly one purpose?
2. Walk through envelope encryption and name four benefits.
3. What is crypto-shredding, and when is it useful?
4. Why does rotating a KEK not, by itself, protect data wrapped under the old KEK?
5. List the fields a key inventory entry should contain.
6. What made recovery from the 2023 CI secrets breach slow for many customers?

Key Management Services, BYOK and HYOK

Most organisations no longer manage keys by hand. They use key management services: cloud KMS offerings, enterprise key managers, and the HSMs underneath them. This chapter explains how these services work, how to design their access policies, what bring-your-own-key and hold-your-own-key really give you, and how standards such as KMIP and PKCS#11 connect applications to keys.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Explain the architecture and trust model of a cloud KMS and an enterprise key manager
- Write least-privilege key policies and separate key administration from key use
- Compare BYOK, HYOK and external key management and the guarantees each provides
- Choose between cloud KMS, enterprise key managers and dedicated HSMs for a given workload

IN THIS CHAPTER

- | | |
|------|--------------------------------------|
| 21.1 | What a key management service does |
| 21.2 | Cloud KMS |
| 21.3 | Designing key policies |
| 21.4 | BYOK, HYOK and external keys |
| 21.5 | Enterprise key managers and KMIP |
| 21.6 | PKCS#11 and other cryptographic APIs |
| 21.7 | Choosing the right home for a key |

21.1 What a key management service does

A **key management service (KMS)** stores keys, usually inside HSMs, and performs operations with them on request, so that applications never handle the master keys directly. A typical KMS offers:

- **Key creation** with chosen algorithms, often with the option of HSM-backed storage.
- **Cryptographic operations by API:** encrypt and decrypt small data, wrap and unwrap DEKs, sign and verify, generate random bytes, compute MACs.
- **Access policies** controlling which identities can perform which operations on which keys.
- **Automatic rotation** with key versions.
- **Audit logs** of every operation.
- **Integrations** with storage, databases and other services, which use the KMS for envelope encryption behind the scenes.

The defining property: **master keys are non-exportable**. You can ask the KMS to use a key, but you cannot ask it to give you the key.

21.2 Cloud KMS

All major cloud providers offer a KMS. The details differ, but the model is the same:

1. You create a **customer-managed key** in a region.
2. Cloud services (object storage, block storage, managed databases, data warehouses) are configured to protect your data with it, using envelope encryption.
3. When a service needs to read your data, it asks the KMS to unwrap a DEK, and the KMS checks that the request is allowed by your key policy.
4. Every request appears in your audit log.

Clouds typically offer several protection levels: keys protected by software inside the provider's hardened KMS, keys in multi-tenant HSMs validated to FIPS 140-3 level 3, and dedicated single-tenant HSMs that you manage. They also distinguish **provider-managed keys** (encryption on by default, but you control nothing) from **customer-managed keys** (you control policy, rotation and deletion).

IN PRACTICE Default encryption with provider-managed keys protects against stolen disks in a data centre, a threat you could not address anyway. Customer-managed keys add something real: you control who may decrypt, you can see every use, and you can make data unreadable by disabling the key.

21.3 Designing key policies

A KMS is only as safe as its access policy. Good policies follow these rules:

- **Separate administration from use.** Key administrators can create, rotate, disable and schedule deletion of keys, but cannot encrypt or decrypt with them. Applications can use keys but cannot change their policies. No human routinely has both.
- **Least privilege by operation.** An application that only writes data needs `encrypt` or `generate data key`, not `decrypt`. A verifier needs only the public key.
- **Scope keys narrowly.** One key per application, environment and data class, rather than one company-wide key. A breach or misconfiguration then has a limited blast radius.
- **Use conditions.** Restrict use to particular networks, services or contexts. Many KMSs support **encryption context**: key-value pairs bound to each ciphertext as associated data and checked on decryption, so a wrapped key for tenant A cannot be unwrapped in a request for tenant B.
- **Protect deletion.** Deletion is irreversible and destroys all data under the key. Use mandatory waiting periods (typically 7 to 30 days), alerts, and separate approval.
- **Watch the logs.** Alert on policy changes, disabling or scheduling deletion, decryption from unusual identities or locations, and spikes in volume.

21.4 BYOK, HYOK and external keys

As organisations moved sensitive data to the cloud, they asked: can the provider read our data? Several models answer this, with very different guarantees.

Model	How it works	What it gives you	What it does not give you
Provider-managed keys	Provider creates and controls keys	Encryption at rest by default	Control, visibility or revocation

Model	How it works	What it gives you	What it does not give you
Customer-managed keys (CMK)	You create and control keys in the provider's KMS	Policy control, audit, ability to disable	Independence from the provider's KMS
BYOK (bring your own key)	You generate key material in your own HSM and import it into the provider's KMS	Proof of how the key was generated; an escrow copy outside the cloud	Protection from the provider: once imported, the key is used inside their KMS
HYOK / external key management (EKM, XKS)	The master key stays in your own key manager or HSM, outside the cloud. The cloud KMS calls your key manager for each wrap or unwrap	The provider cannot decrypt without your system's cooperation; you can cut off access instantly	Simplicity and availability: your key manager is now in the critical path of every cloud data access

Two points are often misunderstood.

BYOK is mostly about generation and escrow, not control. Importing a key gives you assurance about its origin and lets you keep a backup, but the provider's KMS uses it like any other key. If your goal is that the provider *cannot* decrypt, BYOK alone does not achieve it.

HYOK trades availability for control. If your external key manager is unreachable, every service depending on it stops. HYOK suits a small set of highly sensitive datasets, often driven by data sovereignty requirements, rather than everything.

CURRENT DEVELOPMENT European data sovereignty requirements have driven rapid growth in external key management and "sovereign cloud" offerings, where keys are held by an EU-based operator or the customer. Regulators increasingly ask not only "is data encrypted?" but "who can access the keys, and under which jurisdiction?"

21.5 Enterprise key managers and KMIP

Inside their own data centres, organisations use **enterprise key managers**: appliances or software, usually backed by HSMs, that centrally manage keys for storage arrays, databases, backup systems, tape libraries and virtualisation platforms.

Many of these consumers speak the **Key Management Interoperability Protocol (KMIP)**, an OASIS standard. KMIP defines a common language for creating, retrieving, locating, rotating and destroying keys and other managed objects, so that a storage array from one vendor can use a key manager from another. Typical KMIP uses:

- Self-encrypting drives and storage arrays fetching their encryption keys at boot.

- Database transparent data encryption (TDE) master keys.
- Backup and tape encryption keys.
- Hypervisor VM encryption keys.

When evaluating an enterprise key manager, check: KMIP version and profile support, HSM backing and FIPS 140-3 validation level, high availability and clustering across sites, backup and disaster recovery of key material, role separation and quorum controls, and audit integration with your security monitoring.

21.6 PKCS#11 and other cryptographic APIs

Applications reach keys in HSMs and key managers through a handful of APIs:

API	What it is	Typical users
PKCS#11 (Cryptoki)	The standard C interface to cryptographic tokens and HSMs: sessions, objects, mechanisms	CAs, TLS servers, signing systems, OpenSSL via providers
Microsoft CNG / KSP	Windows cryptography API with pluggable key storage providers	Windows services, AD CS
Java JCA/JCE	Java's provider-based cryptography architecture	Java applications
OpenSSL 3 providers	Plug-in modules that route OpenSSL operations to HSMs, TPMs or KMSs	Linux services
KMIP	Key lifecycle management protocol	Storage, databases, backup
Cloud KMS REST APIs	HTTPS APIs with cloud identity	Cloud-native applications

A practical rule: put the decision about **where a key lives** in configuration, not in application code. Code that calls a generic interface (a PKCS#11 provider, an OpenSSL provider, or a small internal key service) can move from a software key to an HSM or a cloud KMS without rewriting, which is exactly the agility the post-quantum transition will demand.

21.7 Choosing the right home for a key

Need	Usually best served by
Encrypting data in cloud services	Cloud KMS with customer-managed keys

Need	Usually best served by
Sovereignty or provider-independence for a sensitive dataset	External key management (HYOK) for that dataset only
Storage arrays, databases, backups on premises	Enterprise key manager with KMIP
Root CA, code signing, payment keys	Dedicated HSMs with quorum controls (Chapter 23)
Very high-volume signing or TLS offload	HSMs or cloud HSMs sized for throughput
Developer secrets and API tokens	A secrets manager (Chapter 22), not a KMS

HANDS-ON LAB

Lab: least-privilege key policy

You can do this lab in any major cloud's free tier, or with the open-source **OpenBao** transit engine locally.

1. Create a symmetric key for an application called `orders`.
2. Create two identities: `orders-writer` and `orders-reader`.
3. Grant `orders-writer` only the operations needed to encrypt or generate data keys; grant `orders-reader` only decrypt.
4. Encrypt a value as the writer with an encryption context of `{"app": "orders", "env": "prod"}`.
5. Try to decrypt as the writer (denied), as the reader with the same context (allowed), and as the reader with `{"env": "test"}` (denied).
6. Open the audit log and find all five requests, including the denials.
7. Disable the key and confirm that even the reader can no longer decrypt. Re-enable it.

Write down how you would detect, within minutes, if someone scheduled this key for deletion.

CASE STUDY

Failure story: the key policy that let everyone in

A recurring pattern in cloud security reviews and published breach reports is not broken cryptography but **over-broad key policies**. A common example: a key created early in a project grants decrypt permission to a wildcard principal, such as every role in the

account, "to make things work". Data is encrypted with a customer-managed key, dashboards show "encrypted at rest: 100 percent", and auditors are satisfied. But any compromised workload or over-privileged user in the account can decrypt everything.

In the 2019 breach of a large US bank's cloud environment, an attacker exploited a server-side request forgery flaw in a web application firewall to obtain the credentials of the role it ran under. That role had permission to list and read storage buckets containing over 100 million customer records. The data was encrypted at rest, but the stolen role was authorised to read it, so the platform decrypted it transparently.

Encryption at rest only helps against attackers who are *not* authorised to use the key. The value of a KMS is in its policy: narrow keys, least-privilege grants, conditions, and monitored use.

KEY TAKEAWAYS

Chapter summary

- A KMS performs operations with non-exportable master keys on request, with policy and audit, and powers envelope encryption across cloud and enterprise services.
- Customer-managed keys add real control; provider-managed keys mainly protect against physical theft.
- Separate key administration from key use; grant least privilege by operation; bind encryption context; protect deletion; monitor logs.
- BYOK proves key origin and gives escrow; HYOK keeps the provider from decrypting but puts your key manager in the critical path.
- KMIP, PKCS#11 and provider interfaces let applications and devices reach keys; keep key location in configuration for agility.

CHECK YOURSELF

Review questions

1. What does "non-exportable" mean for a KMS master key, and why does it matter?
2. Why should key administrators not be able to decrypt data?
3. What does encryption context protect against?
4. Explain why BYOK does not stop a cloud provider from using your key.
5. When is HYOK worth its availability cost?
6. In the 2019 cloud breach, why did encryption at rest not protect the data?

Secrets Management and Workload Identity

Passwords, API keys, database credentials, tokens and private keys are scattered through every modern system, and leaked secrets are now one of the most common starting points for breaches. This chapter explains how to store and deliver secrets safely, how dynamic and short-lived credentials remove whole classes of risk, and how workload identity is replacing stored secrets altogether.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Explain why secrets sprawl and how leaked secrets are exploited
- Design central secrets storage, delivery and rotation
- Use dynamic secrets and short-lived credentials to shrink exposure
- Explain workload identity, OIDC federation and SPIFFE and when they remove the need for secrets

IN THIS CHAPTER

- | | |
|------|--|
| 22.1 | What counts as a secret |
| 22.2 | Secret sprawl |
| 22.3 | Central secrets management |
| 22.4 | Rotation |
| 22.5 | Dynamic secrets and short-lived credentials |
| 22.6 | Workload identity: replacing secrets with identity |
| 22.7 | Detecting and responding to leaks |

22.1 What counts as a secret

A **secret** is any value that grants access or capability if someone else learns it:

- Passwords for databases, service accounts and administrators.
- API keys and access tokens for internal and third-party services.
- Cloud access keys.
- Private keys for TLS, SSH, signing and encryption.
- Webhook signing secrets, HMAC keys and session encryption keys.
- Connection strings that embed any of the above.

Keys managed in a KMS or HSM (Chapter 21) are a special case: they never leave their protected boundary. Most other secrets must be delivered to the software that uses them, which is where the trouble starts.

22.2 Secret sprawl

Secrets spread because copying them is easy and every copy feels harmless:

- Hard-coded in source code "temporarily".
- Committed in configuration files and `.env` files.
- Baked into container images and virtual machine templates.
- Pasted into chat messages, tickets and wiki pages.
- Stored as long-lived variables in CI/CD systems.
- Printed in logs and error messages.

Once in a Git repository, a secret is effectively permanent: deleting the file does not remove it from history, forks or clones. Attackers scan public repositories continuously, and leaked cloud keys are typically abused within minutes of being pushed. Industry studies by secret-scanning vendors have counted tens of millions of secrets leaked to public repositories in a single year, with a large share still valid years later.

IN PRACTICE Treat any secret that has appeared in a repository, image, ticket or log as compromised. Revoke and replace it first; clean up history second.

22.3 Central secrets management

A **secrets manager** is a hardened service that stores secrets encrypted (usually with envelope encryption under a KMS or HSM key), controls access to them by identity and policy, audits every read, and supports rotation. Examples include the secrets services of every major cloud, open-source systems such as **OpenBao**, Kubernetes secret operators that sync from them, and commercial privileged access management products.

Core features to look for:

Feature	Why it matters
Encryption at rest under a KMS/HSM key	Storage compromise does not reveal secrets
Identity-based access policies	Each workload reads only its own secrets
Audit log of every read and change	You can answer "who accessed this?"
Versioning and rotation	Change a secret without breaking consumers
Leases and expiry	Secrets that expire on their own
Dynamic secrets	Credentials created on demand, per consumer (section 22.5)
High availability	The secrets manager becomes a dependency of every service

Getting secrets to applications

Storing secrets centrally solves half the problem. Delivery is the other half:

- **Read at start-up through an API or agent**, keeping the secret only in memory. Preferred.
- **Mounted files** written by an agent or sidecar into a memory-backed filesystem, refreshed when rotated.
- **Environment variables**. Common but weaker: they leak into crash reports, child processes and debugging tools.
- **Kubernetes Secrets**. By default only base64-encoded in etcd. Enable encryption at rest with a KMS provider, restrict RBAC tightly, or sync from an external secrets manager.

For configuration stored in Git (GitOps), tools such as **SOPS** and **Sealed Secrets** encrypt secret values so the encrypted file can live in the repository and be decrypted only by authorised deployment systems.

■ The bootstrap problem

To read secrets, an application must authenticate to the secrets manager, which seems to need a secret of its own: the "secret zero". The answer is to authenticate with something the platform already proves about the workload, such as its cloud instance identity, its Kubernetes service account token, or its SPIFFE identity (section 22.6). Those identities are issued and rotated by the platform, so no human-handled secret is needed.

22.4 Rotation

Every static secret should have an owner, a rotation schedule and a tested rotation procedure. Rotation without downtime usually follows a **two-phase** pattern:

1. Create the new secret and make it valid **alongside** the old one (for example, a database user with two valid passwords, or an API with two active keys).
2. Update every consumer to use the new secret, and verify.
3. Revoke the old secret.

Systems that allow only one valid credential at a time force a risky switch-over and are a common reason rotation is postponed forever. Prefer systems that support overlapping credentials.

22.5 Dynamic secrets and short-lived credentials

The strongest way to reduce secret risk is to stop having long-lived secrets at all.

Dynamic secrets are created on demand for one consumer and expire automatically. When an application starts, it asks the secrets manager for database credentials; the secrets manager creates a new database user with exactly the needed permissions and a lease of, say, one hour, renews it while the application runs, and deletes it when the lease ends. Benefits:

- Every credential is unique, so the audit trail shows exactly which instance did what.
- A leaked credential expires on its own, usually before anyone could use it.
- There is nothing to rotate, because nothing lives long enough.

The same principle appears throughout this book: SSH certificates valid for hours (Chapter 13), TLS certificates valid for days (Chapter 19), OAuth access tokens valid for minutes (Chapter 14). **Short lifetimes are the most reliable revocation mechanism ever invented.**

22.6 Workload identity: replacing secrets with identity

Modern platforms can prove a workload's identity cryptographically, without any stored secret. The workload then exchanges that proof for short-lived credentials.

Cloud workload identity and OIDC federation

Cloud platforms give each virtual machine, function or container an identity, and issue short-lived credentials for it automatically. Across platforms, **OIDC federation** connects them: a CI/CD system or a Kubernetes cluster issues a signed OIDC token (Chapter 14) describing the workload, such as "repository X, branch main, workflow deploy", and the cloud provider exchanges that token for temporary cloud credentials, after checking a trust policy.

Federation has removed long-lived cloud keys from many CI/CD pipelines. The trust policy is now the security boundary: it must check the token's issuer, audience and the specific subject claims (repository, branch, environment), or any workflow from any repository on the same CI platform could assume the role.

SPIFFE and SPIRE

SPIFFE (Secure Production Identity Framework for Everyone), a graduated Cloud Native Computing Foundation project, standardises workload identity across platforms:

- Every workload gets a **SPIFFE ID**, a URI such as `spiffe://example.org/payments/api`.
- The identity is delivered as an **SVID** (SPIFFE Verifiable Identity Document): a short-lived X.509 certificate for mTLS, or a JWT.
- **SPIRE**, the reference implementation, **attests** each workload, checking evidence such as the node it runs on and its process or container properties, before issuing an SVID, and rotates SVIDs automatically, typically every hour or less.

Service meshes use the same idea: each workload gets an automatically rotated certificate, and services authenticate each other with mTLS, with no passwords or API keys between them.

22.7 Detecting and responding to leaks

Even with good practices, secrets leak. Layer detection:

- **Pre-commit and CI scanning** for secret patterns before code is pushed.

- **Repository and image scanning** across history and registries.
- **Platform push protection**: major code-hosting platforms block pushes containing recognised secrets and notify issuers, many of whom revoke leaked keys automatically.
- **Honeytokens**: fake credentials placed where attackers look, which alert when used.
- **Anomaly detection** on secrets manager and cloud audit logs.

And prepare the response before you need it: for each class of secret, a runbook that says how to revoke, replace and verify, who owns it, and how to find every place it was used.

HANDS-ON LAB

Lab: from static secret to dynamic credentials

Use OpenBao in development mode (or any secrets manager you have access to) and a PostgreSQL container.

1. **Static secret.** Store a database password in the key-value engine, read it with a token, and note the audit log entry.
2. **Dynamic secret.** Enable the database secrets engine, configure it with an administrative connection to PostgreSQL, and define a role that creates users with read-only access and a one-hour time to live. Request credentials twice and observe two different usernames. Connect with one.
3. **Expiry.** Revoke the lease, or wait for it to expire, and confirm the database user no longer exists.
4. **Scan for leaks.** Run an open-source secret scanner such as `gitleaks` against a test repository where you have committed a fake key, and then against its history after "deleting" the file.

CASE STUDY

Failure story: the CI pipeline with keys to everything

In 2022, a ride-sharing company suffered a breach that began with social engineering: an attacker bought a contractor's stolen password and repeatedly sent multi-factor approval requests until the contractor accepted one. Once inside the network, the attacker found a network share containing PowerShell scripts. One script contained hard-coded

administrator credentials for the company's privileged access management system, the very system meant to protect secrets. With those, the attacker obtained access to cloud consoles, internal tools and the security team's own systems.

The pattern is common: a single long-lived secret, stored in a place many people could read, unlocks the vault that holds every other secret. The defences are the ones in this chapter: no secrets in scripts or shares; scanning to find the ones that slip through; strong, phishing-resistant authentication for people (Chapter 14); and workload identity and short-lived credentials so that the most powerful access is never a static string someone can copy.

KEY TAKEAWAYS

Chapter summary

- Secrets sprawl into code, images, CI systems, logs and chat; leaked secrets are exploited within minutes.
- Store secrets centrally, encrypted under KMS keys, with identity-based access and audit; deliver them in memory, not in images or repositories.
- Rotate with overlapping credentials; prefer systems that allow two valid secrets at once.
- Dynamic secrets and short-lived credentials remove the need for rotation and limit damage from leaks.
- Workload identity (cloud identity, OIDC federation, SPIFFE) replaces stored secrets with platform-attested identity; scan for and rehearse responses to leaks.

CHECK YOURSELF

Review questions

1. Why is a secret committed to Git and then deleted still compromised?
2. What is "secret zero", and how does workload identity solve it?
3. Describe a two-phase rotation and why systems need overlapping credentials.
4. How do dynamic database credentials improve both security and auditability?
5. What must an OIDC federation trust policy check, and what goes wrong if it does not?
6. In the 2022 breach, which single secret turned a contractor compromise into broad access?

HSMs, TPMs and Confidential Computing

Software can be copied, dumped and debugged; well-designed hardware resists all three. This chapter explains how hardware security modules protect the most valuable keys in the world, how FIPS 140-3 and Common Criteria certify them, how TPMs and secure enclaves bring hardware protection to every device, and how confidential computing extends protection to data while it is being used.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Explain what an HSM protects against and how its key hierarchy, partitions and quorum controls work
- Interpret FIPS 140-3 levels and Common Criteria claims in procurement
- Plan a key ceremony, backup and high availability for HSM-protected keys
- Explain TPMs, secure enclaves, remote attestation and confidential computing and their limits

IN THIS CHAPTER

- | | |
|------|---|
| 23.1 | Why hardware |
| 23.2 | Inside an HSM |
| 23.3 | Certification: FIPS 140-3 and Common Criteria |
| 23.4 | Operating HSMs |
| 23.5 | TPMs and secure elements |
| 23.6 | Confidential computing |

23.1 Why hardware

A key stored in a file or in application memory can be copied by anyone who gains administrator access, reads a backup, captures a crash dump (as in the Storm-0558 case, Chapter 9), or exploits a memory-disclosure bug (Heartbleed, Chapter 12). Once copied, it can be used anywhere, forever, without leaving a trace.

A **hardware security module (HSM)** is a hardened device that generates, stores and uses keys inside a tamper-resistant boundary. Applications send requests ("sign this", "unwrap that"); the HSM performs the operation and returns the result. **The key never leaves the boundary in usable form.** An attacker who fully compromises the server attached to an HSM can misuse it while they have access, but cannot take the key away. Remove the attacker and the key is safe again.

23.2 Inside an HSM

Component	Purpose
Tamper-resistant enclosure	Sensors detect drilling, opening, voltage or temperature attacks and erase (zeroise) secrets
Secure cryptographic processor	Performs operations with side-channel countermeasures
True random number generator	Physical entropy source for key generation
Internal key hierarchy	A master key protects all stored keys; keys stored outside the HSM are wrapped under it
Partitions (or slots, domains)	Isolated logical HSMs for different applications or tenants, each with its own keys and credentials
Role-based access	Separate security officer, crypto officer and crypto user roles
Quorum (M-of-N) authentication	Sensitive actions require several people with separate credentials, often smart cards
Audit log	Tamper-evident record of administrative and key operations

HSMs come in several forms: network-attached appliances shared by many servers, PCIe cards inside a single server, USB devices for small CAs and offline roots, and cloud HSM services where the provider hosts dedicated or multi-tenant HSMs that you control.

Key attributes matter

When a key is created in an HSM, its **attributes** decide what anyone can ever do with it. In PKCS#11 terms:

- `CKA_SENSITIVE = true`: the key value cannot be read in the clear.
- `CKA_EXTRACTABLE = false`: the key cannot be exported at all, even wrapped.
- `CKA_SIGN`, `CKA_DECRYPT`, `CKA_WRAP`: which operations are allowed.

A key created as extractable can later be exported, which defeats much of the point. Review attributes during key creation ceremonies.

23.3 Certification: FIPS 140-3 and Common Criteria

Buyers rely on independent certification to know an HSM does what it claims.

FIPS 140-3

FIPS 140-3 is the US and Canadian standard for cryptographic modules, administered by NIST's **Cryptographic Module Validation Program (CMVP)** and aligned with ISO/IEC 19790. It has four security levels:

Level	Physical security requirement	Typical products
1	No physical protection beyond production-grade components	Software cryptographic libraries
2	Tamper evidence (seals, coatings) and role-based authentication	Some USB tokens, smart cards
3	Tamper resistance with active zeroisation; identity-based authentication; separation of key entry	Most network HSMs and cloud HSMs
4	Complete envelope protection against environmental and physical attacks	Specialised high-assurance modules

A FIPS certificate covers a specific module version, configuration and set of approved algorithms. Running the module outside its validated configuration, such as with non-approved algorithms enabled, means you are not operating a validated module.

CURRENT DEVELOPMENT The older FIPS 140-2 standard reached its end: on 21 September 2026, all remaining FIPS 140-2 validation certificates moved to the CMVP's Historical list. Historical modules may continue in existing deployments, but US federal agencies should not include them in new procurements. Check that any HSM, library or appliance you buy now holds an active FIPS 140-3 certificate, and ask vendors when their post-quantum algorithms (ML-KEM, ML-DSA, SLH-DSA) will appear in a validated configuration.

Common Criteria

Common Criteria (ISO/IEC 15408) is an international certification framework. Products are evaluated against **Protection Profiles** to an assurance level (EAL). In Europe, HSMs used for qualified electronic signatures under eIDAS must be certified against specific Common Criteria protection profiles. Common Criteria certificates name exact versions and configurations too.

Payment HSMs

Payment networks use specialised **payment HSMs** for PIN processing, card verification and key exchange with card schemes, certified under **PCI PTS HSM**. Their key management follows payment-specific standards such as ANSI X9.24 (including TR-31 key blocks), with strict dual control and split knowledge.

23.4 Operating HSMs

Owning an HSM is easy; operating one well takes discipline.

Key ceremonies. Generating and using the most sensitive keys, such as root CAs, follows a scripted, witnessed, recorded ceremony (Chapter 18), with quorum smart cards held by different people.

Backup and recovery. Keys inside an HSM die with it unless backed up. HSM vendors provide secure backup: keys exported only in encrypted form to another HSM of the same family or to backup devices, protected by the HSM's domain key. Test restoration regularly. An HSM key that cannot be recovered after a hardware failure can be as damaging as one that was stolen.

High availability. Production HSMs run in clusters, with keys replicated between devices and sites, and clients that fail over automatically. Plan capacity for peak operations per second, especially for signing-heavy workloads such as TLS termination or token issuance.

Firmware and lifecycle. HSM firmware updates fix vulnerabilities and add algorithms, but can change the validated configuration. Plan updates as projects, not patches. Track hardware end-of-support dates years ahead; migrating keys between HSM generations is a significant project.

Monitoring. Forward HSM audit logs to your security monitoring. Alert on administrative logins, partition changes, failed authentications, and unusual operation volumes.

23.5 TPMs and secure elements

A **Trusted Platform Module (TPM)** is a small security chip, or firmware equivalent, present in almost every modern PC and server. TPM 2.0 provides:

- **Protected key storage:** keys created in the TPM can be made non-exportable and bound to that machine.
- **Measured boot:** each stage of the boot process records a hash of the next stage in **Platform Configuration Registers (PCRs)**. Keys can be **sealed** so they are released only if the PCRs match an expected, trusted state.
- **Remote attestation:** the TPM signs a statement of the PCR values, letting a remote service verify what software the machine booted.

Practical uses: full-disk encryption keys sealed to measured boot (BitLocker and Linux equivalents), device identity certificates for network access, protected storage for credentials such as Windows Hello keys, and host attestation in data centres.

Phones and laptops have similar dedicated **secure elements** and **secure enclaves** that protect biometric templates, payment credentials and passkeys (Chapter 14). They are why a passkey's private key cannot be copied off a device.

WATCH OUT TPMs are not HSMS. They are inexpensive, slow, and certified to modest levels. Some have had serious flaws: ROCA (Chapter 7) affected TPMs, and TPM-Fail (2019) extracted ECDSA keys through timing. Use TPMs for device-bound keys and attestation, not as a substitute for an HSM for CA or high-value signing keys.

23.6 Confidential computing

Encryption protects data at rest and in transit, but data must be decrypted to be processed, and then it is exposed to the operating system, the hypervisor, and whoever runs the infrastructure. **Confidential computing** protects data **in use** by running workloads in hardware-isolated **trusted execution environments (TEEs)**:

Technology	What it isolates
Intel SGX	Small application enclaves inside a process
Intel TDX	Whole virtual machines from the hypervisor
AMD SEV-SNP	Whole virtual machines, with memory encryption and integrity
Arm CCA	Realms isolated from the hypervisor on Arm servers
Confidential GPUs	AI and data processing on accelerators, with encrypted transfers

Memory inside a TEE is encrypted with keys held by the processor, so even a malicious administrator or cloud operator cannot read it. **Remote attestation** lets a relying party verify, before sending data or releasing keys, that a workload is running inside a genuine TEE with the expected code. A common pattern ties this to key management: the KMS releases a decryption key only to a workload that presents a valid attestation report.

Confidential computing is growing quickly, particularly for processing sensitive data in shared clouds and for protecting AI models and their inputs. Its limits are real, though: researchers have published many side-channel and fault attacks against TEEs, particularly SGX, and the security of the whole depends on trusting the processor vendor's design and attestation keys. Treat it as an extra layer that significantly raises the bar, not as an absolute guarantee.

HANDS-ON LAB

Lab: a software HSM with PKCS#11

SoftHSM2 is a free software implementation of a PKCS#11 token, ideal for learning the interfaces real HSMs use. It provides no hardware protection.

```
sudo apt-get install softhsm2 opensc
softhsm2-util --init-token --slot 0 --label "lab" --so-pin 12345678 --pin 1234
MOD=/usr/lib/softhsm/libsofthsm2.so
pkcs11-tool --module $MOD --login --pin 1234 --keypairgen --key-type EC:prime256v1 \
  --label signing-key --id 01
pkcs11-tool --module $MOD --login --pin 1234 --list-objects
```

Note the key attributes printed (sensitive, never extractable). Now sign data without the key ever leaving the token:

```
echo "release 1.0" > data.txt
pkcs11-tool --module $MOD --login --pin 1234 --sign --id 01 -m ECDSA-SHA256 \
  --input-file data.txt --output-file data.sig
```

Try to export the private key with `--read-object --type privkey`. It is refused. Then explore using the token from OpenSSL 3 through a PKCS#11 provider, the same way a CA or web server would use a real HSM.

CASE STUDY

Failure story: the cloud key in a crash dump

The Storm-0558 incident (Chapter 9) is, at its heart, an HSM story. The stolen consumer signing key had been created in 2016 and was used in a way that allowed it to appear in process memory. Microsoft's investigation suggested a crash dump containing the key was moved from a highly isolated production environment into a debugging environment, where the attacker later found it, although the company later said it could not confirm this sequence with certainty.

If the key had been generated and used exclusively inside an HSM with non-extractable attributes, no crash dump could have contained it. In its response, Microsoft announced that it had moved consumer and enterprise signing keys into HSM-backed key storage with automatic rotation. The US Cyber Safety Review Board recommended that cloud providers protect identity signing keys with hardware and rotate them regularly. The lesson for every organisation: the keys that can mint identities, sign software or issue certificates belong inside hardware, where memory bugs and operational accidents cannot reach them.

KEY TAKEAWAYS

Chapter summary

- HSMS keep keys inside a tamper-resistant boundary; attackers can at most misuse them while present, never take them away.
- Key attributes such as non-extractable are set at creation and decide what can ever be done with a key.
- FIPS 140-3 levels and Common Criteria certify modules; FIPS 140-2 certificates moved to Historical status in September 2026.
- Operating HSMS means ceremonies, tested backups, high availability, planned firmware updates and monitored logs.
- TPMS and secure enclaves bring hardware key protection and attestation to every device; confidential computing protects data in use, with real but bounded assurance.

CHECK YOURSELF

Review questions

1. What can an attacker who compromises a server attached to an HSM do, and what can they not do?
2. Why do key attributes like CKA_EXTRACTABLE matter at key creation time?
3. What does FIPS 140-3 Level 3 add over Level 2?
4. What does it mean that FIPS 140-2 certificates are now Historical, and how does it affect purchasing?
5. Explain measured boot and sealing in a TPM.
6. How can remote attestation be combined with a KMS to release keys only to trusted workloads?

Cryptographic Architecture, Agility and Governance

Individual tools are not enough; they must be assembled into systems that remain secure as people, threats and algorithms change. This chapter brings Parts I to VI together into architecture practice: threat modelling cryptographic systems, designing for crypto agility, mapping controls to the major compliance regimes, and responding when a key or algorithm fails.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Threat model a cryptographic system and find its weakest trust boundary
- Design for crypto agility at the protocol, application and organisational levels
- Map cryptographic controls to common compliance frameworks
- Run incident response for a key compromise or an algorithm break

IN THIS CHAPTER

- | | |
|------|--|
| 24.1 | Thinking in systems |
| 24.2 | Threat modelling cryptographic systems |
| 24.3 | Crypto agility |
| 24.4 | Compliance landscape |
| 24.5 | Governance |
| 24.6 | Incident response for cryptographic failures |

24.1 Thinking in systems

A cryptographic system is more than its algorithms. It includes:

- The **keys** and where they live (KMS, HSM, files, devices).
- The **identities** that may use each key (people, workloads, services).
- The **protocols** that carry protected data.
- The **trust anchors** that decide what is believed (root certificates, JWKS endpoints, pinned keys).
- The **processes** around them: issuance, rotation, approval, backup, monitoring and recovery.

Attackers look for the weakest link across all of these. In practice it is almost never the algorithm.

24.2 Threat modelling cryptographic systems

Use a structured method. A simple, effective one has five steps:

1. **Draw the data flow.** Where is sensitive data created, stored, transmitted and processed? Mark each **trust boundary**: the points where data crosses between systems, networks or organisations with different levels of trust.
2. **List the keys.** For each protection on the diagram, identify the key, where it lives, who can use it, and how it is rotated.
3. **Ask the key questions** at every boundary: - Who can decrypt this data, in practice, including administrators, cloud providers and backups? - Who can sign or issue something others will trust? - What happens if this key leaks? How would we know? How fast could we replace it? - What does this trust anchor accept, and who can change it?
4. **Apply a threat catalogue.** STRIDE (spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege) is a good checklist: spoofing maps to authentication and certificates, tampering to integrity and signatures, repudiation to signing and audit logs, and so on.
5. **Prioritise by blast radius.** A key that protects one record is less important than one that can mint every identity. Spend protection where a single failure has the widest effect.

Blast radius as a design principle

Blast radius is how much is affected if one thing fails. Many of this book's recommendations simply reduce it:

Design choice	Blast radius reduced
Per-object DEKs (envelope encryption)	One leaked key exposes one object
Per-tenant or per-application KEKs	One misconfigured policy exposes one tenant
Separate issuing CAs by purpose	One compromised CA affects one certificate type
Name constraints on CAs	A stolen CA key cannot issue for other domains
Short-lived credentials and certificates	A leaked credential works for hours, not years
Ephemeral key exchange	A stolen server key does not expose past sessions

24.3 Crypto agility

Crypto agility is the ability to change algorithms, keys, certificates or providers quickly and safely, without redesigning systems. Every major event in recent cryptographic history has tested it: SHA-1's retirement, the removal of TLS 1.0, ROCA, mass revocations, CA distrusting, and now the post-quantum transition, which requires replacing nearly all public-key cryptography.

Agility has three layers.

Protocol and format agility

Protocols should negotiate algorithms, carry algorithm identifiers with every key, signature and ciphertext, and allow new options without breaking old clients. TLS 1.3 is a good example: adding the hybrid X25519MLKEM768 group needed no protocol change. Formats should record which algorithm and key version protected each object, so data can be re-protected later.

Agility has a cost: every negotiable option is attack surface, as two decades of downgrade attacks showed. The modern balance is **agile, but with few, strong choices at any time**, and deliberate retirement of old ones.

Application agility

Applications become rigid when algorithm choices are hard-coded throughout the code. Make them agile by:

- Calling cryptography through a small number of internal libraries or services, not directly throughout the code base.
- Keeping algorithm, key size and key location in configuration.
- Avoiding assumptions about sizes. Post-quantum keys and signatures are much larger than today's (Chapter 26); fixed-length database columns, protocol fields and buffers will break.
- Testing with alternative algorithms in continuous integration.

■ Organisational agility

The hardest layer is organisational:

- An accurate **inventory** of keys, certificates and cryptographic usage (a CBOM, Chapter 27).
- **Owners** for every cryptographic asset.
- **Automation** for issuance, rotation and deployment.
- **Rehearsed procedures** for mass replacement.
- **Procurement rules** that require vendors to support algorithm upgrades and publish timelines.

24.4 Compliance landscape

Most organisations must satisfy one or more regimes with cryptographic requirements. The table is a map, not legal advice.

Framework	Scope	Key cryptographic requirements
PCI DSS v4.0.1	Payment card data	Strong cryptography for stored and transmitted card data; documented key management with dual control and split knowledge; an inventory of cryptographic suites and protocols; annual review of cryptographic choices
FIPS 140-3 and FIPS-approved algorithms	US federal systems and many regulated buyers	Validated modules; approved algorithms only
NIST SP 800-57, 800-131A	Guidance used widely beyond government	Key management practice; algorithm transitions and deadlines
SOC 2	Service organisations	Encryption and key management as part of security and confidentiality criteria
ISO/IEC 27001:2022	Information security management	Control 8.24 "Use of cryptography": policy, key management, lifecycle

Framework	Scope	Key cryptographic requirements
HIPAA	US health data	Encryption as an addressable safeguard; breach safe harbour for properly encrypted data
GDPR	EU personal data	Encryption as an appropriate technical measure; reduced breach notification duties for encrypted data
EU NIS2 and DORA	Critical sectors and EU financial entities	Cryptography policies, key management, and ICT risk management; DORA applies since January 2025
eIDAS 2.0	EU electronic identification and trust services	Qualified signatures, certificates and wallets, with certified hardware
CNSA 2.0	US national security systems	Specific algorithms (AES-256, SHA-384, ML-KEM-1024, ML-DSA-87, LMS/XMSS) and post-quantum deadlines

Two practical observations:

- Regulators increasingly require a **cryptographic inventory**. PCI DSS 4.0 already does, and post-quantum guidance from governments makes it a near-universal expectation.
- Compliance is the floor. A system can pass an audit with keys in files readable by every administrator. Design to the threat model; then show compliance.

24.5 Governance

A cryptography programme needs:

- **A policy** approved by leadership: approved algorithms, key lengths, protocols, storage requirements and lifetimes, reviewed at least annually.
- **Standards and patterns**: approved, reusable designs (for example "how we encrypt a database field", "how services authenticate to each other") so teams do not reinvent them.
- **A design review** step for new systems that handle sensitive data or keys.
- **Exceptions** with owners and end dates.
- **Metrics** reported to leadership: inventory coverage, automation coverage, policy compliance, expiry incidents, time to rotate a key on demand.
- **Clear ownership** for PKI, KMS and secrets platforms, each with an on-call rotation.

24.6 Incident response for cryptographic failures

Two kinds of cryptographic incident need prepared playbooks.

■ Key compromise

When a key is known or suspected to be exposed:

1. **Contain.** Disable the key or remove its access where possible (disable a KMS key, revoke a token's permissions, remove a compromised CA from trust stores).
2. **Assess scope.** Use the inventory: what does the key protect or sign, where is it used, who trusts it, and since when could it have been exposed?
3. **Replace.** Issue new keys and certificates, deploy them everywhere, and verify.
4. **Revoke and re-protect.** Revoke certificates after replacement (Chapter 17). Re-wrap or re-encrypt data protected by a compromised KEK. Re-sign artefacts if a signing key was compromised, and publish which signatures to distrust.
5. **Investigate and notify.** Determine how the exposure happened, review logs of the key's use for misuse, and meet regulatory and contractual notification duties.
6. **Learn.** Fix the root cause and update the playbook.

■ Algorithm or implementation break

When a widely used algorithm, library or device is found weak (ROCA, Heartbleed, a new attack on a mode):

1. Query the inventory for every affected system, key and certificate.
2. Prioritise by exposure and blast radius.
3. Patch, then **replace keys** generated by or exposed to the flaw.
4. Track remediation to zero and report progress.

Speed in both cases depends almost entirely on work done in advance: the inventory, automation and rehearsals.

HANDS-ON LAB

Lab: threat model a key hierarchy

Choose a system you know, or use this example: a multi-tenant SaaS application that stores customer documents in cloud object storage, encrypts them with per-document DEKs wrapped by per-tenant KEKs in a cloud KMS, and lets support staff view documents

on request.

1. Draw the data flow and trust boundaries.
2. List every key, where it lives, who can use it, and its rotation.
3. Apply STRIDE at each boundary.
4. Find the single change that would most reduce blast radius.
5. Write the first three steps of a playbook for "a support engineer's credentials were phished".
6. Identify everything that would need to change for the post-quantum transition (Part VII).

Compare your answer with a colleague's. The differences are usually where the real risks hide.

CASE STUDY

Failure story: the update server everyone trusted

In June 2017, a destructive malware outbreak known as NotPetya spread across the world, causing an estimated 10 billion US dollars in damage to shipping lines, pharmaceutical companies, logistics firms and many others. It began with the update server of a widely used Ukrainian accounting software package. Attackers had compromised the vendor's infrastructure and pushed a malicious update that customers' systems installed automatically, because it came through the trusted update channel.

Cryptography was not broken; it was bypassed by an architecture that concentrated trust in one weakly protected place. The update mechanism had limited protection, and once the attackers controlled the vendor's servers, every customer trusted what they delivered. Inside victim networks, the malware then spread using stolen administrator credentials, showing the second half of the blast radius problem.

The lessons are architectural: sign updates with keys kept in hardware and separate from distribution servers; verify signatures on the client; limit what automatically installed software can reach; segment networks so one compromised host cannot reach everything; and protect administrative credentials as the high-value keys they are.

KEY TAKEAWAYS

Chapter summary

- Cryptographic systems include keys, identities, protocols, trust anchors and processes; attackers seek the weakest link across all of them.
- Threat model by drawing data flows and trust boundaries, listing keys, asking who can decrypt or sign, and prioritising by blast radius.
- Crypto agility spans protocols and formats, applications and organisations; build it before you need it, because the post-quantum transition requires it.
- Compliance frameworks increasingly require inventories and documented key management, but compliance is the floor, not the goal.
- Prepare playbooks for key compromise and algorithm breaks; speed depends on inventory, automation and rehearsal.

CHECK YOURSELF

Review questions

1. Why is the algorithm rarely the weakest link in a cryptographic system?
2. Give four design choices that reduce blast radius and explain each.
3. What are the three layers of crypto agility, and which is usually hardest?
4. Why can unlimited algorithm negotiation be a security risk?
5. Which compliance regimes in section 24.4 require a cryptographic inventory, explicitly or implicitly?
6. Outline the six steps of a key compromise playbook.
7. What architectural weaknesses turned one compromised update server into a global incident in 2017?

PART VII

The Post-Quantum Transition

Why quantum computers threaten today's public-key cryptography, the new standards that replace it, and how to migrate.

In this part

- 25 The Quantum Threat
- 26 The New Algorithms: ML-KEM, ML-DSA, SLH-DSA and Beyond
- 27 Migrating to Post-Quantum Cryptography

The Quantum Threat

A sufficiently large quantum computer would break RSA, Diffie-Hellman and elliptic curve cryptography, the public-key foundations of the internet. No such machine exists yet, but data stolen today can be decrypted later, and replacing cryptography across an organisation takes many years. This chapter explains what quantum computers can and cannot do, how close they are, and the deadlines governments have set.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Explain in plain terms why Shor's algorithm breaks RSA and ECC and Grover's algorithm only weakens symmetric cryptography
- Assess quantum risk using data lifetime, migration time and threat timeline
- Describe the current state of quantum hardware and resource estimates
- Summarise the main government deadlines for post-quantum migration

IN THIS CHAPTER

- | | |
|------|--|
| 25.1 | What a quantum computer is, briefly |
| 25.2 | Shor's algorithm: the end of RSA and ECC |
| 25.3 | Grover's algorithm: a smaller threat to symmetric crypto |
| 25.4 | How close are we? |
| 25.5 | Mosca's inequality |
| 25.6 | Harvest now, decrypt later |
| 25.7 | Government timelines |
| 25.8 | What not to do |

25.1 What a quantum computer is, briefly

A classical computer stores bits that are either 0 or 1. A quantum computer uses **qubits**, which can be in a **superposition** of 0 and 1, and can be **entangled** so that their states are linked. A quantum algorithm arranges a computation so that, when the qubits are measured, wrong answers cancel each other out and right answers reinforce each other, through **interference**.

Two misconceptions are worth clearing away:

- Quantum computers are **not** simply faster classical computers that "try every answer at once". For most problems they offer no advantage at all.
- They are good at specific problems with special mathematical structure. Unfortunately for us, the problems behind today's public-key cryptography have exactly that structure.

Today's machines have from tens to around a thousand **physical qubits**, which are noisy: they make errors constantly. Useful, long computations need **error correction**, which combines many physical qubits into one reliable **logical qubit**. A machine able to break cryptography is usually called a **cryptographically relevant quantum computer (CRQC)**.

25.2 Shor's algorithm: the end of RSA and ECC

In 1994, Peter Shor showed that a quantum computer could find the **period** of a function efficiently. Factoring a number and computing a discrete logarithm can both be turned into period-finding problems. So a CRQC running **Shor's algorithm** could:

- Factor RSA moduli, recovering private keys from public keys.
- Solve discrete logarithms, breaking Diffie-Hellman and DSA.
- Solve elliptic curve discrete logarithms, breaking ECDH, ECDSA and EdDSA.

Larger keys do not help: the work grows only modestly with key size. **Every public-key algorithm in Parts III to VI, as deployed today, falls to Shor's algorithm.** This includes the key exchange in TLS, SSH, IPsec and messaging, the signatures on certificates, software and tokens, and RSA key wrapping in HSMs.

25.3

Grover's algorithm: a smaller threat to symmetric crypto

Grover's algorithm (1996) speeds up unstructured search. Finding one key among 2^n possibilities takes about $2^{(n/2)}$ quantum steps instead of 2^n classical ones. On paper, this halves the security of symmetric keys and hash preimages: AES-128 drops to 64-bit security.

In practice the threat is much smaller:

- Grover's steps must run **one after another** on an error-corrected machine; they cannot be split efficiently across many machines the way classical key searches can.
- Each step involves running AES as a quantum circuit, which is enormously expensive.

Expert consensus, reflected in NIST guidance, is that AES-128 remains acceptable and AES-256 is comfortably secure against quantum attack. SHA-256 and SHA-3 remain secure for their uses. The response to Grover is simply to prefer 256-bit symmetric keys for long-term protection.

Cryptography	Quantum attack	Effect	Response
RSA, DH, ECDH, ECDSA, EdDSA	Shor	Completely broken	Replace with ML-KEM, ML-DSA, SLH-DSA
AES-128	Grover	Weakened on paper, practically safe	Prefer AES-256 for long-lived data
AES-256	Grover	Safe	None needed
SHA-256, SHA-384, SHA-3	Grover	Safe for practical purposes	None needed
HMAC, HKDF	Grover	Safe	None needed

25.4

How close are we?

No one knows exactly when a CRQC will exist, but three trends inform the estimate.

Hardware is improving. Companies and national programmes are building machines with more and better qubits. In December 2024, Google demonstrated error correction improving as the logical qubit grew, a key milestone toward practical fault tolerance. Several companies have published roadmaps targeting fault-tolerant machines around the end of this decade.

Resource estimates are falling. In 2019, the best estimate for factoring 2048-bit RSA was about 20 million noisy qubits running for eight hours. In 2025, a revised analysis by Craig Gidney at Google estimated that fewer than one million noisy qubits could do it in under a week. Algorithmic improvements, not only hardware, are shrinking the gap.

Expert surveys of quantum researchers, such as those published annually by the Global Risk Institute, put a meaningful probability, often estimated at one in three or higher, on a CRQC within about 10 to 15 years, with wide disagreement.

The honest summary: a CRQC is not imminent, its arrival date is uncertain, and that uncertainty itself is the risk, because migration takes years.

25.5 Mosca's inequality

Michele Mosca proposed a simple way to decide whether you are already late. Define:

- **X:** how many years your data must stay confidential (its shelf life).
- **Y:** how many years it will take to migrate your systems to quantum-safe cryptography.
- **Z:** how many years until a CRQC exists.

Mosca's inequality. If $X + Y$ is greater than Z , you have a problem: some of your data will still need protection after attackers can break it.

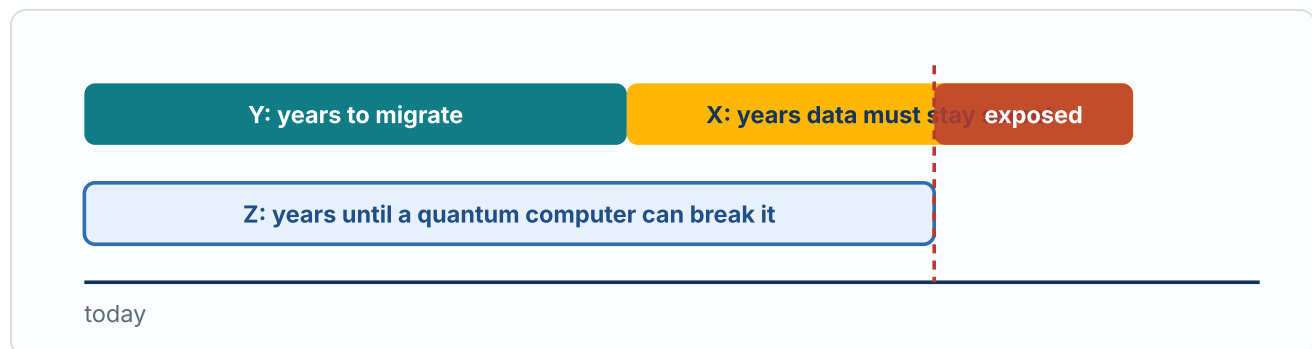


Figure 25.1 Mosca's inequality. When migration time plus data shelf life extends past the arrival of a quantum computer, the overlap is data that will be readable by attackers.

For example, if medical records must stay confidential for 25 years (X), and migration will take 7 years (Y), then even a CRQC 30 years away (Z) leaves a gap. Large organisations typically estimate Y at 5 to 15 years, because cryptography is embedded in so many products, protocols and partners.

25.6 Harvest now, decrypt later

The most urgent threat is not future attacks on future traffic. It is **harvest now, decrypt later (HNDL)**: adversaries recording encrypted traffic and stealing encrypted data **today**, to decrypt once a CRQC exists. Intelligence agencies are widely assumed to do this, and data storage is cheap.

HNDL affects **confidentiality**, through key exchange and public-key encryption. It is why the first post-quantum upgrades, deployed by browsers, messaging apps and SSH since 2023 and 2024, all protect key exchange.

Signatures face a different timeline. A signature only needs to be unforgeable until it is verified, so a CRQC in 2035 cannot retroactively forge a TLS handshake from 2026. But long-lived signature keys must be replaced before a CRQC exists: root CA keys, firmware and code signing keys burned into devices, document signatures that must remain valid for decades, and identity signing keys. If a CRQC could derive a root CA's private key from its public key, it could issue certificates for anything.

Risk	Driven by	Urgency
Recorded traffic decrypted later	Key exchange (ECDH, RSA key transport)	Highest: affects data sent today
Stored data wrapped with RSA or ECC keys decrypted	Public-key encryption and key wrapping	High for long-lived data
Forged signatures on new artefacts	Signature keys still trusted when a CRQC arrives	Must complete before CRQC; long lead times for roots and devices
Forged signatures on old artefacts	Long-term signatures without post-quantum timestamps	Specific to archives and legal documents

25.7 Government timelines

Governments have converted uncertainty into deadlines. The main ones, as published by late 2026:

Authority	Key dates
US NIST (IR 8547, draft, 2024)	Deprecate 112-bit RSA and ECC after 2030; disallow RSA, ECDSA, EdDSA, DH and ECDH entirely after 2035

Authority	Key dates
US NSA (CNSA 2.0)	Prefer quantum-safe algorithms for software and firmware signing, browsers, servers and cloud services from 2025; new national security system acquisitions must be CNSA 2.0 compliant from January 2027; exclusive use in many categories by 2030 to 2033; all national security systems quantum-resistant by 2035
US federal policy	National Security Memorandum 10 (2022) and the Quantum Computing Cybersecurity Preparedness Act require agencies to inventory cryptography and plan migration, with 2035 as the overall goal
UK NCSC (2025)	By 2028: discovery and migration plan complete; by 2031: high-priority migrations complete; by 2035: migration complete
European Union (coordinated roadmap, 2025)	Begin transition by the end of 2026; protect high-risk use cases by the end of 2030; complete as far as feasible by 2035
Others	Australia's guidance targets ceasing traditional asymmetric cryptography by 2030; Canada, Germany, France, the Netherlands, Japan, Singapore and others have published plans with similar horizons

CURRENT DEVELOPMENT These dates are converging on a common pattern: **inventory now, protect high-risk systems by about 2030, finish by 2035**. Regulators in finance and critical infrastructure increasingly ask organisations to show a post-quantum plan. Check your sector's guidance, and treat 2030 as the practical deadline for anything important.

25.8 What not to do

- **Do not wait for certainty** about the CRQC date. Migration time is the long pole.
- **Do not deploy unvetted "quantum-proof" products.** Use the NIST-standardised algorithms in Chapter 26.
- **Do not rely on quantum key distribution (QKD)** as a general solution. QKD uses quantum physics to distribute keys over special optical links, but it needs dedicated hardware, works only point to point over limited distances, does not authenticate on its own, and is not recommended by the NSA, NCSC, ANSSI or BSI as a replacement for post-quantum cryptography in most uses.
- **Do not panic about symmetric cryptography.** Moving to AES-256 where feasible is enough.

HANDS-ON LAB

Lab: estimate your own exposure

Pick one system you know. Fill in this table and apply Mosca's inequality.

Question	Your answer
What data does it protect, and how many years must it stay confidential (X)?	
Which public-key algorithms protect it in transit and at rest?	
Could an adversary be recording its traffic today?	
How long would it take to change its cryptography, including vendors and partners (Y)?	
Which signature keys does it trust, and how long are they trusted for?	
Using $Z = 2035$ as a planning assumption, is $X + Y$ greater than Z minus today?	

Then check your browser: visit a few sites and look at the key exchange in the developer tools' Security panel. Note which use X25519MLKEM768. That is post-quantum protection already working.

CASE STUDY

Failure story: the long tail of a cryptographic transition

The retirement of SHA-1 is the best rehearsal we have for the post-quantum transition. Theoretical weaknesses were published in 2005. NIST deprecated SHA-1 for signatures in 2011. Browsers began warning about SHA-1 certificates in 2014 and stopped accepting them in early 2017, the same year the first real collision was produced.

Even with 12 years of warning, the transition was painful. Payment terminals, medical devices, industrial systems and older phones could not verify SHA-256 certificates, so some CAs requested exemptions to keep issuing SHA-1 certificates for payment processors. Software signing and Git kept SHA-1 for years longer. Some legacy systems still use it today.

The post-quantum transition is far larger: it replaces public-key algorithms, not one hash function, across protocols, certificates, hardware, firmware and partners, with larger keys and signatures that break assumptions everywhere. If SHA-1 took about 15 years, starting the post-quantum migration in the late 2020s with a 2035 deadline leaves little slack.

KEY TAKEAWAYS

Chapter summary

- Shor's algorithm on a cryptographically relevant quantum computer breaks RSA, DH and all elliptic curve cryptography; larger keys do not help.
- Grover's algorithm only modestly weakens symmetric cryptography; prefer AES-256 for long-lived data.
- No CRQC exists, but hardware is improving and resource estimates have fallen sharply; uncertainty is the risk.
- Mosca's inequality: if data shelf life plus migration time exceeds time to a CRQC, act now. Harvest-now-decrypt-later makes key exchange the first priority; long-lived signature keys come next.
- Governments converge on inventory now, high-risk systems by about 2030, complete by 2035.

CHECK YOURSELF

Review questions

1. Why does increasing RSA key size not protect against Shor's algorithm?
2. Why is Grover's algorithm a much smaller threat in practice than on paper?
3. State Mosca's inequality and apply it to data that must stay secret for 20 years.
4. Why is key exchange more urgent than signatures, and which signature keys are urgent anyway?
5. Summarise the common pattern in government post-quantum deadlines.
6. Why is quantum key distribution not a general replacement for post-quantum cryptography?

The New Algorithms: ML-KEM, ML-DSA, SLH-DSA and Beyond

In August 2024, after an eight-year public competition, NIST published the first post-quantum cryptography standards. This chapter explains each algorithm in plain terms: how it works, how big its keys and signatures are, where it fits, and what can go wrong. It also covers the standards still on the way, the stateful hash-based signatures already in use for firmware, and how hybrid designs combine old and new.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Explain how lattice-based KEMs and signatures work at an intuitive level
- Compare ML-KEM, ML-DSA, SLH-DSA, FN-DSA, HQC, LMS and XMSS on size, speed and assurance
- Choose parameter sets and algorithms for common use cases
- Explain hybrid key exchange, composite signatures and implementation risks

IN THIS CHAPTER

- | | |
|------|-------------------------------|
| 26.1 | How the standards were chosen |
| 26.2 | Lattices in one page |
| 26.3 | The algorithms, compared |
| 26.4 | Choosing an algorithm |
| 26.5 | Hybrid and composite designs |
| 26.6 | Implementation risks |

26.1 How the standards were chosen

In 2016, NIST launched an open competition for post-quantum public-key algorithms, modelled on the AES and SHA-3 competitions. Researchers worldwide submitted 82 proposals. Over three rounds and six years, the community attacked them publicly; several were broken outright, including one finalist, SIKE, which fell in 2022 to an attack that ran in about an hour on a single laptop core. That break is a reminder of why open competitions and conservative choices matter.

In August 2024, NIST published three standards:

Standard	Algorithm (original name)	Type	Mathematical basis
FIPS 203	ML-KEM (CRYSTALS-Kyber)	Key encapsulation	Module lattices (Module-LWE)
FIPS 204	ML-DSA (CRYSTALS-Dilithium)	Signature	Module lattices
FIPS 205	SLH-DSA (SPHINCS+)	Signature	Hash functions only

Two more are in progress:

- **FN-DSA** (Falcon), a compact lattice signature, as FIPS 206. The draft was submitted for approval in 2025; final publication is expected around the end of 2026 or in 2027.
- **HQC**, a code-based KEM selected in March 2025 as a backup to ML-KEM with different mathematics; a draft standard is expected first, with a final version around 2027.

NIST also runs an "on-ramp" competition for additional signature schemes with different trade-offs, so more options will follow later in the decade.

26.2 Lattices in one page

Recall the idea from Chapter 2: **Learning With Errors (LWE)**. Take a secret vector s . Publish many random linear equations in s , each with a small random error added. Without the errors, ordinary algebra recovers s instantly. With the errors, recovering s means solving a hard lattice problem, believed hard for both classical and quantum computers.

Module-LWE, used by ML-KEM and ML-DSA, organises the numbers into small polynomials to make keys compact and arithmetic fast, using a technique called the **Number Theoretic Transform (NTT)**, a cousin of the fast Fourier transform.

How ML-KEM works, intuitively

- 1. Key generation.** Bob's public key is a matrix A and a vector $t = A \cdot s + e$, where s is his secret and e a small error. Recovering s from t is the LWE problem.
- 2. Encapsulation.** Alice picks her own small random values, computes a ciphertext from Bob's public key that hides a random message, and derives a shared key from that message.
- 3. Decapsulation.** Bob uses s to remove most of the noise and recover the message, then derives the same shared key.

The small errors mean Bob's recovery is approximate, with a tiny chance of failure (below 2^{-138} for ML-KEM-768). ML-KEM wraps this in the **Fujisaki-Okamoto transform**: Bob re-encrypts the recovered message and checks it matches, rejecting any manipulated ciphertext with a pseudo-random key. That is what makes ML-KEM IND-CCA2 secure (Chapter 10), and safe to use with long-lived keys.

A KEM is not Diffie-Hellman

ML-KEM is a **key encapsulation mechanism**, not a Diffie-Hellman replacement in every respect:

- DH is symmetric: both sides contribute and compute. A KEM is asymmetric: one side encapsulates to the other's public key.
- Protocols built on DH's special properties (for example, combining static and ephemeral keys in Signal's X3DH) needed redesign, which is why PQXDH and similar protocols took effort.
- For ephemeral key exchange, as in TLS, the KEM interface fits naturally: the client sends a public key, the server returns a ciphertext.

26.3 The algorithms, compared

ML-KEM (FIPS 203)

Parameter set	Security category	Public key	Ciphertext	Shared secret
ML-KEM-512	1 (about AES-128)	800 bytes	768 bytes	32 bytes
ML-KEM-768	3 (about AES-192)	1,184 bytes	1,088 bytes	32 bytes
ML-KEM-1024	5 (about AES-256)	1,568 bytes	1,568 bytes	32 bytes

For comparison, an X25519 public key is 32 bytes. ML-KEM is very fast, often faster than X25519, but much larger. **ML-KEM-768** is the default choice; it is what X25519MLKEM768 uses in TLS. CNSA 2.0 requires ML-KEM-1024 for US national security systems.

■ ML-DSA (FIPS 204)

Parameter set	Security category	Public key	Signature
ML-DSA-44	2	1,312 bytes	2,420 bytes
ML-DSA-65	3	1,952 bytes	3,309 bytes
ML-DSA-87	5	2,592 bytes	4,627 bytes

ML-DSA is the general-purpose post-quantum signature: fast to sign and verify, with no secret state and "hedged" randomness by default. Its cost is size: an ECDSA P-256 signature is 64 bytes and its public key 64 bytes. A TLS certificate chain with ML-DSA keys and signatures grows by roughly 10 to 15 kilobytes, which matters for handshake performance and constrained devices.

■ SLH-DSA (FIPS 205)

SLH-DSA is built **only from hash functions**, so its security rests on the most conservative assumption in cryptography: that SHA-2 or SHAKE behave as expected. No lattice mathematics is involved. The price is large, slow signatures:

Parameter set	Public key	Signature	Signing speed
SLH-DSA-128s ("small")	32 bytes	7,856 bytes	Slow
SLH-DSA-128f ("fast")	32 bytes	17,088 bytes	Faster, larger
SLH-DSA-256s	64 bytes	29,792 bytes	Slow

SLH-DSA suits uses where signatures are created rarely and must be trusted for a very long time, and where a second mathematical basis is valuable: root CA certificates, firmware roots of trust, and long-term archives.

■ FN-DSA (draft FIPS 206)

FN-DSA (Falcon) offers the smallest lattice signatures: about 666 bytes with an 897-byte public key at the lower security level. It is attractive where size matters, such as certificate chains. Its signing uses floating-point arithmetic and is notoriously hard to

implement in constant time, so expect it mainly for verification-heavy uses and in well-reviewed libraries.

■ HQC (selected 2025)

HQC is a KEM based on error-correcting codes rather than lattices. It is larger and slower than ML-KEM, but provides a fallback with independent mathematics in case a breakthrough against lattices emerges.

■ Stateful hash-based signatures: LMS and XMSS

LMS (RFC 8554) and **XMSS** (RFC 8391), approved by NIST in SP 800-208 in 2020, are hash-based signatures that have been available longer than the FIPS 203 to 205 standards. They are compact and fast to verify, but **stateful**: each one-time key inside them must be used only once, and the signer must reliably track which have been used. Reusing state, for example by restoring a backup or cloning a VM, can destroy security.

That makes them unsuitable for general use but well suited to **firmware and software signing**, where signatures are created rarely, in controlled HSM-based environments. CNSA 2.0 requires LMS or XMSS for firmware signing, and many device makers are already using them.

26.4 Choosing an algorithm

Use case	Recommended choice
Key exchange in TLS, SSH, VPNs	Hybrid X25519 (or P-256) with ML-KEM-768; ML-KEM-1024 for CNSA 2.0 systems
Encrypting data to a public key (HPKE, key wrapping)	ML-KEM-768 or 1024, in hybrid during transition
General signatures: TLS certificates, tokens, documents	ML-DSA-65 (ML-DSA-87 for CNSA 2.0); FN-DSA where size is critical, once standardised
Root CAs and long-lived trust anchors	ML-DSA-87, or SLH-DSA for maximum conservatism
Firmware and boot signing	LMS or XMSS in an HSM; SLH-DSA or ML-DSA as alternatives
Symmetric encryption and hashing	Keep AES-256, SHA-384 or SHA-3; no change of algorithm needed

26.5 Hybrid and composite designs

During the transition, most deployments combine a classical algorithm with a post-quantum one, so that security holds if **either** survives. This protects against a quantum computer (if the classical part breaks) and against an undiscovered flaw in the new algorithms or their implementations (if the post-quantum part breaks).

- **Hybrid key exchange** feeds both shared secrets into the key derivation function. X25519MLKEM768 in TLS 1.3, mlkem768x25519-sha256 in SSH, and PQXDH in Signal all work this way.
- **Composite signatures** pair a classical and a post-quantum signature in a single certificate or signature object, valid only if both verify. The IETF LAMPS working group has specified composite ML-DSA formats for X.509, alongside "pure" ML-DSA certificates.
- **Dual certificates or chains** are an alternative: issue a classical and a post-quantum certificate, and let each relying party use what it supports.

Some agencies, notably the NSA for national security systems, prefer moving directly to pure post-quantum algorithms rather than hybrids for signatures; others, including Germany's BSI and France's ANSSI, recommend hybrids during the transition. Follow your sector's guidance.

26.6 Implementation risks

New algorithms bring new implementation pitfalls:

- **Side channels.** In early 2024, researchers found **KyberSlash**: several implementations of Kyber used a division step whose timing depended on secret data, allowing key recovery. Mature libraries fixed it; immature ones may not have.
- **Decapsulation failures and validation.** ML-KEM implementations must perform the re-encryption check exactly as specified, and validate input sizes.
- **Randomness.** As always, key generation needs a strong random source (Chapter 3).
- **State for LMS and XMSS.** Lose or duplicate state and security is gone.
- **Size assumptions.** Protocol fields, database columns, HSM buffers, certificate size limits, and network packets sized for 64-byte signatures will break.

Use well-maintained libraries that implement the final FIPS versions: recent releases of OpenSSL (3.5 and later include ML-KEM, ML-DSA and SLH-DSA), BoringSSL and AWS-LC, the Go standard library, Bouncy Castle for Java, and the Open Quantum Safe project's liboqs for experimentation. Prefer FIPS-validated modules where compliance requires it, as vendors add post-quantum algorithms to their validated configurations.

HANDS-ON LAB

Lab: post-quantum cryptography with OpenSSL

You need OpenSSL 3.5 or later (`openssl version`).

1. List the algorithms.

```
openssl list -kem-algorithms | grep -i ml-kem
openssl list -signature-algorithms | grep -iE "ml-dsa|slh-dsa"
```

2. Generate and compare key sizes.

```
openssl genpkey -algorithm ML-DSA-65 -out mldsa.key
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out ec.key
openssl pkey -in mldsa.key -pubout -outform DER | wc -c
openssl pkey -in ec.key -pubout -outform DER | wc -c
```

3. Sign and verify with ML-DSA.

```
echo "post-quantum hello" > msg.txt
openssl pkeyutl -sign -inkey mldsa.key -in msg.txt -rawin -out msg.sig
wc -c msg.sig # about 3,309 bytes
openssl pkeyutl -verify -inkey mldsa.key -in msg.txt -rawin -sigfile msg.sig
```

4. Encapsulate and decapsulate with ML-KEM.

```
openssl genpkey -algorithm ML-KEM-768 -out kem.key
openssl pkey -in kem.key -pubout -out kem.pub
openssl pkeyutl -encap -inkey kem.pub -pubin -out ct.bin -secret ss_sender.bin
openssl pkeyutl -decap -inkey kem.key -in ct.bin -secret ss_receiver.bin
cmp ss_sender.bin ss_receiver.bin && echo "shared secrets match"
```

5. Make a post-quantum certificate. Create a self-signed ML-DSA-65 certificate with `openssl req -x509 -newkey ML-DSA-65 ...` and inspect it. Compare its size with an ECDSA certificate.

CASE STUDY

Failure story: SIKE, broken in an hour

SIKE (Supersingular Isogeny Key Encapsulation) was one of the most elegant candidates in NIST's competition. It had the smallest keys of any post-quantum KEM, it had survived years of analysis, and it was selected in 2022 as a fourth-round candidate for possible standardisation. Major technology companies had experimented with it in TLS.

In July 2022, Wouter Castryck and Thomas Decru published an attack, building on a 25-year-old mathematical result, that recovered SIKE's secret key in about an hour on a single classical computer core for the lowest security level, and in about a day for the highest. The scheme was withdrawn.

Nobody was harmed, because SIKE had only been used experimentally and always in hybrid with classical key exchange. The episode is the strongest argument for three practices in this chapter: prefer algorithms with long and broad analysis (ML-KEM, ML-DSA, SLH-DSA), use hybrid designs during the transition, and keep systems agile enough to swap algorithms when the unexpected happens.

KEY TAKEAWAYS

Chapter summary

- NIST standardised ML-KEM (FIPS 203), ML-DSA (FIPS 204) and SLH-DSA (FIPS 205) in 2024; FN-DSA and HQC are coming.
- ML-KEM and ML-DSA rest on Module-LWE lattices: fast, but with keys and signatures tens of times larger than today's.
- SLH-DSA relies only on hashes: very conservative, with large signatures; LMS and XMSS are compact stateful hash signatures suited to firmware.
- Use ML-KEM-768 in hybrid for key exchange, ML-DSA-65 for general signatures, and stronger sets or SLH-DSA for long-lived roots.
- Hybrid and composite designs hedge against both quantum attacks and new-algorithm flaws; implementation risks include side channels, state handling and size assumptions.

CHECK YOURSELF

Review questions

1. Describe Learning With Errors in two sentences.
2. Why is a KEM not a drop-in replacement for Diffie-Hellman in every protocol?
3. Compare ML-DSA-65 and ECDSA P-256 public key and signature sizes, and name one system that might break.
4. When would you choose SLH-DSA over ML-DSA, and why?
5. What makes LMS and XMSS stateful, and why does that suit firmware signing?
6. What is the argument for hybrid designs, and what did SIKE's break demonstrate?

Migrating to Post-Quantum Cryptography

Knowing the new algorithms is the easy part. Replacing the public-key cryptography inside thousands of applications, devices, certificates, protocols and vendor products is a multi-year programme. This chapter gives you a practical migration method: build a cryptographic inventory, prioritise by risk, protect key exchange first, plan the PKI transition, work with vendors, and measure progress against the 2030 and 2035 milestones.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Build a cryptographic inventory and CBOM and use it to prioritise
- Sequence a migration: key exchange first, then signatures, then trust anchors
- Plan post-quantum certificates and PKI changes and their operational impact
- Run a migration programme with vendors, testing and metrics

IN THIS CHAPTER

- | | |
|------|--|
| 27.1 | The shape of the problem |
| 27.2 | A five-phase method |
| 27.3 | Discovery: the cryptographic inventory |
| 27.4 | Prioritisation |
| 27.5 | Migrating key exchange first |
| 27.6 | Migrating signatures and PKI |
| 27.7 | HSMs, KMS and vendors |
| 27.8 | Testing and performance |
| 27.9 | Governance and metrics |

27.1 The shape of the problem

Post-quantum migration is unlike most security projects:

- **It touches everything.** Public-key cryptography is in web servers, load balancers, VPNs, SSH, databases, identity providers, code signing, HSMs, smart cards, IoT devices, payment terminals, mobile apps and partner connections.
- **Much of it is not yours.** Cryptography is embedded in libraries, operating systems, appliances and SaaS products. You migrate by upgrading and by pressing vendors.
- **The new keys are bigger.** Systems sized for 32-byte keys and 64-byte signatures will meet kilobytes.
- **It has a deadline you cannot see.** Planning dates (2030 for high-priority systems, 2035 for completion) stand in for an unknown CRQC arrival.

The good news: a well-run migration also builds crypto agility, inventory and automation that pay off for every future change.

27.2 A five-phase method

Phase	Goal	Main outputs
1. Prepare	Ownership, scope and budget	Executive sponsor, programme lead, policy update, initial timeline
2. Discover	Know where cryptography is used	Cryptographic inventory and CBOMs
3. Prioritise	Decide what to migrate first	Risk-ranked list using data lifetime, exposure and migration effort
4. Migrate	Change algorithms, keys and certificates	Upgraded systems, hybrid deployments, vendor commitments
5. Verify and sustain	Prove it worked and stays that way	Testing, metrics, continuous discovery, crypto agility built in

The phases overlap. You should be migrating easy, high-value systems such as public TLS endpoints while discovery of the long tail continues.

27.3 Discovery: the cryptographic inventory

A **cryptographic inventory** records where and how cryptography is used. It extends the key and certificate inventories of Chapters 19 and 20 to algorithms and protocols:

Field	Example
System and owner	Customer portal, team "web-platform"
Use	TLS server, mTLS client, data-at-rest wrapping, JWT signing, code signing
Algorithms and parameters	ECDHE X25519, ECDSA P-256 certificate, AES-256-GCM
Implementation	OpenSSL 3.0.13, vendor appliance firmware 17.1, cloud KMS
Keys and certificates involved	Certificate serial, KMS key ID, HSM partition
Data protected and its shelf life	Personal data, 7 years
Exposure	Internet-facing; internal; partner link
Upgrade path	Library upgrade; vendor roadmap; hardware replacement

Sources of discovery

- **Network scanning** of TLS, SSH and VPN endpoints, recording negotiated groups and certificate algorithms.
- **Certificate and key inventories** from CLM tools, KMS and HSMs.
- **Code and dependency scanning** for cryptographic library calls and algorithm names.
- **Binary and container image scanning** for embedded libraries and versions.
- **Configuration analysis** of load balancers, web servers, VPNs and identity providers.
- **Vendor questionnaires** for appliances and SaaS.

CBOM

A **cryptographic bill of materials (CBOM)** describes the cryptographic assets in one application or product, in a machine-readable form. The CycloneDX standard (version 1.6 onward) includes a CBOM format covering algorithms, keys, certificates, protocols and their relationships. Requiring CBOMs from internal teams and vendors turns discovery from a one-off project into a continuous process.

IN PRACTICE Do not try to inventory everything perfectly before acting. Start with the systems you know matter most: internet-facing TLS, VPNs, identity providers, PKI, code signing and data stores holding long-lived sensitive data. Discovery of the long tail can run in parallel.

27.4 Prioritisation

Rank each system with four questions:

- 1. Shelf life:** how long must the data it protects stay confidential? (Mosca's X)
- 2. Exposure to harvesting:** does its traffic cross networks an adversary could record, such as the internet or partner links?
- 3. Trust lifetime:** does it hold signature keys or trust anchors that must remain trustworthy past 2030? (root CAs, firmware keys, devices in the field for 15 years)
- 4. Effort and lead time:** library upgrade, vendor dependency, or hardware replacement?

A simple scoring matrix places systems into waves:

Wave	Typical systems	Why first
1 (now)	Internet-facing TLS, VPNs, SSH, partner links carrying sensitive data	Harvest-now-decrypt-later; often just a configuration or library upgrade
2	Long-lived trust anchors, firmware signing, devices with long field lives, data-at-rest wrapped with RSA/ECC	Long lead times; must finish before a CRQC
3	Internal service-to-service TLS, mTLS, internal PKI, tokens and identity signing	Lower harvesting exposure, large volume
4	Everything else, legacy systems scheduled for retirement	Migrate or retire

27.5 Migrating key exchange first

Key exchange is the most urgent and often the easiest:

- **TLS.** Upgrade servers, load balancers, CDNs and proxies to versions that support **X25519MLKEM768**, and enable it. Browsers already offer it. Check every termination point, then internal service-to-service TLS, client libraries and language runtimes.

- **SSH.** Upgrade to OpenSSH 10 or equivalent, where mlkem768x25519-sha256 is the default.
- **VPNs.** Plan IKEv2 upgrades with post-quantum KEMs (RFC 9370) as vendors deliver them; for WireGuard, consider the pre-shared key option as an interim measure.
- **Messaging and application protocols.** Use HPKE with post-quantum KEMs where data is encrypted to public keys.

Test for **ossification**: the larger ClientHello in hybrid TLS has broken some middleboxes, firewalls and old servers that mishandle multi-packet handshakes. Roll out gradually and monitor connection failures.

27.6 Migrating signatures and PKI

Signatures are harder, because certificates and trust stores connect organisations and devices that upgrade on different schedules.

Choices for certificates

Approach	How it works	Trade-offs
Pure post-quantum chains	ML-DSA root, intermediates and leaves	Clean; only works where every relying party supports ML-DSA
Composite certificates	Each certificate carries combined classical and ML-DSA keys and signatures	Secure if either holds; larger; requires composite-aware software
Parallel hierarchies	Separate classical and post-quantum chains for the same services	Flexible; doubles issuance and management

For **private PKI**, which you control end to end, you can start now: stand up an ML-DSA or composite test hierarchy, issue certificates to internal services that support it, and measure handshake sizes and performance. Generate new long-lived roots with post-quantum algorithms, or at least plan a post-quantum root well before 2030.

For the **public web PKI**, the path depends on browsers and the CA/Browser Forum. Post-quantum certificates make TLS handshakes significantly larger, and browsers have been exploring designs to reduce that cost, such as trust anchor negotiation and **Merkle Tree Certificates**, which replace most per-handshake signatures with compact inclusion proofs. Expect public post-quantum certificates to arrive later than post-quantum key exchange; follow your CAs' and browsers' announcements.

Code, firmware and documents

- **Firmware and boot chains:** devices shipping now with 10 to 20 year lives must be able to verify post-quantum signatures, using LMS or XMSS (stateful, in HSMs) or ML-DSA. This is one of the most urgent signature migrations because hardware cannot be updated later.
- **Software signing:** plan dual signatures (classical plus ML-DSA) on packages and updates during the transition.
- **Long-term documents and archives:** add post-quantum timestamps over existing signatures, so evidence remains verifiable.

Identity and tokens

Identity providers and token systems must move to ML-DSA (or composite) signatures, publish post-quantum keys in JWKS, and ensure every verifier can handle larger keys and tokens. JOSE and COSE specifications for ML-DSA are being finalised at the IETF.

27.7 HSMs, KMS and vendors

- **HSMs and KMS:** confirm that your HSMs and key management services support ML-KEM, ML-DSA, SLH-DSA and LMS, ideally within FIPS 140-3 validated configurations. Firmware upgrades may be needed; some older hardware will need replacement. Large keys and signatures may stress HSM throughput and buffer limits.
- **Vendors:** ask every supplier of security-relevant products for a written post-quantum roadmap: which algorithms, in which versions, by when, with FIPS validation dates. Add post-quantum and crypto agility requirements to procurement and contract renewals now; contracts signed in 2026 will govern products still running in 2035.

27.8 Testing and performance

Plan for these effects:

Effect	Where it bites	Mitigation
Larger handshakes	TLS over slow or lossy links; IoT; first-byte latency	Measure; prefer ML-KEM-768; keep chains short
Larger certificates and signatures	Certificate stores, smart cards, headers, tokens, logs	Review size limits and fields

Effect	Where it bites	Mitigation
Fixed-size assumptions	Database columns, protocol fields, HSM buffers, QR codes	Code and schema review
Middlebox ossification	Firewalls, IDS, proxies	Staged rollout with failure monitoring
CPU cost	Signature-heavy services	ML-KEM and ML-DSA are fast; SLH-DSA signing is slow

Run post-quantum configurations in test environments early, add them to continuous integration, and use canary rollouts in production.

27.9 Governance and metrics

Treat the migration as a programme with leadership visibility:

- **A named owner** and a cross-functional working group (security architecture, PKI, platform, application teams, procurement, risk).
- **Policy updates** that add post-quantum algorithms to the approved list, set dates after which new systems must use them, and set deprecation dates for RSA and ECC aligned with your regulators.
- **Metrics**, reported quarterly:

Metric	Example target
Inventory coverage	100 percent of high-priority systems inventoried by end of first year
Hybrid key exchange coverage for internet-facing TLS	Over 90 percent within 18 months
Systems with a documented migration path	100 percent of wave 1 and 2
Vendors with a written post-quantum roadmap	All critical vendors
Long-lived trust anchors with a post-quantum plan	All, before 2028
New systems approved with classical-only public-key crypto	Zero after the policy date

HANDS-ON LAB

Lab: run a mini migration

- 1. Discover.** Scan five of your own (or permitted) TLS endpoints with `testssl.sh` or a short `openssl s_client` loop and record the negotiated key exchange group and certificate signature algorithm for each.
- 2. Upgrade.** In a test environment, upgrade nginx or another server to a build on OpenSSL 3.5 or later, enable `X25519MLKEM768`, and confirm with a current browser and with `openssl s_client -groups X25519MLKEM768`.
- 3. Measure.** Capture handshakes before and after in Wireshark and compare the ClientHello and ServerHello sizes.
- 4. Prototype PKI.** Build a small ML-DSA-65 hierarchy with OpenSSL (as in Chapter 18, replacing EC keys with ML-DSA) and serve a post-quantum certificate to a test client that supports it. Record chain size.
- 5. Write the CBOM** for one small application: algorithms, libraries, keys, certificates and protocols, in CycloneDX format if you can.

CASE STUDY

Failure story: the migration that could not reach the field

Payment terminals, smart meters, cars, medical devices and industrial controllers are designed to run for 10 to 25 years. Many verify firmware updates against a public key written into hardware at manufacture. When the SHA-1 and RSA-1024 transitions arrived, manufacturers repeatedly found devices in the field that could not verify anything else, and the choices were costly: keep trusting a weakened algorithm, or physically replace devices.

The post-quantum transition makes this pattern acute. A device shipped today with only an ECDSA or RSA firmware verification key cannot be fixed with software if that key becomes forgeable: anyone able to forge the signature could push malicious firmware. This is why CNSA 2.0 sets early dates for firmware signing, why device makers are adopting LMS and XMSS now, and why procurement teams should be asking, for every long-lived device bought from now on, "can this verify post-quantum signatures, and can its trust anchor be rotated?"

KEY TAKEAWAYS

Chapter summary

- Post-quantum migration touches nearly every system, much of it vendor-owned, with larger keys and an uncertain deadline; run it as a multi-year programme.
- Discover with a cryptographic inventory and CBOMs; prioritise by data shelf life, harvesting exposure, trust lifetime and effort.
- Migrate key exchange first (hybrid TLS, SSH, VPNs), then long-lived trust anchors and firmware signing, then internal PKI, tokens and the long tail.
- Plan PKI changes: pure, composite or parallel hierarchies; public post-quantum certificates will lag key exchange.
- Engage vendors and HSM providers now, test for size and ossification effects, and track progress with clear metrics.

CHECK YOURSELF

Review questions

1. Why is post-quantum migration harder than most security upgrades?
2. What does a CBOM contain, and why does it matter for vendor products?
3. Rank four types of systems into migration waves and justify the order.
4. Compare composite certificates with parallel hierarchies.
5. Why are long-lived devices with hard-coded firmware keys an urgent priority?
6. Name three effects of larger keys and signatures and how to test for each.

PART VIII

Frontiers and Reference

Where cryptography is heading next, plus a glossary and a guide to further study.

In this part

28 Frontiers: Zero Knowledge, MPC, FHE and What Comes Next

A Glossary

B Standards, Further Reading and Practice

C Command Reference

Frontiers: Zero Knowledge, MPC, FHE and What Comes Next

Beyond encryption and signatures lies a family of techniques that once seemed like magic: proving a statement without revealing why it is true, computing on data nobody can see, and signing with a key that never exists in one place. These techniques are moving from research into products. This chapter explains what each one does, where it is used today, and how to judge when it is ready for your problem.

AFTER THIS CHAPTER YOU WILL BE ABLE TO

- Explain zero-knowledge proofs and their uses in identity and verification
- Explain multi-party computation and threshold signatures and where they protect keys
- Explain homomorphic encryption and its current practical limits
- Assess emerging cryptography for readiness, cost and risk

IN THIS CHAPTER

- | | |
|------|--|
| 28.1 | Zero-knowledge proofs |
| 28.2 | Multi-party computation and threshold cryptography |
| 28.3 | Homomorphic encryption |
| 28.4 | Other techniques worth knowing |
| 28.5 | Judging readiness |
| 28.6 | Where to go from here |

28.1 Zero-knowledge proofs

A **zero-knowledge proof (ZKP)** lets a prover convince a verifier that a statement is true without revealing anything beyond the fact that it is true. The classic illustration: proving you know the password to a door in a circular cave by coming out of whichever side the verifier calls, many times in a row, without ever saying the password.

Modern ZKPs prove statements about computations: "I know a secret that hashes to this value", "this transaction balances without revealing amounts", "I am over 18 according to a government-signed credential, without revealing my birth date".

Two families dominate:

Family	Characteristics	Notes
zk-SNARKs	Very small proofs, fast verification; some need a trusted setup	Widely used in blockchain systems; many rely on elliptic curve pairings, which are not quantum-safe
zk-STARKs and hash-based proofs	Larger proofs, no trusted setup	Rely on hash functions; considered quantum-resistant

Where ZKPs are appearing:

- **Privacy-preserving identity.** Digital identity wallets can prove attributes (age, residency, licence) without revealing the whole credential. The EU's digital identity wallet framework under eIDAS 2.0 and several national schemes are adopting selective disclosure, and some are adding zero-knowledge techniques, such as Google's open-sourced ZK age verification in 2025.
- **Blockchain scaling and privacy,** the largest current deployment of ZK technology.
- **Verifiable computation:** proving a cloud service ran a computation correctly, including emerging work on proving properties of machine learning inference.

28.2 Multi-party computation and threshold cryptography

Secure multi-party computation (MPC) lets several parties jointly compute a function of their private inputs, learning only the output. Two banks can find customers they share without revealing their customer lists (**private set intersection**); several hospitals can compute statistics over combined patient data without pooling it.

The most commercially important application today is **threshold cryptography**: splitting a private key into **shares** held by different parties or machines, so that a signature or decryption requires a threshold of them (for example, 3 of 5) to cooperate, and **the full key never exists in any one place**, not even during signing.

- **Threshold signatures** such as FROST for Schnorr and Ed25519 signatures (standardised as RFC 9591 in 2024) and threshold ECDSA protocols protect digital asset custody, high-value signing keys and cloud key services.
- **Distributed key generation** creates the shares without anyone ever holding the whole key.

Compare this with an HSM's M-of-N quorum (Chapter 23): there, several people authorise use of a key that does exist inside one device. With threshold cryptography, the key itself is distributed. The two approaches are complementary, and some systems combine them.

NIST is running a project on multi-party threshold cryptography to develop guidance and possibly standards, including for post-quantum threshold schemes, which are an active research area.

28.3 Homomorphic encryption

Homomorphic encryption lets someone compute on encrypted data without decrypting it. A client encrypts data, a server computes on the ciphertext, and only the client can decrypt the result.

- **Partially homomorphic** schemes support one operation. RSA is multiplicatively homomorphic; Paillier encryption supports addition, which is enough for encrypted voting tallies and some statistics.
- **Fully homomorphic encryption (FHE)**, first shown possible by Craig Gentry in 2009, supports arbitrary computation. Modern schemes (BGV, BFV, CKKS for approximate arithmetic, TFHE for bits) are based on lattices, the same family as ML-KEM, and are believed quantum-resistant.

FHE remains expensive: typically thousands to millions of times slower than computing on plaintext, depending on the task, though performance has improved by orders of magnitude over the last decade and dedicated hardware accelerators are being built. It is practical today for specific workloads, such as private queries, simple analytics on encrypted data, and encrypted machine learning inference on small models. ISO and an industry consortium are standardising FHE schemes.

28.4 Other techniques worth knowing

Technique	What it does	Where you see it
Private information retrieval	Query a database without the server learning which record you wanted	Private contact discovery, some password breach checks
Oblivious pseudo-random functions and PAKE	Password-based protocols where the server never sees the password; OPAQUE, standardised by the IETF	Password-based login without exposing passwords
Privacy Pass and blind signatures	Anonymous tokens proving "this user is legitimate" without tracking	CAPTCHA alternatives, private access tokens
Differential privacy	Statistical noise that limits what aggregate results reveal about individuals	Census data, telemetry
Content provenance (C2PA)	Signed manifests describing how images, video and audio were created and edited	Camera makers, news organisations, AI-generated content labelling
Key transparency	Public, auditable logs of users' public keys	End-to-end encrypted messaging (Chapter 13)
Formal verification	Machine-checked proofs that protocols and code match their specifications	TLS 1.3 analysis, verified libraries such as HACL*, AWS-LC and others

Two trends cut across these. First, **AI** is both a consumer and a subject of cryptography: signed provenance for generated media, confidential computing and FHE for private inference, and verification of model integrity. Second, **formal verification** is moving from research to industry: several widely used cryptographic libraries now include machine-verified implementations of core algorithms, including post-quantum ones.

28.5 Judging readiness

When a vendor or colleague proposes advanced cryptography, ask:

- ☐ **Is the problem real?** Could a simpler design (access control, an HSM, separating duties) solve it?
- ☐ **Is the scheme standardised or widely analysed?** RFCs, NIST, ISO, or years of public cryptanalysis?
- ☐ **What are the assumptions?** Trusted setup? Non-colluding parties? Hardware trust? Quantum resistance?

- ☐ **What does it cost?** Performance, bandwidth, latency, operational complexity.
- ☐ **Are there mature, audited implementations?**
- ☐ **What is the failure mode?** If an assumption breaks, what leaks?

Advanced cryptography is powerful precisely because it changes who must be trusted. Make sure the new trust assumptions are better than the old ones, not just more complicated.

28.6 Where to go from here

You have now covered the whole path from modular arithmetic to post-quantum migration. To keep growing:

1. **Pick a specialisation** that matches your work: PKI and certificate operations, key management and HSMs, protocol security, post-quantum migration, implementation security, or one of the frontier areas in this chapter.
2. **Read primary sources.** RFCs, NIST publications, and CA/Browser Forum ballots are more readable than their reputation suggests. Appendix B lists the most important.
3. **Follow the field.** The IACR ePrint archive, the Real World Crypto conference talks, root program announcements, and NIST's post-quantum project pages report changes as they happen.
4. **Practise.** Cryptopals and CryptoHack challenges, labs like the ones in this book, and small tools you build yourself make the ideas permanent.
5. **Teach.** Explaining a concept to a colleague is the fastest way to discover what you have not yet understood.

KEY TAKEAWAYS

Chapter summary

- Zero-knowledge proofs prove statements without revealing why they are true; they power selective-disclosure identity and verifiable computation.
- MPC computes on private inputs; threshold cryptography distributes keys so they never exist in one place, now standardised for Schnorr-family signatures.
- Fully homomorphic encryption computes on encrypted data; it is lattice-based and improving fast but still costly.
- Private retrieval, OPAQUE, Privacy Pass, provenance, key transparency and formal verification are entering mainstream products.

- Judge emerging cryptography by the problem, analysis, assumptions, cost, implementations and failure modes.

CHECK YOURSELF

Review questions

1. Give a real use of zero-knowledge proofs in digital identity.
2. How does a threshold signature differ from an HSM quorum?
3. Why is fully homomorphic encryption believed to be quantum-resistant, and what limits its use today?
4. What problem does OPAQUE solve that ordinary password login does not?
5. What does C2PA content provenance sign, and why does it matter for AI-generated media?
6. Apply the readiness checklist to a proposal to use FHE for all analytics in your organisation.

APPENDIX

A



Glossary

Plain-language definitions of the terms used in this book, in alphabetical order. The chapter in brackets is where each term is explained in depth.

ACME.

Automatic Certificate Management Environment (RFC 8555), the protocol that lets software obtain and renew certificates without human involvement. (19)

AEAD.

Authenticated encryption with associated data: encryption that also detects any tampering, and can authenticate extra unencrypted data such as headers. AES-GCM and ChaCha20-Poly1305 are examples. (5)

AES.

Advanced Encryption Standard, the world's standard block cipher, with 128, 192 or 256-bit keys. (4)

Associated data.

Information authenticated, but not encrypted, by an AEAD, binding a ciphertext to its context. (5)

Asymmetric cryptography.

Cryptography using a key pair: a public key that can be shared and a private key kept secret. Also called public-key cryptography. (1)

Attestation.

Cryptographic evidence, usually signed by hardware, about the state or identity of a device or workload. (23)

Birthday bound.

The rule that repeats among random values become likely after about the square root of the number of possibilities. (3)

Block cipher.

An algorithm that encrypts fixed-size blocks of data under a key. (4)

BYOK and HYOK.

Bring your own key: importing your key material into a provider's key service. Hold your own key: keeping the key outside the provider, which must ask you for each use. (21)

CA (certificate authority).

An organisation or system that signs certificates. (15, 16)

CAA record.

A DNS record listing which CAs may issue certificates for a domain. (16)

CBOM.

Cryptographic bill of materials: a machine-readable list of the cryptographic algorithms, keys, certificates and protocols in a system. (27)

Certificate.

A signed document binding a public key to a name and permitted uses. (15)

Certificate Transparency (CT).

Public, append-only logs of all publicly trusted certificates, allowing anyone to detect mis-issuance. (16)

Chain (certificate chain).

The sequence of certificates linking a leaf certificate to a trusted root. (15)

Ciphertext.

Encrypted data. (1)

CRL.

Certificate revocation list: a signed list of revoked certificate serial numbers. (17)

CRQC.

Cryptographically relevant quantum computer: one large and reliable enough to break RSA and elliptic curve cryptography. (25)

Crypto agility.

The ability to change algorithms, keys, certificates or providers quickly without redesigning systems. (24)

Cryptoperiod.

The time a key is permitted to be used. (20)

CSPRNG.

Cryptographically secure pseudo-random number generator, whose output cannot be predicted without its secret state. (3)

CSR.

Certificate signing request: a file containing a public key and requested names, signed with the private key, sent to a CA. (15)

DEK and KEK.

Data encryption key, which encrypts data; key encryption key, which encrypts (wraps) other keys. (20)

Diffie-Hellman (DH, ECDH).

A method for two parties to agree on a shared secret over a public channel; ECDH is the elliptic curve form. (8)

Digital signature.

A value created with a private key that anyone with the public key can verify, proving origin and integrity. (9)

Discrete logarithm problem.

Given g and g^x in a group, find x . Believed hard classically; broken by quantum computers. (2)

ECDSA and EdDSA.

Elliptic curve signature algorithms; Ed25519 is the most common EdDSA instance. (9)

Elliptic curve.

A mathematical curve whose points form a group used for compact, efficient public-key cryptography. (2)

Encrypted Client Hello (ECH).

A TLS extension that encrypts the server name and other sensitive handshake fields. (11)

Entropy.

A measure of unpredictability, in bits. (3)

Envelope encryption.

Encrypting data with a DEK and wrapping the DEK under a KEK held in a key management service. (20)

FIPS 140-3.

The US and Canadian standard for validating cryptographic modules, with four security levels. (23)

Forward secrecy.

The property that stealing a long-term key later does not expose past sessions, achieved with ephemeral key exchange. (8)

Hash function.

A function producing a fixed-size fingerprint of any input, resistant to preimages and collisions. (6)

HKDF.

HMAC-based key derivation function, used to derive multiple keys from one strong secret. (6)

HMAC.

A message authentication code built from a hash function. (6)

HSM.

Hardware security module: a tamper-resistant device that generates, stores and uses keys without exposing them. (23)

Hybrid (key exchange or signature).

Combining a classical and a post-quantum algorithm so security holds if either survives. (26)

IND-CCA2, IND-CPA.

Formal security definitions for encryption: indistinguishability under chosen-ciphertext or chosen-plaintext attack. (10)

Intermediate CA.

A CA certificate signed by a root, used for day-to-day issuance. (15)

JWT.

JSON Web Token: a signed (or encrypted) token carrying claims, used in OAuth and OpenID Connect. (14)

KEM.

Key encapsulation mechanism: a public-key method where one party encapsulates a fresh shared key to another's public key. (8, 26)

Key ceremony.

A scripted, witnessed and recorded procedure for generating or using high-value keys such as root CA keys. (18)

KMIP.

Key Management Interoperability Protocol, a standard for clients such as storage arrays to manage keys in a key manager. (21)

KMS.

Key management service: a system that stores keys and performs operations with them on request, under access policies and audit. (21)

Lattice.

A regular grid of points in many dimensions; hard lattice problems underpin ML-KEM and ML-DSA. (2, 26)

LMS and XMSS.

Stateful hash-based signature schemes used for firmware signing. (26)

MAC.

Message authentication code: a keyed tag proving integrity and origin between parties sharing a key. (6)

ML-DSA.

Module-Lattice-Based Digital Signature Algorithm, FIPS 204. (26)

ML-KEM.

Module-Lattice-Based Key-Encapsulation Mechanism, FIPS 203. (26)

Mode of operation.

A method for using a block cipher on data of any length, such as CBC, CTR or GCM. (5)

mTLS.

Mutual TLS, where both client and server authenticate with certificates. (11)

Nonce.

A number used once: a value that must never repeat under the same key. (3, 5)

OCSP.

Online Certificate Status Protocol, for asking a CA whether one certificate is revoked; being retired on the public web. (17)

Passkey.

A phishing-resistant login credential based on a per-site key pair stored on the user's devices. (14)

PKCS#11.

The standard programming interface to HSMs and cryptographic tokens. (21, 23)

PKI.

Public key infrastructure: the CAs, certificates, policies and processes that bind keys to identities. (15 to 19)

Plaintext.

Unencrypted data. (1)

Post-quantum cryptography (PQC).

Public-key algorithms believed secure against quantum computers, running on ordinary computers. (25, 26)

Private key and public key.

The secret and shareable halves of an asymmetric key pair. (1)

Revocation.

Declaring a certificate untrustworthy before its expiry date. (17)

Root CA.

A self-signed CA certificate trusted because it is present in a trust store. (15)

RSA.

A public-key algorithm based on the difficulty of factoring, used for signatures and encryption. (7)

SAN.

Subject Alternative Name: the certificate extension listing the names a certificate is valid for. (15)

Secret.

Any value granting access if disclosed: passwords, API keys, tokens, private keys. (22)

Shor's and Grover's algorithms.

Quantum algorithms that respectively break RSA and elliptic curve cryptography, and modestly speed up brute-force search. (25)

Side-channel attack.

An attack that learns secrets from timing, power use, cache behaviour or other physical effects of computation. (4, 9)

SLH-DSA.

Stateless Hash-Based Digital Signature Algorithm, FIPS 205. (26)

SPIFFE.

A standard for workload identity, issuing short-lived certificates or tokens to workloads after attestation. (22)

Symmetric cryptography.

Cryptography where the same secret key is used by both parties. (1)

Threat model.

A description of what is protected, from whom, with what capabilities, and for how long. (1, 24)

TLS.

Transport Layer Security, the protocol that secures most internet connections. (11, 12)

TPM.

Trusted Platform Module: a security chip in computers providing protected keys, measured boot and attestation. (23)

Trust store.

The set of root certificates a client trusts. (15)

X.509.

The standard format for public-key certificates. (15)

X25519.

Diffie-Hellman key exchange on Curve25519, the default classical key exchange in modern protocols. (8)

Zero-knowledge proof.

A proof that a statement is true which reveals nothing else. (28)

B



Standards, Further Reading and Practice

The primary sources behind this book, grouped by topic, plus the best books, courses and practice resources for going deeper. Standards change; check each body's website for the current version.

IN THIS CHAPTER

- | | |
|-----|-------------------------|
| B.1 | Core standards by topic |
| B.2 | Books |
| B.3 | Courses and practice |
| B.4 | Staying current |

B.1 Core standards by topic

Topic	Key documents
Algorithms (symmetric)	FIPS 197 (AES); SP 800-38A to 38G (modes, including GCM in 38D); RFC 8439 (ChaCha20-Poly1305); RFC 8452 (AES-GCM-SIV)
Hashes, MACs, KDFs	FIPS 180-4 (SHA-2); FIPS 202 (SHA-3); FIPS 198-1 and RFC 2104 (HMAC); RFC 5869 (HKDF); RFC 9106 (Argon2); SP 800-132 (PBKDF2)
Public-key algorithms	FIPS 186-5 (signatures, including EdDSA); RFC 8017 (PKCS#1, RSA); RFC 7748 (X25519); RFC 8032 (Ed25519); RFC 6979 (deterministic ECDSA); SP 800-186 (elliptic curves); RFC 9180 (HPKE)
Random numbers	SP 800-90A, 90B, 90C
Key management	SP 800-57 Parts 1 to 3; SP 800-130 and 800-152 (key management systems); SP 800-131A (algorithm transitions)
TLS and protocols	RFC 8446 (TLS 1.3); RFC 8996 (deprecating TLS 1.0 and 1.1); RFC 9849 (Encrypted Client Hello); RFC 7296 (IKEv2); RFC 9420 (MLS)
Certificates and PKI	RFC 5280 (X.509 profile); RFC 6960 (OCSP); RFC 6962 (Certificate Transparency); RFC 8659 (CAA); RFC 3647 (CP/CPS framework); CA/Browser Forum Baseline Requirements
Automation	RFC 8555 (ACME); RFC 9773 (ACME Renewal Information); RFC 7030 (EST)
Identity	RFC 6749 (OAuth 2.0); RFC 7636 (PKCE); RFC 7519 (JWT); RFC 8725 (JWT best practices); RFC 9449 (DPoP); OpenID Connect Core; W3C WebAuthn
Hardware	FIPS 140-3 and ISO/IEC 19790; OASIS PKCS#11 v3; OASIS KMIP; TCG TPM 2.0
Post-quantum	FIPS 203 (ML-KEM); FIPS 204 (ML-DSA); FIPS 205 (SLH-DSA); draft FIPS 206 (FN-DSA); SP 800-208 (LMS, XMSS); SP 800-227 (KEM guidance); NIST IR 8547 (transition, draft); NSA CNSA 2.0
Inventories	CycloneDX 1.6 and later (CBOM)

B.2 Books

- **Serious Cryptography**, Jean-Philippe Aumasson (2nd edition, 2024). A practical, readable tour of modern cryptography and its pitfalls.
- **Real-World Cryptography**, David Wong (2021). Protocols and applied systems with clear diagrams.
- **A Graduate Course in Applied Cryptography**, Dan Boneh and Victor Shoup. Free online; the rigorous reference for definitions and proofs.

- **Introduction to Modern Cryptography**, Jonathan Katz and Yehuda Lindell. The standard university text on security definitions.
- **Cryptography Engineering**, Niels Ferguson, Bruce Schneier and Tadayoshi Kohno. How to build systems, and why they fail.
- **Bulletproof TLS and PKI**, Ivan Ristić (2nd edition). The practitioner's reference for deploying TLS and certificates.
- **Crypto 101**, Laurens Van Houtven. Free; a gentle introduction.

B.3 Courses and practice

- **Cryptopals** challenges: eight sets of hands-on attacks, from repeated-key XOR to padding oracles and ECDSA nonce reuse.
- **CryptoHack**: browser-based challenges with a strong mathematics track.
- **Dan Boneh's Cryptography I** (online course): the classic introduction to the theory.
- **Real World Crypto** conference talks (free recordings): where industry and research meet each year.
- **Open Quantum Safe** project: libraries and demos for experimenting with post-quantum algorithms.

B.4 Staying current

Source	What it tells you
IACR ePrint archive	New research and attacks, often months before publication
NIST CSRC (Computer Security Resource Center)	Draft and final standards, comment periods, post-quantum updates
CA/Browser Forum ballots and minutes	Changes to public certificate rules
Root program announcements (Chrome, Mozilla, Apple, Microsoft)	Distrusts, new requirements, policy timelines
IETF working groups (TLS, LAMPS, ACME, CFRG, PQUIP)	Protocol and format standards in progress
National agencies (NCSC, BSI, ANSSI, CISA, NSA)	Guidance and migration deadlines
Let's Encrypt blog and community	Practical changes to the public PKI

C



Command Reference

The commands used most often in this book's labs and in daily practice, grouped by task. Commands use OpenSSL 3.5 or later unless stated. Replace file names and hosts with your own.

IN THIS CHAPTER

- | | |
|------------|---|
| C.1 | Inspecting certificates and connections |
| C.2 | Keys and requests |
| C.3 | Formats |
| C.4 | Encryption, hashing and signing |
| C.5 | SSH |
| C.6 | DNS and the web PKI |

C.1 Inspecting certificates and connections

Task	Command
Show a server's certificate chain	<code>openssl s_client -connect host:443 -servername host -showcerts</code> (press Ctrl+D to close)
Read a certificate	<code>openssl x509 -in cert.pem -noout -text</code>
Show expiry dates	<code>openssl x509 -in cert.pem -noout -dates</code>
Show names (SAN)	<code>openssl x509 -in cert.pem -noout -ext subjectAltName</code>
Fingerprint	<code>openssl x509 -in cert.pem -noout -fingerprint -sha256</code>
Verify a chain	<code>openssl verify -CAfile root.pem -untrusted intermediate.pem leaf.pem</code>
Check key and certificate match	Compare <code>openssl x509 -in cert.pem -noout -pubkey</code> with <code>openssl pkey -in key.pem -pubout</code> ; they must be identical
Test a specific TLS version	<code>openssl s_client -connect host:443 -tls1_2</code> or <code>-tls1_3</code>
Request post-quantum key exchange	<code>openssl s_client -connect host:443 -groups X25519MLKEM768</code>
Scan a server	<code>testssl.sh https://host</code>

C.2 Keys and requests

Task	Command
EC P-256 key	<code>openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out key.pem</code>
Ed25519 key	<code>openssl genpkey -algorithm ED25519 -out key.pem</code>
RSA 3072 key	<code>openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:3072 -out key.pem</code>
ML-DSA-65 key	<code>openssl genpkey -algorithm ML-DSA-65 -out key.pem</code>
ML-KEM-768 key	<code>openssl genpkey -algorithm ML-KEM-768 -out key.pem</code>
Public key from private	<code>openssl pkey -in key.pem -pubout -out pub.pem</code>
CSR with SANs	<code>openssl req -new -key key.pem -out req.csr -subj "/CN=host" -addext "subjectAltName=DNS:host"</code>
Read a CSR	<code>openssl req -in req.csr -noout -text -verify</code>

Task	Command
Self-signed certificate	<code>openssl req -x509 -new -key key.pem -days 30 -subj "/CN=test" -out cert.pem</code>

C.3 Formats

Task	Command
PEM to DER	<code>openssl x509 -in cert.pem -outform DER -out cert.der</code>
DER to PEM	<code>openssl x509 -in cert.der -inform DER -out cert.pem</code>
Create PKCS#12	<code>openssl pkcs12 -export -inkey key.pem -in cert.pem -certfile chain.pem -out bundle.p12</code>
Extract from PKCS#12	<code>openssl pkcs12 -in bundle.p12 -nodes -out all.pem</code>
Read a CRL	<code>openssl crl -in ca.crl -inform DER -noout -text</code>

C.4 Encryption, hashing and signing

Task	Command
SHA-256 of a file	<code>openssl dgst -sha256 file</code>
HMAC-SHA256	<code>openssl dgst -sha256 -hmac "secret" file</code>
Random bytes	<code>openssl rand -hex 32</code>
Sign (Ed25519 or ML-DSA)	<code>openssl pkeyutl -sign -inkey key.pem -rawin -in file -out file.sig</code>
Verify	<code>openssl pkeyutl -verify -inkey pub.pem -pubin -rawin -in file -sigfile file.sig</code>
Sign with RSA-PSS	<code>openssl dgst -sha256 -sigopt rsa_padding_mode:pss -sign key.pem -out sig file</code>
Benchmark	<code>openssl speed -evp aes-256-gcm</code>

C.5 SSH

Task	Command
Ed25519 user key	<code>ssh-keygen -t ed25519 -f id_ed25519</code>
Create a CA	<code>ssh-keygen -t ed25519 -f user_ca</code>
Sign a user certificate for 8 hours	<code>ssh-keygen -s user_ca -I alice -n alice -V +8h id_ed25519.pub</code>
Inspect a certificate	<code>ssh-keygen -L -f id_ed25519-cert.pub</code>
See negotiated key exchange	<code>ssh -v host</code> , then look for the line starting "kex: algorithm"

C.6 DNS and the web PKI

Task	Command
CAA records	<code>dig +short CAA example.com</code>
HTTPS record (ECH config)	<code>dig +short HTTPS example.com</code>
Certificates for a domain in CT	Search https://crt.sh/?q=example.com
HSTS header	<code>curl -sI https://example.com</code> , then look for Strict-Transport-Security

Trust at Scale

Cryptography protects every login, payment, software update and private message, yet most engineers learn it in fragments. This book connects the fragments. It starts from the mathematics, builds through symmetric and public-key cryptography and real protocols, and arrives at the work that keeps organisations running: certificates and PKI, key and secret management, hardware security modules, and the migration to post-quantum cryptography already under way.

Every term is defined in plain words. Every chapter ends with labs, real failure stories and review questions, so you can turn understanding into practice.

Bolaji Akinyele